Dr Milan Milanovic

Laws of Software Engineering

TechWorld WithMilan
simplifying complex topics

Laws of Software Engineering

OceanofPDF.com

Laws of Software Engineering by Dr Milan Milanovic

Copyright © 2026 Milan Milanovic. All rights reserved.

Published by First Principles Consultancy, PR.

Edited by Lana Todorovic

No part of this book may be reproduced or used in any form without the written permission of the copyright owner or publisher, except for the use of brief quotations in a book review. For more info, please contact: milan@milanovic.org.

Tech World With Milan logo is a licensed trademark of First Principles Consultancy, PR.

First edition, April 2026

lawsofsoftwareengineering.com

OceanofPDF.com

Praise for Laws of Software Engineering

Laws of Software Engineering is an invaluable book that should be on the shelf of any technical leader, architect, or aspiring senior developer.

—*Adam Tornhill*
Author of Your Code as a Crime Scene, CTO at CodeScene

A delightfully compact book, infused with hard-earned lessons of wisdom. You can either spend decades discovering these laws through the scars of experience or allow Milan to unlock their secrets in a single weekend.

—*Maarten Dalmijn*
Author of Driving Value with Sprint Goals

Milan has done the community an invaluable service by compiling these timeless laws into one accessible resource. What sets this book apart isn't just its comprehensiveness but that each law is presented with the clarity and context needed to actually apply it. I wish I'd had this compilation 20 years ago. It would have saved me from learning some of these lessons the hard way.

—*Matthias Patzak*
CTO, Author of "All Hands on Tech," Executive in Residence at AWS

Domain-driven design teaches us that the most effective way to tackle complex systems is to work with multiple models, each fine-tuned to address a specific problem. This book takes the same approach at a higher level of abstraction: It provides a catalog of effective thinking models for system design, organizational dynamics, project management, and other aspects of software engineering. *Laws of Software Engineering* won't make decisions for you but will provide the clarity to make them deliberately.

—*Vlad Khononov*
Author of Balancing Coupling in Software Design
and Learning Domain-Driven Design

I love a great reference book, and this is one! A single place to learn and remind yourself about the laws and patterns that help us steer software projects toward success. What makes this collection truly great is the inclusion of all the "socio" patterns, as well as the technical ones. The humans who use and create software systems are the most important part. This book will become a well-thumbed reference on my shelf.

—*Jacqui Read*

Lead Technical Architect, Author of Communication Patterns: A Guide for Developers and Architects

I've been collecting mental models from tech for 20 years—Conway's Law here, Hyrum's Law from a Google talk, and Tesler's Law from the guy at Xerox. They lived in different corners of my brain, never talking to each other. Milan put them in the same room. What struck me wasn't learning new laws—I'd heard most of them. It was seeing them together. That kind of clarity only comes when someone does the hard work of finding the original sources, tracing the connections, and putting them in a sequence that actually flows.

If you've been in tech long enough to learn these lessons the hard way—through failed projects and late-night debugging sessions—this book will feel like someone finally organized your messy notes. Useful whether you're a developer, a manager, or a co-founder. The laws don't care about your title.

—Thiago Ghisi
Director of Engineering, ex-Nubank, Apple & American Express

The author has produced something genuinely useful: a Rosetta Stone for the shared vocabulary of experienced software engineering. The book's greatest strength is its curation. Assembling laws that span systems architecture, organisational dynamics, cognitive biases, and coding principles under one roof, and organising them in a way that mirrors how real problems actually emerge in engineering work. As a "shared language" resource to hand to your team—especially one with mixed experience levels —it's hard to beat.

—Alex Rashkov
Engineering Leader, Meta

This book is both a fun-read cover-to-cover and a useful piece of reference material for professionals looking for guidance in their day-to-day work.

—Sean Goedecke
Staff Software Engineer, GitHub

Milan's book is one of those rare "long time no see" reads. It took me back to my early days as a programmer, when I read every book I could find, not just to learn concepts, but to discover guidance grounded in real-world experience.

—Marvin Ferreira
Engineering Manager, Uber

This book offers a carefully structured and thoughtfully categorized introduction to core software engineering concepts, enriched with interesting references to established laws.

—*Luca Mezzalira*
Principal Solutions Architect, Amazon

There's a career's worth of wisdom here. Milan is an experienced engineer who has encountered these patterns many times over, and it shows. If you want a practical reference for the forces that shape software systems and teams, this is it.

—*Owain Lewis*
Director of Software Engineering, Oracle

Laws of Software Engineering does a great job of turning a broad set of familiar ideas into something practical and genuinely useful. What stood out to me is that it does not treat these laws as rigid rules, but as context-dependent mental models you can use when real engineering trade-offs show up, whether that is architecture, delivery, quality, or team design. It feels less like a book you read once and more like a reference you come back to when you are making decisions in the real world.

—*Ryan Murphy*
Engineering Manager, Yelp

I enjoyed reading this book, because it distills decades of software engineering reality into a compact, structured reference. It is a practical catalog of laws and principles you can return to when systems grow, coordination costs rise, and trade-offs become unavoidable.

—*Romano Roth*
Global Chief of Cybernetic Transformation, Zühlke Group

As AI transforms the very nature of software engineering, this timely collection of enduring principles gives practitioners both the theoretical grounding and the practical toolkit they need to understand their systems and navigate what comes next.

—*Annie Vella*
Distinguished Engineer at Westpac New Zealand

Product teams inevitably fall into traps. Work becomes more complex than necessary, progress slows, and delivering real value starts to feel impossible. While proven practices exist to get teams unstuck, they are scattered and hard to apply in the moment. This book brings them together. It is a practical companion that helps teams break free from the maze and operate as true value maximisers.

—*David Pereira*

Product Leader, Author of Untrapping Product Teams

This book is an inventory of the most frequently referenced principles we now consider common knowledge. Many of these ideas are so deeply embedded in our industry's lingo that we even forgot their origins.

Reading the book felt like touring an almanac of software engineering history. It will help any tech leader formulate a useful abstraction of their viewpoint, while also giving beginners a roadmap to the set of assumptions that define our work today.

—*Jovan Cukalovic*
Senior Director of Engineering at Nutanix

What makes this book good is that it captures those patterns clearly and without fluff. It's easy to read, easy to understand, and immediately applicable to real work. An absolute must-read for any tech professional, whether a manager or an engineer!

—*Fiodar Sazanavets*
Senior AI Engineer, ex-Microsoft

High-performing teams are defined by a culture of shared practices. For those in an early-growth phase, this book bridges that gap perfectly. It isn't just a manual on syntax, but a strategic compass for new tech leads and engineering managers. If you approach it with an open mind and consult it when things get sticky, you'll leverage a massive reservoir of industry experience to navigate your team toward success.

—*Uros Popovic*
Staff Software Engineer

This book brings theory-heavy laws into practical context, helping readers understand the reasoning behind the decisions made when building software and teams. The result is a grounded, credible perspective that resonates with engineers and leaders who operate under real constraints and trade-offs.

—*Tosho Trajanov*
CTO & Co-founder, Adeva

This book cuts through architectural theory to focus on laws that describe how systems actually behave. Milan presents them clearly, without unnecessary complexity. Having built dozens of systems over the last 15 years, I immediately recognized the value of this book: It belongs on your desk, ready for reference, not gathering dust on a shelf.

—*Maciej Jędrzejewski*

Software Architect, Author of Master Software Architecture

OceanofPDF.com

For my daughter, Sofija.

OceanofPDF.com

Table of Contents

Foreword by Dr Rebecca Parsons

Foreword by Addy Osmani

Preface

Part I System and Architecture

1. Gall's Law
2. The Law of Leaky Abstractions
3. Tesler's Law (Conservation of Complexity)
4. The CAP Theorem
5. Hyrum's Law
6. The Second-System Effect
7. Fallacies of Distributed Computing
8. The Law of Unintended Consequences
9. Zawinski's Law

Part II People, Teams, and Organizations

10. Conway's Law
11. Brooks's Law
12. Dunbar's Number
13. The Ringelmann Effect
14. Price's Law
15. Putt's Law
16. The Peter Principle
17. The Bus Factor
18. The Dilbert Principle

Part III Time, Estimation, and Planning

19. Hofstadter's Law
20. Parkinson's Law
21. The Ninety-Ninety Rule
22. Goodhart's Law
23. Gilb's Law
24. Knuth's Optimization Principle

Part IV Quality, Maintenance, and Evolution

25. Murphy's/Sod's Law
26. Postel's Law (The Robustness Principle)
27. The Broken Windows Theory
28. The Boy Scout Rule
29. Technical Debt
30. Linus's Law
31. Kernighan's Law
32. The Testing Pyramid
33. The Pesticide Paradox
34. Lehman's Laws of Software Evolution
35. Sturgeon's Law

Part V Scale, Performance, and Growth

36. Amdahl's Law
37. Gustafson's Law
38. Metcalfe's Law

Part VI Coding and Design Principles

39. DRY (Don't Repeat Yourself)
40. KISS (Keep It Simple, Stupid)
41. YAGNI (You Aren't Gonna Need It)
42. The SOLID Principles
43. The Law of Demeter
44. The Principle of Least Astonishment

Part VII Decision-Making and Cognitive Biases

45. The Dunning–Kruger Effect
46. Hanlon's Razor
47. Occam's Razor
48. The Sunk Cost Fallacy
49. The Map Is Not the Territory
50. Confirmation Bias
51. The Hype Cycle & Amara's Law
52. The Lindy Effect

53. First-Principles Thinking
54. Inversion
55. The Pareto Principle (80/20 Rule)
56. Cunningham's Law

Part VIII Conclusion

1. When the Laws Collide
2. Using the Laws Together
3. Why These Laws Last

Acknowledgments

OceanofPDF.com

Foreword by Dr Rebecca Parsons

As software professionals, we often learn from our mistakes. We might have decided to move to a microservices architecture too early. We might have been the victim of an organization who thought that they could just throw more people at a project to either get it done faster or to allow for more features. However, if the only way we learn these lessons is through our own mistakes, our knowledge is necessarily limited to our experiences. The book *Laws of Software Engineering* by Milan Milanovic allows us to learn from other people's mistakes. Indeed, Milan states that his rationale for doing the research to write the book stemmed from his desire to learn from others' mistakes rather than his own.

The laws in the book are not really laws. Some have proofs attached to them, like the CAP Theorem. Others just seem to hold up, even if we don't actually understand why. Some just make sense. Others can feel counter-intuitive at first sight. However, ignoring these laws should not be done lightly. These laws have come into usage simply because they tend to be true, or at least seem true.

To be useful, these laws should influence the decisions we make about our code, our architectures, our teams, and our plans. The book facilitates this by giving real examples of how the laws manifest themselves. These examples allow us to see the impacts of the laws' consequences and thus make better decisions.

As new tools and technologies arrive on the scene, new laws may be discovered. However, these laws also still apply, but perhaps we

must interpret them differently, or view them from a different perspective. Having these laws collected in one place, rather than scattered throughout the world wide web or in our brains, allows us to re-examine these laws in light of advances such as AI-assisted software development, for example.

Enjoy learning not only about the laws themselves, but about their origins and their consequences as well. Doing so will allow you to leverage the mistakes others have made, and hopefully, therefore, allow you to avoid mistakes.

—Dr Rebecca Parsons

OceanofPDF.com

Foreword by Addy Osmani

I still remember the first time I truly felt the weight of Brooks's Law. I was mid-career at Google, watching a project spiral as well-meaning leadership added engineers to a schedule that was already slipping. The newcomers needed onboarding. The existing team got pulled into context-switching. The deadline moved further away. Nobody was surprised—not really. We'd all felt this force before. We just hadn't named it cleanly, or known where it fit in the broader landscape of forces that shape software.

That's the quiet frustration this book resolves. These laws have been operating on our work for decades. They've been responsible for late nights, failed rewrites, bloated codebases, and teams that somehow produce less the larger they grow. But they've been scattered—tucked into conference talks, footnotes, and internal wikis. Dr Milan Milanovic has done something genuinely valuable here: He's gathered them into one place and made them legible.

Over 14 years at Google, I worked to make it easier to build on the web to serve billions of users. The problems changed. The laws didn't. Gall's Law played out every time a team decided to rewrite from scratch rather than evolve. Hyrum's Law was quietly at work every time a Chrome DevTools API change broke some obscure extension that had come to depend on an undocumented behavior. Tesler's Law of Conservation of Complexity shaped every product decision about what the developer should configure versus what the tool should just handle. These weren't abstractions. They were the texture of daily engineering.

The Law of Leaky Abstractions is one I've returned to constantly, especially as AI-assisted development has matured. When developers use an AI coding tool, they're working at a high level of abstraction. It feels frictionless until it doesn't. The moment a generated function interacts with a subtle runtime behavior, or a framework assumption fails at the edge case, the abstraction leaks. You still need to understand what's underneath. The law hasn't changed; the layers have just multiplied.

What I appreciate most about Milan's framing is intellectual honesty. He doesn't oversell these as laws in the scientific sense. They're patterns—empirical observations that have proven durable across many contexts. Some have solid theoretical backing. Others are more heuristic. All of them are tools for reasoning faster under uncertainty, which is most of what engineering leadership actually is.

We're in an era where AI is reshaping what it means to write software. The velocity of production has changed. The cost of generating code has plummeted. And yet, Zawinski's Law still holds. Conway's Law still holds. The Second-System Effect will still find teams that believe AI makes the second rewrite different from all the ones before it. The laws in this book describe forces that operate at the level of human cognition, team dynamics, and system complexity. Those haven't been automated away.

If anything, the shift toward AI-assisted development makes this book more relevant, not less. As more people who aren't career engineers build software, they'll encounter these forces without the scar tissue to name them. This book gives them vocabulary. And

vocabulary matters: You can’t defend against a force you can’t describe.

Use this book the way Milan suggests—as a reference, a lookup table, a map of the terrain. Skim it once to know what’s available. Return to it when you feel a familiar gravity pulling a project in a direction you can’t quite articulate. You’ll find the law that names it. And once you can name it, you can decide what to do about it.

—Addy Osmani

Director, Google Cloud AI

addyosmani.com

OceanofPDF.com

Preface

You've already learned most of these laws, but not from the book. You learned them through a project that failed, perhaps a deadline that was not realistic, or a system that could not scale.

That's how most engineers learn. A failed rewrite teaches you that a complex system that works always evolved from a simple system that worked. A missed deadline teaches you that adding people to a late project makes it later. A production incident at 2 am teaches you that if something can go wrong, it will. The knowledge is real, but the tuition is expensive.

These patterns are knowledge that people have learned from making software for a long time, like 40 or 50 years. Some of these ideas have been around since the 1960s, and some are more recent. Each law in the book describes the forces that software developers have already felt in reality, even if they did not know their name, and that other people experienced. These patterns explain why estimates are always wrong, why that one simple change broke three other things, and why the architecture we defined didn't hold up in production.

Every company I worked for has some of these rules written down somewhere: an internal wiki, an onboarding doc, or a Confluence page. Every senior engineer knows them, but people who are coming to work in this field do not. So they learn them the hard way. This book skips the hard way.

I spent 20 years recognizing these patterns before I started collecting them or hearing about them from other people. At some

point, writing them down became more useful than finding they exist at the next company, with a different team making the same mistakes.

This book brings together 63 of those patterns into one place. The structure follows how problems actually compound in practice: Architecture decisions constrain people decisions, people decisions affect timelines, timelines drive quality trade-offs. That's why the book moves through Architecture, People, Time, Quality, Scale, Code, and Decision-Making, in that order.

One thing to keep in mind from the start: These are not laws in the scientific or legal sense. Some of these have been scientifically researched. Others are things that people have noticed and passed down over time. They are all guidelines. They work most of the time, but not always. So, use them as a way of looking at things, not as definitive answers.

Each law is described in the following way: what the law says, where the law came from, why the law is important, and what the law looks like when you experience it in real life. The law is not going to be in this book if it cannot assist you in making a decision or preventing you from making a mistake that you would probably make anyway. The law has to be helpful, so if it is not going to help you make a decision or stop you from making a mistake, then it is not included in this book.

These laws are tools, not rules. When your team is arguing microservices versus monolith, Gall's Law gives you the right frame. When stakeholders want to add three engineers to an already late

project, Brooks's Law explains exactly why that won't help. You'll still make decisions on your own, but these laws will make you do it faster, with better context, and without paying the same price multiple times.

You can read this book front to back or treat it as a reference. Most people will do both, first skim it once to see what's here, then come back when a specific problem hits. The "Related to" sections at the end of each chapter are there to help you understand how laws are connected to each other and how to move between them.

These patterns were true when Brooks wrote about them in 1975, and they showed up again when Zawinski watched Netscape bloat from a browser into a platform in 1995. And they're still visible today, in cloud migrations that have the same issues as 1970s mainframe projects, in distributed systems that fail on the same mistakes, but also in teams that continue to learn the same lessons about people, deadlines, and complexity.

The technology changes, but these patterns don't. They are timeless. This version captures what matters now, and future editions will expand as new ideas emerge.

That's why they're worth knowing before the next project starts.

OceanofPDF.com

Part I
System and Architecture

The laws in this category explain why structure, coupling, and hidden complexity show up even when you try to keep designs clean.

OceanofPDF.com

1. Gall's Law

A complex system that works is invariably found to have evolved from a simple system that worked.

Key Takeaways

- *Don't build a complex system from scratch. Instead, make a simple version that works, then iterate and add complexity. Reflect on your own projects: What is your simple system starting point? Consider this approach as a launching pad for experimentation and innovation.*
- *Successful large systems often grow organically. A fully designed-from-scratch complex system usually fails because you can't foresee all interactions.*
- *This law argues for creating a Minimum Viable Product (MVP), a simple working core that is to be developed further, rather than a big-bang approach.*
- *Systems that grow tend to adapt and handle complexity better because they've been tested and refined at each stage. An overly complex initial design can break when something unexpected happens.*

Overview

John Gall observed that *successful complex systems start as successful simple systems*. If you attempt to create a complex

system from the beginning, it usually doesn't work because there are too many unknowns you still haven't validated.

Instead, we start with a simple architecture and then learn from real-world usage. Then, you add features or complexity incrementally, ensuring each change still works.

When you start with a small, functional core, you can validate assumptions early (MVP approach). Every new capability that is added incrementally is tested against reality, which forces the system to adapt. Over time, complexity emerges in a controlled way, which is grounded in proven behavior rather than imagination.

When teams attempt to design a fully featured system from the start, they rely on imagined requirements, unknown interactions, and untested abstractions. These systems look impressive on paper but fail in practice because no one knows how they behave under real conditions.

Gall's Law also explains why rewrites often fail when the original system never had a simple, working foundation. If a system never had a simple, working foundation, its complexity rests on unstable ground.

Origins

John Gall, an American pediatrician with a sideline in systems theory, published *Systemantics: How Systems Work and Especially How They Fail* in 1975. Thirty publishers rejected the book before it eventually became a cult classic.

The third edition (2002) of the book is titled *The Systems Bible*.

Examples

Facebook started as a simple website for Harvard students, essentially a basic user profile system. It worked at that small scale, then expanded gradually (adding features, handling more users, evolving into what it is today). Had Mark Zuckerberg tried to build today's Facebook from scratch in 2004, it would almost certainly have collapsed under its own complexity.

A great example of this law is Instagram, which was originally a much more complex application called Burbn. It offered features such as check-ins, gaming, and sharing photos. The application was very messy, and it was not catching on, but the founders took a bold approach. They decided to simplify the application to its most basic, successful feature: sharing photos. This simple, working core was eventually expanded into the more complex social application we all now know today as Instagram.

Unix is another example. It started as a small, simple OS for a specific purpose, which then evolved to handle many tasks. Contrast that with super ambitious OS projects that tried to do everything from the start and failed (e.g., Apple Copland, IBM OS/2, and Windows Mobile).

Google Wave also failed to implement this law. Google tried to integrate many functionalities, such as forums, social network, messenger, and so forth. They wanted it all in one product. But what happened is that the product had no purpose. As a result, the project failed because it didn't meet any user expectations. In project

management, Gall's Law says, *"If your goal is a complex e-commerce platform, first build a simple storefront that works (maybe with one product category and a basic cart). Once that's working, expand."*

This law is especially important for startups and new businesses. Many startups pivot or refactor heavily because the initial simple product taught them what the real needs were. Those who try to launch with a massive feature set often collapse—they either never finish or launch a buggy mess.

In the era of microservices, a common piece of advice, influenced by Gall's Law, is as follows: *Don't start with microservices*. Start with a monolith (a simpler architecture) that works. As it grows, identify pieces to split off into separate services. Those pieces have the benefit of prior existence (you know their requirements from the monolith) and thus are more likely to work once isolated.

Gall's Law also suggests that if a complex system doesn't work, making it more complicated (adding more rules) likely won't fix it. It's better to simplify and get something working than grow.

For example, an overengineered piece of software with tons of features but poor core functionality often needs a rewrite that focuses only on the basics. When this is not done, competitors do so.

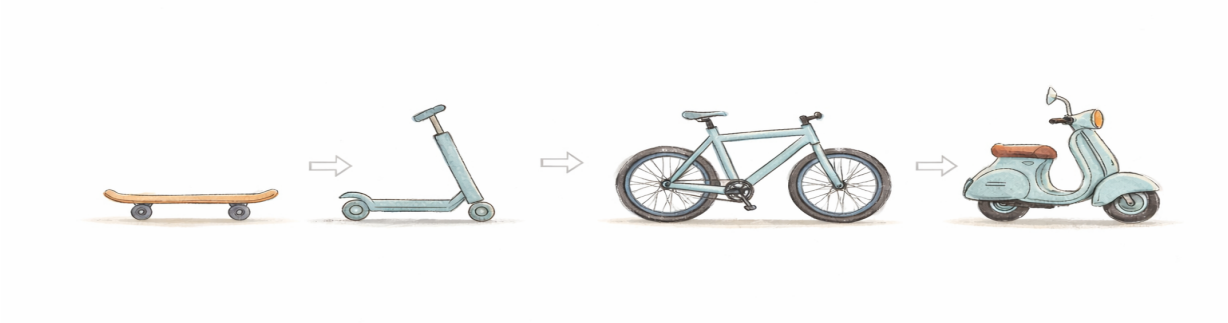

Figure 1.1 **Gall's Law**

Related to

- Conway's Law
- Brooks's Law
- The Law of Leaky Abstractions
- Hofstadter's Law

Further Reading

- Brooks, F. P., Jr. (1995). *The mythical man-month: Essays on software engineering* (Anniversary ed.). Addison-Wesley.
- Fowler, M. (2015). *Monolith first*. MartinFowler.com. https://martinfowler.com/bliki/MonolithFirst.html
- Gall, J. (2002). *The systems bible: The beginner's guide to systems large and small* (Rev. ed.). General Systemantics Press.
- Meadows, D. H. (2008). *Thinking in systems: A primer* (D. Wright, Ed.). Chelsea Green Publishing.

OceanofPDF.com

2. The Law of Leaky Abstractions

All non-trivial abstractions, to some degree, are leaky.

Key Takeaways

- *No matter how well-designed, abstractions (libraries, frameworks, etc.) have edge cases that depend on internal details.*
- *Developers should understand that using a high-level tool doesn't mean that they should not know what's happening underneath, at least at a basic level.*
- *"Leaky" means you might encounter performance issues, bugs, or behavior that force you to consider the underlying system (e.g., networking, OS) on which the abstraction sits.*
- *When creating abstractions, strive to minimize leakage and document the cases in which it might break.*

Overview

This law observes that in complex systems, the abstractions we create, meant to hide complexity, *inevitably fail in some scenarios*, revealing the underlying complexity to the programmer.

For example, consider an ORM (object-relational mapper) that lets you treat database entries as objects. It works most of the time, but occasionally, you may hit a case where performance is terrible, and

you have to think about the SQL queries being generated (because the abstraction leaks the database details).

The law isn't saying abstractions are bad. *They are essential.* However, it is reminding us that *one must be prepared for situations when they break.*

More experienced engineers often know that *"you can't completely abstract away X"* for things like network latency, concurrent programming, etc. Part of this law's implication is the importance of understanding what's beneath your abstractions, so you can handle issues when they pop up.

Origins

The concept has roots in Gregor Kiczales's work on abstraction models (1992), but Joel Spolsky coined the term in a 2002 blog post (https://www.joelonsoftware.com/2002/11/11/the-law-of-leaky-abstractions). He gave examples such as TCP (which abstracts reliable connections over IP, but on a bad network, the abstraction leaks, causing timeouts and requiring you to deal with them) or the `virtual memory` abstraction.

The concept resonates with many earlier thoughts in computing, essentially a restatement of "there's no free lunch" with abstractions.

Examples

A good example is *memory management in high-level languages.* We know that languages such as Java and Python abstract away manual memory allocation (thanks to garbage collection). Yet leaks

still happen (objects not being freed due to lingering references), or you encounter the abstraction's limits (such as the GC pause affecting performance). Suddenly, the developer needs to know about how garbage collection works internally and how the abstraction of "infinite memory" has leaked.

Another example can be seen in array traversal in C++. While appearing as a grid in code, in reality, your 2D array is a strip of memory laid out sequentially row by row. Summing row-wise, the CPU will nicely read in a block of sequentially laid out data, and the cache will be happy. Summing column-wise, the CPU will be jumping all over the memory, thrashing the cache and running 10 times slower in the process. Your code is right either way. It's just a waste of potential performance due to the abstraction, which doesn't actually reflect the way memory works. In fact, many similar libraries from the early days of the field hit this wall: matrix operations. For this reason, you had to fall back to loop optimizations and memory tricks again. The grid wasn't actually a grid, and the hardware didn't care about the abstraction.

Another example is sending *files over a network mount*. Suppose you treat a network file system file like a local file (that's an abstraction). Most of the time, it works, but if the network goes down, an operation on that file might hang or fail in ways local files wouldn't, thus exposing network errors.

Web developers often treat an HTTP request as something fast (abstracting away the network). However, when latency spikes or packets drop, the network's reality leaks into your application.

The law teaches us humility, too. As an engineer, I know that abstractions simplify things but do not *eliminate* underlying complexity or constraints.

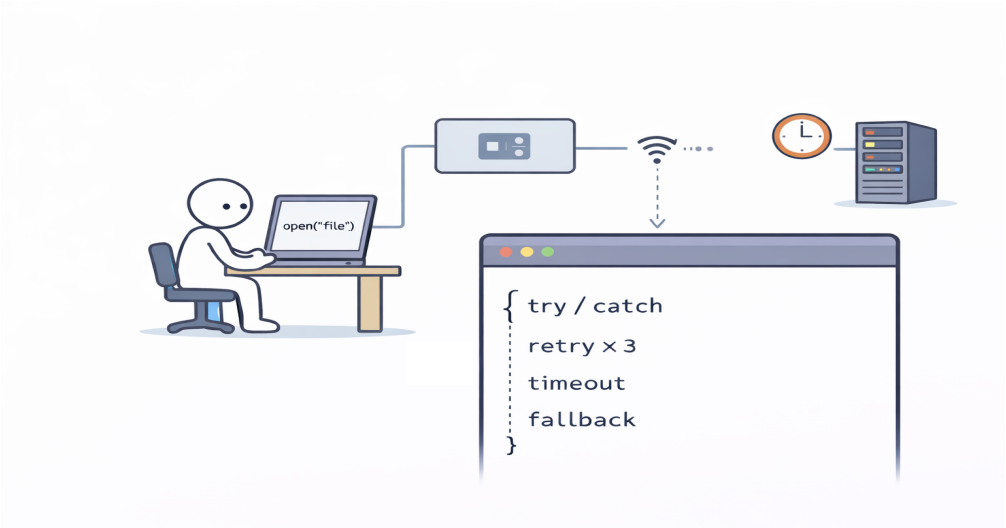

Figure 2.1 **Hidden network realities force extra code (Leaky Abstractions).**

Related to

- Hyrum's Law
- Amdahl's Law
- Gall's Law
- George Box's Law

Further Reading

- Kiczales, G. (1992). *Towards a new model of abstraction in the engineering of software. In IMSA '92*: Workshop on Reflection and Meta-level Architectures. Xerox Palo Alto Research Center.
- Spolsky, J. (2002). *The law of leaky abstractions*. Joel on Software. https://www.joelonsoftware.com/2002/11/11/the-

law-of-leaky-abstractions/
- Waldo, J., Wyant, G., Wollrath, A., & Kendall, S. (1994). *A note on distributed computing*. Sun Microsystems.

Sidenote: All Models Are Wrong (George Box's Law)

George Box said, "All models are wrong, but some are useful." Every abstraction you build, like a class hierarchy, an API contract, or a data model, is a lie. It's basically a simplified version of reality that ignores inconvenient details. The question isn't whether your model is wrong but whether it's bad in ways that matter.

Your database schema assumes users have one email address until they don't. Your payment model treats refunds like reversed payments until chargebacks arrive. Your user authentication treats sessions as binary until you need "remember me" functionality.

The model breaks because reality is messier than your abstraction. Accept that every model will leak, and design for the leak, not against it.

OceanofPDF.com

3. Tesler's Law (Conservation of Complexity)

Every application has an inherent amount of irreducible complexity. You can't eliminate it, you can only shift who handles it.

Key Takeaways

- *Good design often requires that developers absorb most of the complexity (through smart defaults, algorithms, etc.) so that the user's interaction is simpler.*
- *If your UI or API requires the user to manage many settings or steps, you've conserved complexity in the wrong place (on the user side). Often, it's better to design the product to manage that complexity internally.*
- *Good design often hides complexity without eliminating it by handling it internally.*

Overview

Tesler's Law, also known as the Law of Conservation of Complexity, states that for any system, there is a certain amount of complexity that cannot be removed: It's inherent to the problem. The key here is *who deals with it*.

If you design software that's very simple for the user, it likely means the software is doing a lot of complex work under the hood. If you

make the software very simple internally, it might dump complexity on the user (e.g., requiring many manual steps).

A classic example is scheduling meetings, which is inherently complex (finding a standard time). A tool like Calendly hides that complexity from the user (the software handles it), whereas an email chain to schedule a meeting places that complexity on the user to figure out.

Tesler's Law encourages designers to *move complexity away from the user experience* whenever possible. Sometimes that means more code, more sophisticated logic, or handling magic behind the scenes.

There's a limit, though. Some complexity might have to remain for the user if it's about their preferences or trade-offs they must choose. But then the design should at least guide them, not abandon them.

Origins

Larry Tesler, who worked on Apple Lisa and early GUI concepts, formulated this principle in the 1980s. He observed that you can't make everything simple without someone handling complexity. It's a fundamental trade-off in design.

Examples

Setting up a *simple router* can be done via a wizard (simple for the user, with the router firmware handling much of the configuration automatically with sensible defaults) or via an advanced settings

page (the user must configure IP, DNS, etc., with greater complexity). Many products offer both a simple mode and an advanced mode. Tesler's Law is at play here. The total complexity of networking hasn't changed, only who handles it.

Amazon's 1-Click purchase is another good example. The normal flow, cart review, shipping address, payment information, confirmation, and so on all generated problems for users, resulting in drop-offs. But Amazon chose to eat all that complexity on their side. They built the infrastructure to store payment methods, remember addresses, detect potential frauds, and manage inventory holds asynchronously. Thousands of engineering hours went into secure credential storage, session management, error handling, and related areas. The user gets to click one button. The complexity went to the backend logic: It's all Amazon's. Their users saved a minute per purchase because a team of engineers spent years building a system to carry that load for them.

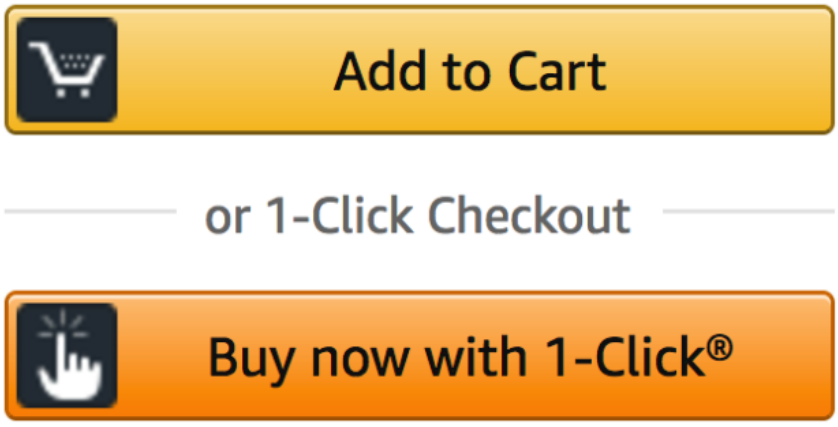

Figure 3.1 **Amazon 1-Click (Source: Wikipedia)**

In UI design, think of *form filling*: A smart form might pre-fill info, validate as you type, or even eliminate fields by deriving info (such as autodetecting location instead of asking for a city). That's the software taking on complexity (via APIs, logic). A naive form just lists 20 fields for the user, placing all the complexity on them (they must know what each field means, with no help).

The best designs find the right balance: for example, a user *wants* some control (like choosing preferences), but anything that can be defaulted or inferred should be.

When considering *programming frameworks*, a framework might hide many details (memory management, network calls), making developers' lives easier. The complexity is still there (the framework code is handling it behind the scenes). If the framework had not handled it, every developer using it would have to handle those details. For example, when you allocate memory in C, you need to free it by calling `free()`. In C# or Java, the garbage collector handles this. GC is doing complex work behind the scenes to track references and manage object generations. So, the complexity shifts from code to the runtime.

A negative example we all saw at least once is enterprise software that requires users to fill many fields and click through many options (complex for users and maybe simpler for devs because they didn't automate anything). A competitor might win by simplifying user flow, which meant they wrote more code to handle assumptions or provide defaults.

In *system architecture,* imagine designing a database vs. putting logic in application code. Sometimes pushing logic into the DB (such as stored procedures) can simplify app code but add complexity to the DB layer, or vice versa. Complexity isn't gone, just relocated.

Tesler's Law teaches us to ask who is better equipped to handle this complexity, the machine or the human? Usually, we try to let the machine take more of it. But there are cases where adding a bit of complexity for the user is acceptable for greater control.

In the end, the law reminds us that no design is without complexity. It's all about smartly allocating complexity to the right place.

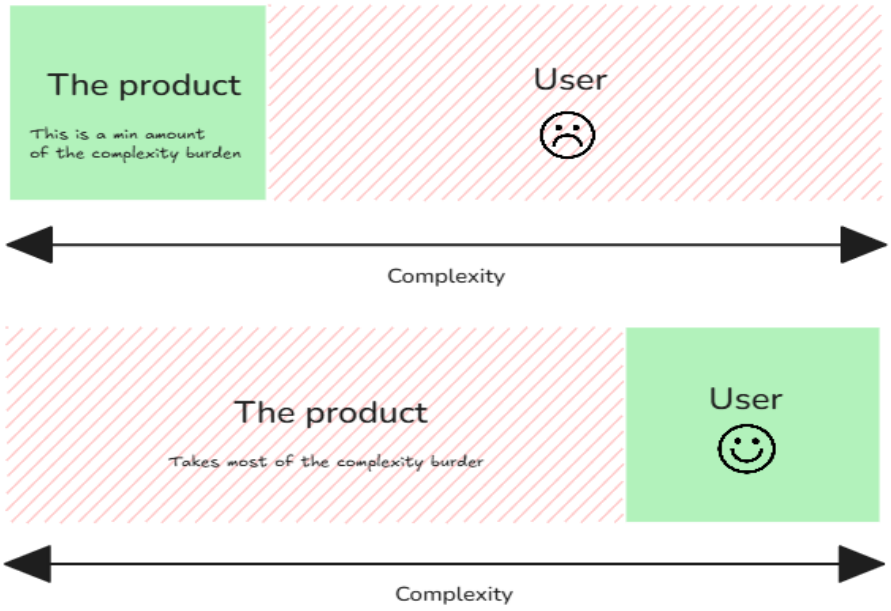

Figure 3.2 **Tesler's Law**

Related to

- Hyrum's Law
- Occam's Razor

Further Reading

- Calleia, M. (n.d.). *Explaining the law of conservation of complexity*. *Humanist*. https://humanist.co/blog/law-of-conservation-of-complexity/
- Colombo, G. (2016). *Simplicity is overrated*. *Marvel*. https://marvelapp.com/blog/simplicity-is-overrated/
- Farnam Street. (n.d.). *Why life can't be simpler*. https://fs.blog/why-life-cant-be-simpler/
- Law of conservation of complexity. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Law_of_conservation_of_complexity

OceanofPDF.com

4. The CAP Theorem

A distributed system can deliver on only two of three desired characteristics: Consistency, Availability, and Partition Tolerance.

Key Takeaways

- *The distributed system can meet at most two of three guarantees: Consistency, Availability, and Partition tolerance. In an ideal network, one could retain consistency as well as availability; however, in the case of failure (Partition), one would have to give up one of these properties.*
- *The theorem implies a trade-off: In the face of a network partition, a distributed system must either remain consistent (but potentially become unavailable to certain operations) or remain available (but accept that data may diverge). No distributed system can be both fully consistent and available in the presence of a partition.*
- *Distributed databases typically lean one way or the other. For example, MongoDB favors consistency: During a partition, it will block or refuse writes so replicas stay synchronized. Cassandra favors availability: It remains online and answering queries, even if replicas have slightly differing data until consistency finally emerges.*

Overview

The CAP Theorem is a fundamental concept in distributed systems and databases. It stands for Consistency, Availability, and Partition Tolerance, three properties a distributed system can have.

Yet, the theorem states that a distributed system can't simultaneously provide all three of these guarantees:

1. **Consistency:** This means that all nodes in the system see the same data simultaneously. If the data on one node changes, all other nodes will see that change immediately.
2. **Availability:** Every request to the system receives a response, but it is not guaranteed to contain the most recent write. The system continues to operate and provide responses even if part of it is down or some nodes are not communicating.
3. **Partition Tolerance:** The system continues to operate even if communication among the nodes is unreliable, and some messages may be lost or delayed.

The CAP Theorem is essential because it helps us understand the trade-offs in designing and using distributed systems. For example, if we create a system in which every read returns the most recent write (Consistency) and the system continues to operate despite network failures (Partition Tolerance), we may have to compromise availability.

Network partitions are standard in real-world applications, so systems must be partition tolerant. This means we usually must choose between consistency (CP) and availability (AP) when designing microservice architectures.

When network partitions are a reality, choosing availability might mean tolerating some data inconsistencies, while opting for consistency could reduce availability.

Origins

Eric Brewer created the theorem in 2000 (in the context of web services), and it was later formalized by Gilbert and Lynch in 2002. Based on large-scale systems at the time, Brewer observed that designers faced the following concerns: keeping data consistent across nodes, keeping the service up, and handling network unreliability.

The formal proof showed that in a distributed system with shared data, you *must sacrifice either consistency or availability when a network partition occurs*. CAP became a guiding principle in the NoSQL movement and distributed database design in the 2000s.

Examples

A good example is the *Domain Name System (DNS)*, designed to be an AP. DNS opts for availability and partition tolerance. If some name servers are partitioned, they still reply (availability), even if their information might be slightly outdated (not strictly consistent until zones sync up).

Yet, a traditional *relational database with replication* often chooses CP. In a network split, some database replicas might refuse writes or become read-only to maintain consistency, sacrificing availability to some clients until the partition is resolved. For example, MongoDB is

a CP (Consistent and Partition-Tolerant) database, and Cassandra is an AP (Available and Partition-Tolerant) database.

Apache Cassandra is a good example of the CAP theorem by design. Its creators chose to favor Availability and Partition tolerance over consistency. In its normal operating environment, Apache Cassandra is a distributed database that maintains copies of data across multiple nodes and data centers. However, when a network partition occurs, which prevents nodes from communicating with each other, Apache Cassandra must make the trade-off presented by the CAP theorem.

Let's say we have a Cassandra cluster with data centers distributed across multiple locations, and a partition between them. Assume we have an update at data center A, where we are updating a user's balance. Meanwhile, at data center B, we are making a read call for the same user's balance. The system will return the old value for data center B, as we are considering Availability over Consistency

Consistency levels are also available for Cassandra to trade off these two factors on a per-operation basis. The ALL consistency level will compromise on availability during a network partition to ensure data consistency, and the ONE consistency level will compromise on data consistency to ensure availability.

While it is not possible to achieve both strong consistency and strong availability during a network partition as per the CAP theorem, it is not an absolute dichotomy but exists along a spectrum.

However, as a design principle, the CAP Theorem has some serious flaws. Martin Kleppmann points out that the definitions used in the

theorem are ambiguous and contradictory. Availability in the Gilbert and Lynch definition is different from what is meant in practice. The proof can be considered valid only if we assume that network partitions can last forever, which renders linearizability and eventual consistency impossible. More helpful than the “choose two of three” approach is considering the sensitivity of delay: How does your system’s delay change as the network slows down? In linearizable systems, delay is linear, while in causally consistent systems, it is independent of network delay. This is a much more useful way to think about trade-offs than arguing about impossibility statements.

In addition, your choice of database shouldn’t be driven by the CAP Theorem. Consider what your needs are: latency requirements, failure modes you want to survive, or consistency requirements for your application.

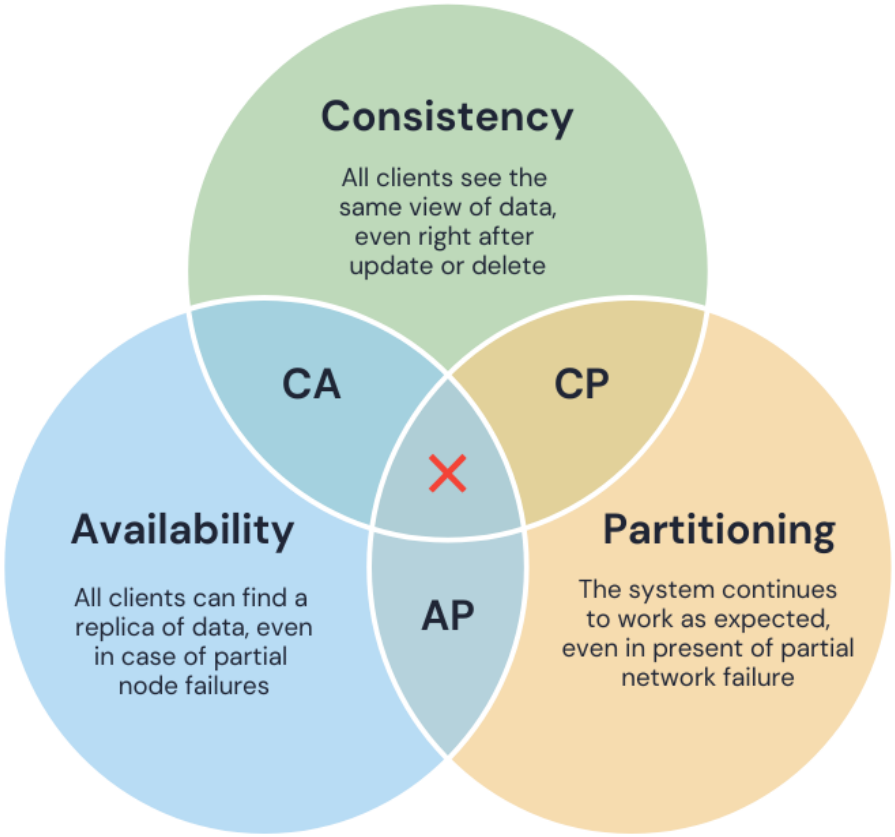

Figure 4.1 **The CAP Theorem**

Related to

- PACELC
- Fallacies of Distributed Computing

Further Reading

- Brewer, E. A. (2000). *Towards robust distributed systems* (abstract). Proceedings of the Annual ACM Symposium on Principles of Distributed Computing (PODC). ACM.
- Brewer, E. A. (2012). CAP twelve years later: How the “rules” have changed. Computer, 45(2), 23–29. IEEE.

- CAP theorem. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/CAP_theorem
- Corbett, J. C., Dean, J., Epstein, M., Fikes, A., Frost, C., Furman, J., Ghemawat, S., Gubarev, A., Heiser, C., Hochschild, P., Hsieh, W., Kanthak, S., Kogan, E., Li, H., Lloyd, A., Melnik, S., Mwaura, J., Nagle, D., Quinlan, S., Rao, R., Rolig, L., Saito, Y., Szymaniak, M., Taylor, C., Wang, R., & Woodford, D. (2012). *Spanner: Google's globally-distributed database. Proceedings of the 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI '12)*, 251–264.
- Gilbert, S., & Lynch, N. (2002). *Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services*. *ACM SIGACT News*, 33(2), 51–59.
- Kleppmann, M. (2015). *A Critique of the CAP Theorem*. University of Cambridge, Computer Laboratory. https://www.cl.cam.ac.uk/research/dtg/archived/files/publications/public/mk428/cap-critique.pdf
- Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.

OceanofPDF.com

5. Hyrum's Law

With a sufficient number of users of an API, it does not matter what you promise in the contract. All observable behaviors of your system will end up depending on somebody.

Key Takeaways

- *As the user count grows, everything your system does becomes a dependency. Even unintended side effects or bugs can become features that someone else's workflow depends on.*
- *Hyrum's Law warns maintainers that any change can break something for someone else. Consumers may have integrated your API in ways you didn't expect, including relying on timing, error messages, formatting, and so forth.*
- *The actual contract of your software isn't just the official API/spec but the exact behavior as observed in the wild. The contract can even be something informal, such as the UI your users are accustomed to.*

Overview

Hyrum's Law describes a phenomenon in which the boundary between a software's *documented interface* and its *implementation details* gets blurred in practice. Regardless of what you promise in an API contract, if people use your system enough, they will eventually depend on outputs and behaviors you never officially supported.

Over time, this means the *implementation becomes an interface*. Every observable trait, performance characteristic, specific error code, or memory layout might be assumed by someone.

The problem is that this behavior drastically limits the freedom to change the system. Even an internal change can break users who wrote code around the old behavior.

In large systems, such as operating systems or widely used libraries, maintaining backward compatibility requires extreme care.

Essentially, Hyrum's Law is why when you have 10,000 users, every strange thing your code does will become someone's dependency.

Origins

The law is named after Hyrum Wright, a software engineer who created this observation while working at Google around 2011–2012. Hyrum noticed that even changes to core Google libraries that should have been invisible would often break some project somewhere, because that project relied on an undocumented behavior.

The term "Hyrum's Law" was coined by his colleague Titus Winters in *Software Engineering at Google* (2020, O'Reilly Media).

Examples

In Google's older JSON APIs, responses were not obligated to preserve key order, but many implementations sorted alphabetically by default. This created problems for clients that relied on that assumption, especially once optimizations began randomizing the order for performance. Examples of this show up in integrations such

as analytics dashboards, where scripts assume positional access. It may also be found in timing assumptions for async calls, where users assume responses within a sub-second timeframe.

A lot of programs began relying on behaviors that Windows happened to exhibit, even though those behaviors were not supposed to happen. These things were not written down anywhere, so the people who made these programs just had to figure them out on their own. Then, when Microsoft Windows was updated, these programs would sometimes stop working because the new version of Microsoft Windows did things differently. Microsoft had to ensure that Microsoft Windows would allow those programs to keep working.

The original SimCity for Microsoft Windows 3.x was one of such programs. It would use some memory and then give that memory back to Microsoft Windows. Then it would try to use that memory again, which is not allowed. This is called a use-after-free bug. In DOS and Windows 3.x, it didn't cause any problems because memory allocated to a program wasn't returned to the operating system immediately. When Microsoft tested its Windows 95 beta release, SimCity crashed. The Microsoft programmers who tested it took apart SimCity's code, identified the cause of the crash, and added special code to Windows 95 that checked whether SimCity was running. If it were, Windows 95 would use its memory allocator in a special mode that never returned memory to the operating system until SimCity stopped running. Thus, Microsoft programmers deliberately left a memory leak in Windows 95.

Web browsers experience this behavior, too. If Chrome or Firefox implements a quirk and web developers build around it, that quirk

effectively becomes part of the web platform. The browser can't just change it without potentially breaking the web.

A similar situation may occur when a library states in the docs that a function returns an unordered list, but it actually returns a sorted list. Some users might start depending on the list being sorted, even though they shouldn't. If the library later becomes truly unordered, the code of such users may malfunction.

Think of a time you used an API and discovered a helpful side effect, perhaps a timestamp returned in a specific format that you then parsed. If the API owner changes that format without realizing anyone is using it, your integration breaks. That's Hyrum's Law.

The law is especially relevant in API and semantic versioning: It teaches that even minor alterations can be breaking changes from someone else's perspective. Thus, maintainers often go through elaborate deprecation cycles, marking features as deprecated but still supporting them for a long time, because they know some unknown user out there depends on them.

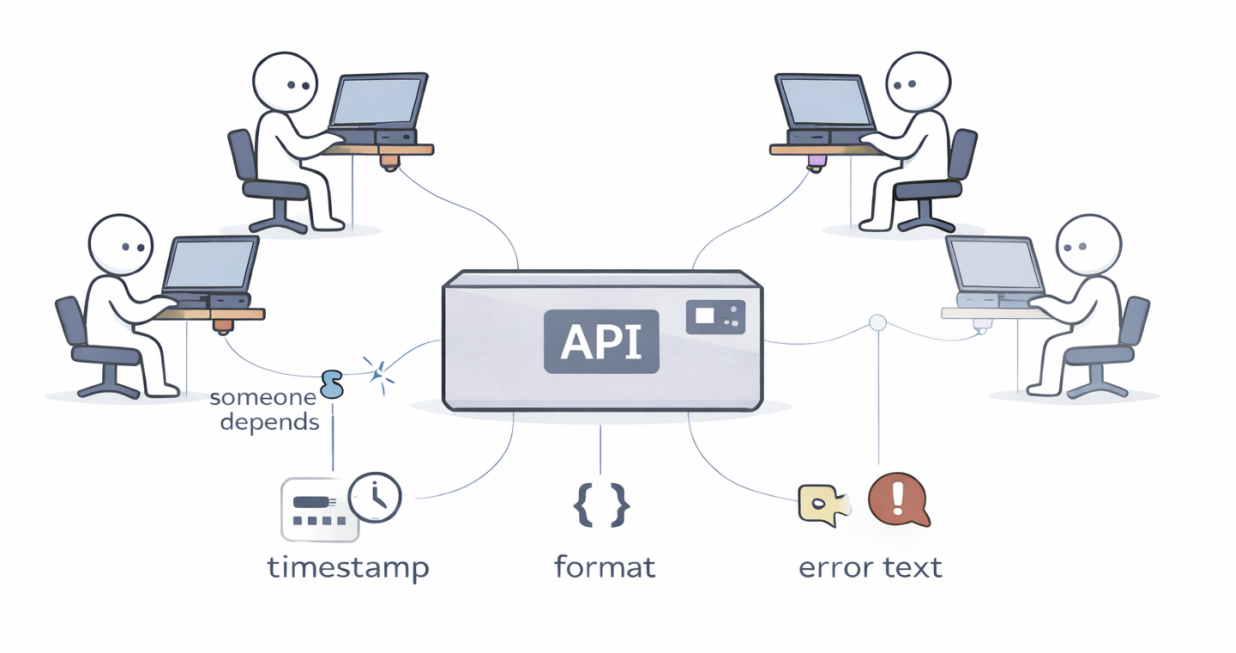

Figure 5.1 **Hyrum's Law**

Related to

- The Law of Leaky Abstractions

Further Reading

- Winters, T., Manshreck, T., & Wright, H. (2020). *Software engineering at Google: Lessons learned from programming over time.* O'Reilly Media.
- Website: hyrumslaw.com

OceanofPDF.com

6. The Second-System Effect

It is the tendency of small, elegant, and successful systems to be succeeded by overengineered, bloated systems, due to inflated expectations and overconfidence.

Key Takeaways

- *The first system someone builds is kept lean (either by constraints or inexperience). When designing its successor, there's a tendency to incorporate all the extras that were initially left out, which leads to feature bloat.*
- *Overconfidence is the main cause. Having succeeded once, the team feels they can tackle much more in the next iteration by underestimating complexity.*
- *The Second-System Effect often results in overengineering. The product has more modules, generality, and features than necessary, which can hurt performance and maintainability.*

Overview

The term describes a pattern observed in system design, where a small, elegant system is built and works well. A second version or a follow-up project is initiated, and the designers, freed from the constraints and caution of the first project, try to build a much bigger and more complex system. They add all sorts of enhancements, address all edge cases, and incorporate wishlist features. The result is often an *overly complex, behind-schedule system*.

The syndrome is an *exercise in hubris*. The team doesn't realize how much harder the bigger scope will be. The Second-System Effect warns engineers to be suspicious of grand redesigns that promise to include everything.

Origins

Fred Brooks coined the term in 1975 in *The Mythical Man-Month*. He observed the phenomenon in practice at IBM. The phrase "second-system syndrome" appears in an essay of the same name. One example that Brooks gives are IBM's successful small systems that were followed by more complex systems (e.g., the jump from simple 7000-series OS to OS/360), which tried to do far more and ran late or failed.

Examples

Let's consider a startup building a minimal web service that's very successful. For Version 2, the team plans a complete rewrite with a microservice architecture, tons of configuration options, plug-in frameworks, and even adding features users never asked for. The project explodes in complexity and misses deadlines, which is a textbook Second-System Effect.

During the 1960s, IBM created a simple and useful operating system for the IBM 7000 series mainframe. Immediately after, they started a much more ambitious project: the OS/360 operating system for new computers. In this project, designers tried to integrate all the features they left out from the first system. This led to an overengineered solution, which took more time and effort to build and stabilize (as

described by Fred Brooks). The system eventually worked, but it became notorious for delays and complexity.

Another example is Netscape Navigator from the 1990s. After initial success, a decision was made to rewrite the browser, leading to the Mozilla Suite. The rewrite took many years, during which Internet Explorer caught up. The new suite was considered too complicated, and Netscape lost its lead.

Brooks also mentions how architecture in the second round can suffer: For example, a compiler written the first time might be one-pass (because of constraints), but the second time, the team decides on a multipass, ultraoptimizing compiler with every conceivable feature, potentially resulting in delays and a slower, larger product.

Figure 6.1 **The Second-System Effect as an Overengineered House**

Related to

- YAGNI
- Feature Creep

Further Reading

- Brooks, F. P., Jr. (1995). *The mythical man-month: Essays on software engineering* (Anniversary ed.). Addison-Wesley.
- Second-system effect. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Second-system_effect

OceanofPDF.com

7. Fallacies of Distributed Computing

It is a set of eight false assumptions that are often made when designing or working with distributed systems.

Key Takeaways

- *In reality, networks drop messages, introduce delays, have finite throughput, can be insecure, and so on. Properly built distributed systems must account for these (retries, timeouts, security measures, dynamic discovery, etc.) rather than assume ideal conditions.*
- *The fallacies often manifest in subtle bugs. For instance, assuming latency is zero might lead one to make chatty remote calls, which work fine locally but become painfully slow over a network.*
- *Taking these fallacies into account up front leads to defensive design. That means using caches (because bandwidth/latency aren't perfect), building in redundancy (networks aren't reliable), and handling dynamic membership (topology changes).*

Overview

The Eight Fallacies serve as a checklist of what *not to assume*. In essence, they all spring from treating a distributed system like a local one. The fallacies are

1. The network is reliable.
2. Latency is zero.

3. Bandwidth is infinite.
4. The network is secure.
5. Topology doesn't change.
6. There is one administrator.
7. Transport cost is zero.
8. The network is homogeneous.

For example, developers might write code as if calling a remote service is just like calling a local function (i.e., ignoring latency and failure). These mistaken assumptions can lead to serious issues such as unhandled errors when the network breaks, poor performance due to ignoring latency, security breaches from failing to authenticate remote calls, and so on.

By calling them "fallacies," creators sought to instill the mindset that the network will fail and not behave ideally, so your system must be designed to tolerate that.

Over time, these became widely taught principles in distributed computing courses.

Origins

The concept is credited primarily to L. Peter Deutsch (with others like James Gosling adding to the list) around 1994 at Sun. Initially, there were seven fallacies. An eighth ("The network is homogeneous.") was added by Gosling a bit later.

Deutsch observed that many engineers, especially those used to local computing, would subconsciously assume the network just

works. Formalizing these assumptions as a list helped teams remember to address each when designing distributed systems.

Examples

On June 13, 2014, a power failure at one Amazon data center took down SimpleDB across all regions for 2 hours. The irony is that the system was designed to handle node failures. But when multiple storage nodes lost power simultaneously, the lock service got hammered with de-registration requests. Healthy nodes tried to verify their status, hit timeout walls due to increased latency, and removed themselves from the cluster, convinced they'd lost connectivity when they hadn't.

The distributed systems fallacies showed up all at once. The architecture assumed reliable networks, negligible latency, and uniform network paths. None of that held under pressure. A localized hardware failure cascaded through these assumptions, and the failure detection mechanism itself became the bottleneck that turned a single data center problem into a global outage.

Consider a developer building a distributed caching system who assumes that "network latency is zero." They might design the cache to fetch data from a remote node for every little lookup. In a LAN or in theory, this seems fine, but in practice, it thrashes with high latency and poor performance, violating the second fallacy ("Latency isn't zero, so minimize round-trip.").

Another example is when a system assumes "the network is secure" (Fallacy 4) and thus sends sensitive data unencrypted or when it

doesn’t validate inputs from other services. This could lead to breaches if the network is compromised.

Yet another example is deploying a service and assuming that only one admin is needed (Fallacy 6). In reality, multiple parties or automated processes administer subsystems, potentially making conflicting changes.

Each fallacy has led to real-world failures: For example, not planning for changing topology (Fallacy 5) has broken systems when machines are added/removed unexpectedly.

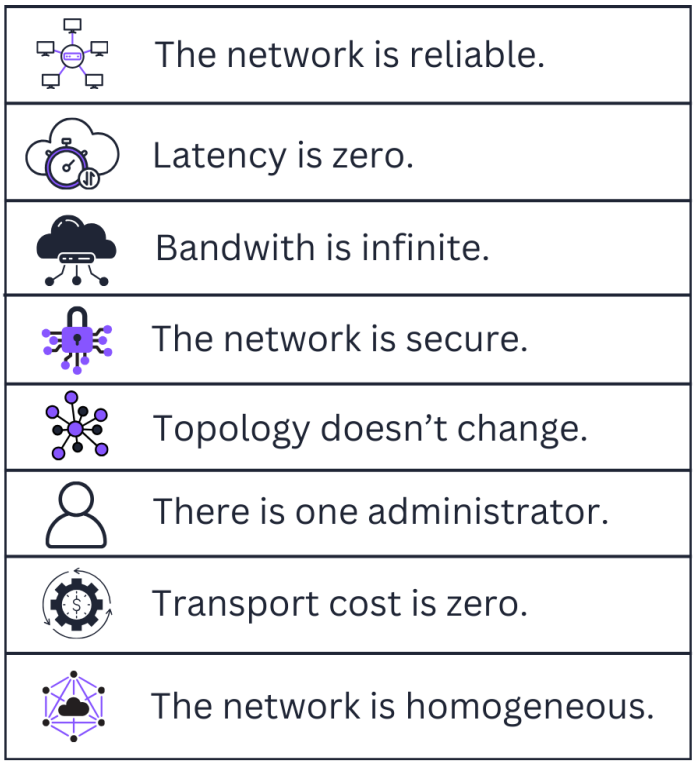

Figure 7.1 **Fallacies of Distributed Computing**

Related to

- The CAP Theorem

- Murphy’s Law

Further Reading

- Fallacies of distributed computing. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing
- Rotem-Gal-Oz, A. (n.d.). *Fallacies of distributed computing explained*. https://arnon.me/wp-content/uploads/Files/fallacies.pdf

OceanofPDF.com

8. The Law of Unintended Consequences

Whenever you change a complex system, expect surprise.

Key Takeaways

- *No matter how well you plan, any significant change in a complex system can produce some unexpected results.*
- *Consequences can be categorized as unexpected benefits (happy surprises), unexpected drawbacks (side effects that hinder), and perverse results (the action makes the original problem worse).*
- *In software engineering, these often manifest as a fix or new feature introducing bugs or performance issues elsewhere.*

Overview

The Law of Unintended Consequences states that *outcomes are not entirely predictable*. Systems have complex interdependencies and human factors that can cause surprises.

For instance, adding a new feature might unexpectedly degrade performance due to its interaction with an unrelated module. Or, simplifying a UI could lead to a heavier backend load because users use the feature more often than expected (an *unexpected drawback* or benefit, depending on perspective).

In essence, it reminds engineers that systems are complex and that our mental models are incomplete. Therefore, when implementing changes, one should anticipate that unknown unknowns will crop up.

Origins

Sociologist Robert K. Merton popularized the term in the 20th century, although the concept dates back further. In software, it's more of a borrowed concept than a specific person's law.

Examples

Forcing users to change their password every 30 days, requiring a mixture of letters and symbols, should boost security, with a goal to make passwords harder to crack. Yet, an unintended consequence is that users start to choose some simple passwords, just appending an increasing number or symbol each time (e.g. Password1, then Password2). This practice made passwords less secure.

In the world of software development, enabling a new logging feature to help with debugging might fill the disk and crash the system (a perverse result relative to the goal of stability). So, a solution to one problem can lead us to even bigger problems.

Another example is a social network changing its algorithm to boost user engagement, but as an unintended consequence, it ends up amplifying outrage or misinformation.

On a smaller scale, a bug fix in one part of the code unexpectedly causes a regression in another module that depended on the original bug behavior (overlap with Hyrum's Law).

These cases illustrate that *complex systems,* whether *software* or *social,* often respond unpredictably to interventions.

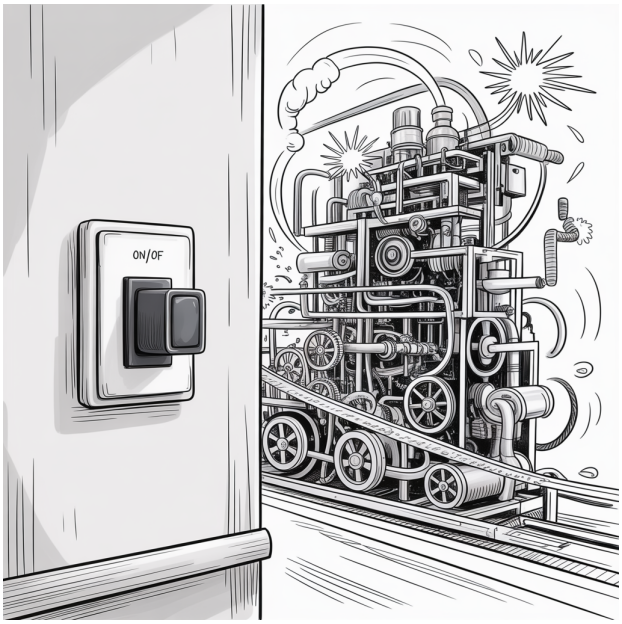

Figure 8.1 **The Law of Unintended Consequences is often seen as a simple system that tries to regulate a complex system.**

Related to

- Murphy's Law
- Hyrum's Law

Further Reading

- Levitt, S. D., & Dubner, S. J. (2005). *Freakonomics: A rogue economist explores the hidden side of everything.* William Morrow.
- Unintended consequences. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Unintended_consequences

OceanofPDF.com

9. Zawinski's Law

Every program attempts to expand until it can read mail. Those programs that cannot grow are replaced by ones that can.

Key Takeaways

- *Feature creep is unavoidable. Over time, software tends to accumulate more features, which leads to software bloat.*
- *A lean, minimal application that gains popularity will continually add features until it becomes as complex as its competitors. Often, a new, simpler competitor will emerge, and eventually, that newcomer will bloat in the same way.*
- *Programs expand because users (and product managers) keep asking for just one more feature. There is constant pressure to incorporate popular capabilities to keep users from leaving for other tools.*
- *Each new feature increases complexity, which makes the product confusing for users. Developers should protect the tool's focus and resist platform sprawl.*

Overview

Zawinski's Law is a humorous observation about software evolution: Applications continually gain features until they do everything, even things completely outside their original scope. In essence, it highlights *feature creep*, that is, the gradual expansion of scope in software development.

This law reflects a real dilemma. As an application attracts more users, it faces growing expectations to add more capabilities. For example, a basic note-taking app might later incorporate chat or sharing, and a simple to-do list might add calendars and messaging.

Zawinski himself clarified that his point was about “platformization”: Once users live in an app for a significant part of their day, the app is being pushed to *become an all-purpose platform*. A text editor becomes a complete IDE with build tools, and a web browser grows into a suite with email, plugins, and even an app store. Over time, a software product can transform from a specialized tool into a bloated ecosystem.

However, the core problem is not the number of features, but that every new feature increases complexity. Interfaces become larger, and codebases grow. The software may become harder to maintain and slower to run (in line with Wirth’s Law, which states that software expands far beyond the rate at which hardware improves).

Thus, Zawinski’s Law states that unchecked expansion can sabotage a product’s original value.

Developers and managers should remember that adding features is easy, but adding only the right features and saying no to the rest is essential.

Origins

Jamie Zawinski (often known as jwz) formulated this law around 1995, during his time at Netscape. Zawinski was a key programmer

on Netscape Navigator (the dominant web browser of the 1990s) and later added the integrated Netscape Mail reader.

He described the browser's evolution as "our contribution to the proof of the Law of Software Envelopment," stating the now-famous saying. The quote encapsulates that Netscape started as a web browser. Still, by version 2.0-3.0, it had expanded to include an email client and news reader, essentially trying to do everything Internet-related in one package.

Zawinski dubbed it the "Law of Software Envelopment" to suggest software envelops increasingly more functionality over time.

It's no accident that "reading mail" was the chosen absurd example. In the mid-1990s, many personal computers ran only one program at a time to check email. It was often necessary to *exit your current application and launch a mail program*.

Examples

Zawinski's Law is observable in many real-world cases. A classic example is *Netscape vs. the rise of Firefox*. Netscape Navigator grew from a slim browser into *Netscape Communicator*, an expansive suite (browser, email, news, web editing, etc.). Over time, it became sluggish and overcomplicated. This development paved the way for new entrants such as the early Mozilla Phoenix/Firefox, which deliberately stripped down to just a fast browser. Firefox was essentially a reaction to Netscape's bloat. Of course, as Firefox itself added plugins, themes, and more features over the years, it too became heavier.

Today's Google Chrome started lean, but it now includes a massive extension ecosystem and many built-in tools (extensions and features we see almost daily when we open it). The cycle keeps repeating.

Modern software products also show Zawinski's Law. Take Google Docs, for example, which initially was a simple online word processor. Over time, it added comments, real-time chat, tagging, task assignments, and email notifications, essentially building in communication features that keep users on the platform. The same is true of Microsoft Word/Office, which gained emailing and sharing functions.

Furthermore, a popular tool, Slack, a team chat app launched in 2013 as a team messaging application, famously set out to eliminate email, but Slack itself integrated voice calls, video meetings, file sharing, bots, and many app plugins. It hasn't literally added an inbox, but it now wants to be a one-stop workplace hub, which is far beyond simple messaging.

Even niche developer tools follow this pattern. For instance, GitHub started by hosting code and then expanded to include issue tracking, wikis, project boards, discussions, CI pipelines, and package registries. Each addition made sense in satisfying user needs, but together, they transformed GitHub from a focused tool into a broad platform. The consequence is that newcomers sometimes prefer a simpler alternative (e.g., a lightweight Git repository), thereby resurrecting the cycle.

In summary, Zawinski's Law is visible in all cases where software bloats with features over time. It reminds us of the trade-off: Adding capabilities can make a product more useful to more people, but it can also dilute the original simplicity.

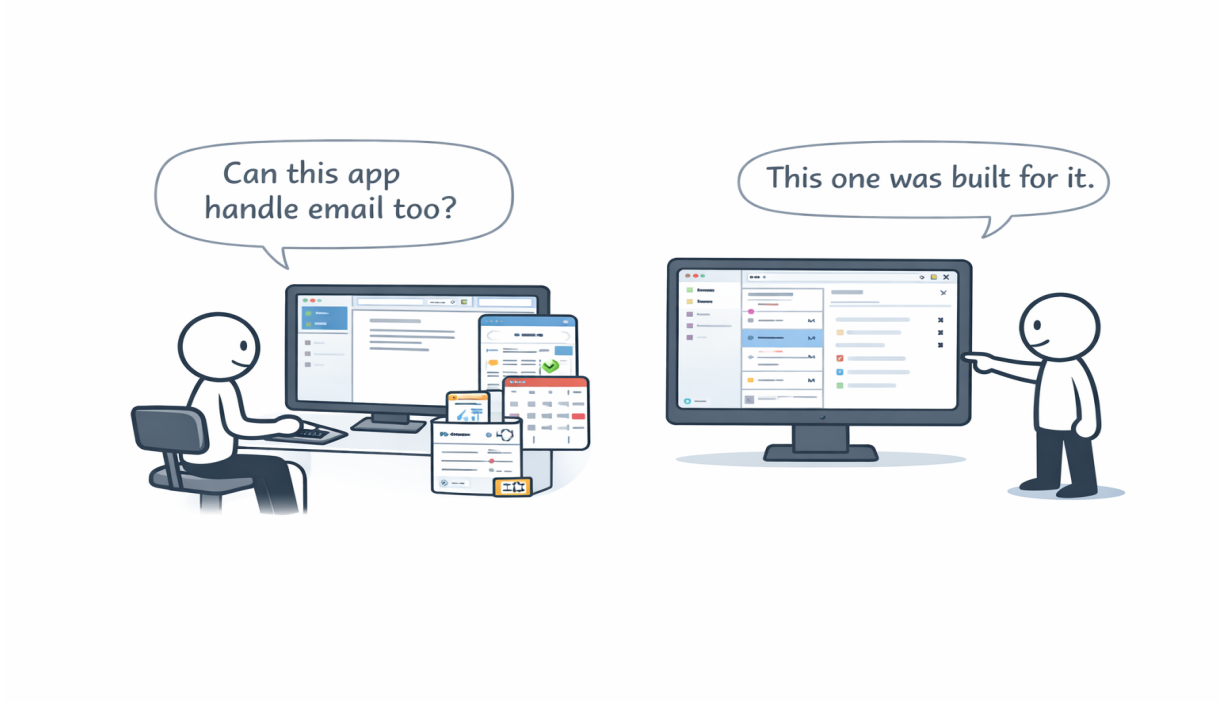

Figure 9.1 **Zawinski's Law**

Related to

- Wirth's Law

Further Reading

- Jamie Zawinski. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Jamie_Zawinski
- Spolsky, J. (2001, April 21). *Don't let architecture astronauts scare you*. Joel on Software.

https://www.joelonsoftware.com/2001/04/21/dont-let-architecture-astronauts-scare-you/

OceanofPDF.com

Part II
People, Teams, and Organizations

These laws cover coordination cost, output concentration, incentives, and why adding people rarely scales delivery linearly.

OceanofPDF.com

10. Conway's Law

Any organization that designs a system will produce a design whose structure mirrors the organization's communication structure.

Key Takeaways

- *The architecture of software systems often mirrors the organization's org chart or team structure.*
- *If your company is organized in silos, you might end up with siloed software modules that don't communicate well, reflecting those barriers.*
- *To achieve a desired software architecture (e.g., microservices), it might be necessary to restructure teams accordingly as they build software aligned with their communication paths.*
- *At the beginning of a project, the way in which teams or departments are split (e.g., frontend vs. backend) will likely lead to software boundaries at the same places.*

Overview

Conway's Law states that *software systems are always as complex as the organization that builds them*. In other words, the structure of a software system is often influenced by the structure of the team building it and their communication patterns.

This principle can result in a software architecture that is not optimal for the problem being solved, as the team may prioritize their own organizational needs over the system's needs. Thus, an organization with small, distributed teams will produce a modular service

architecture, while an organization with large, collocated teams will produce a monolithic architecture.

In a broader sense, taking this law into account, we can even say that HR defines software architectures.

To mitigate problems defined by Conway's Law, we can use *the Inverse Conway Maneuver*. This technique involves software architects, engineers, and leaders in defining organizational structures. Doing it can lead to better software.

Yet we can see that many organizations ignore Conway's Law and believe that organizational structures and software architecture are decoupled, leading to surprises in the end.

Origins

Melvin Conway introduced the idea in his 1967 paper "How Do Committees Invent?" After *Harvard Business Review* rejected the paper for lacking formal proof, it was published by *Datamation* in 1968.

Fred Brooks later named it "Conway's Law" in *The Mythical Man-Month*, establishing the concept as foundational in software engineering.

Examples

A company had separate departments for frontend, backend, and database. The software they built had a three-tier architecture, with each tier independently designed by its respective department. Often, the integration between tiers was painful because the teams had misaligned goals (e.g., the frontend team optimizing for a flashy interface and the DB team optimizing for data integrity, with friction between them).

In the early 2000s, Amazon was a monolithic entity that developed monolithic software. The company's e-commerce business was a tightly coupled system in which changes in one part affected other parts of the system, and coordination across multiple teams was necessary. As the company scaled, Jeff Bezos realized that the system was hindering innovation because teams could not work in isolation due to the system architecture being a reflection of the communication structure of the company.

Amazon recognized Conway's Law back then. Their hierarchical organizational structure was causing their software to be hierarchical and interdependent, too. To counter this, they decided to change their organizational structure and software simultaneously. They broke their monolithic application down into hundreds of microservices, each with its own API and single responsibility. At the same time, they reorganized their teams into "two-pizza teams": autonomous teams of 5–10 people who could be fed by two pizzas. Each two-pizza team was given single-threaded ownership of a particular microservice, from ideation through deployment to ongoing operation of the service. The Payment team owned the payment service. The Product Catalog team owned the catalog service. Teams did not hand off their work to the operations teams; they owned it all.

This change was the Inverse Conway Maneuver in practice. Amazon restructured its organization to match the desired architecture. This enabled more modular, scalable software, allowing teams to innovate faster without depending on each other.

Conversely, in a small startup of three people, a monolithic application is often present, with one team and one codebase.

A recent example is the separation between AI research groups and application development groups (read developers). The AI research group optimizes models using benchmark metrics that are not realistic, yet reports excellent results. The application group delivers low-quality results. Why? Because of Conway's Law. Since there's division within the organization, there's division within the system as well.

Every group tries to optimize their own communication system, not the final result.

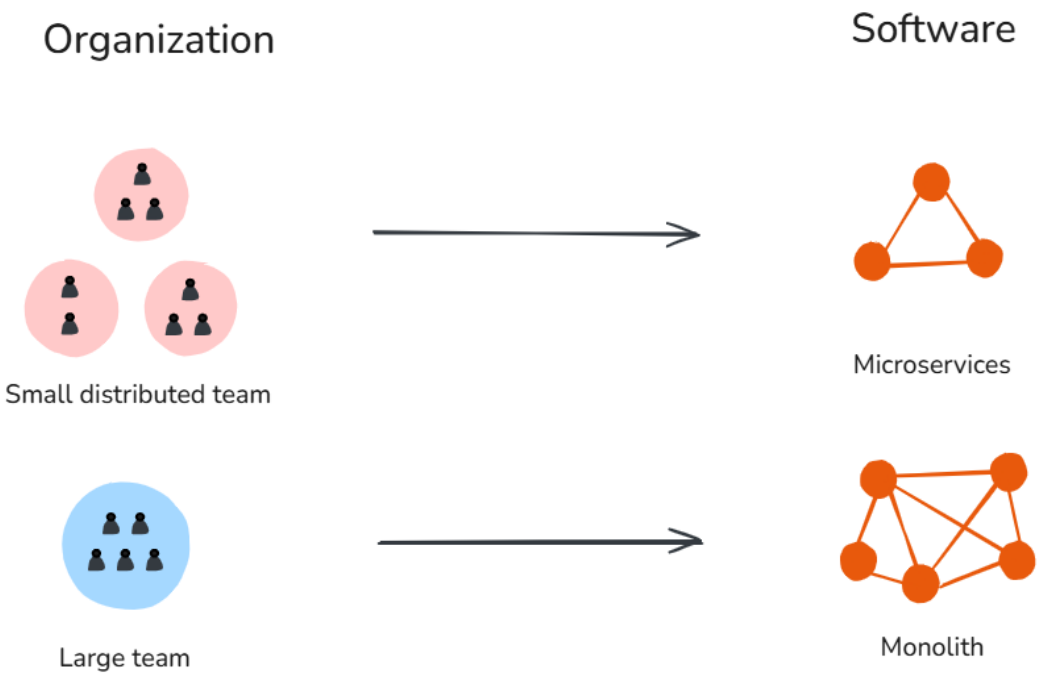

Figure 10.1 **Conway's Law**

Related to

- Brooks's Law
- Gall's Law
- The Law of Leaky Abstractions
- Hyrum's Law

Further Reading

- Conway, M. E. (1968). *How do committees invent? Datamation*. https://melconway.com/Home/pdf/committees.pdf
- Conway's law. (2022, October 20). *Martin Fowler*. https://martinfowler.com/bliki/ConwaysLaw.html
- Skelton, M., & Pais, M. (2025). *Team topologies: Organizing business and technology for fast flow of value* (2nd ed.). IT Revolution Press.
- Spotify engineering culture (part 1). (2014, March 27). *Spotify Engineering*. https://engineering.atspotify.com/2014/3/spotify-engineering-culture-part-1

Sidenote: The Spotify Model

The Spotify Model is an approach to team and organizational structure in which teams are organized around features rather than technologies. It was popularized by Spotify, and in this model, teams operate as cross-functional squads, where each owns specific end-to-end features.

The model introduces the following four key structures:

- Squads are small teams (usually 6–12 people) that work together on related features.
- Tribes are groups of squads working in the same business area.
- Chapters group people with similar skills across different squads.
- Guilds are communities of practice where people share knowledge across the organization.

Other companies treat these snapshots as templates and copy their structure, overlooking that they were built to address Spotify's problems at the time.

The model works when it solves real coordination problems, but it fails when copied as a template.

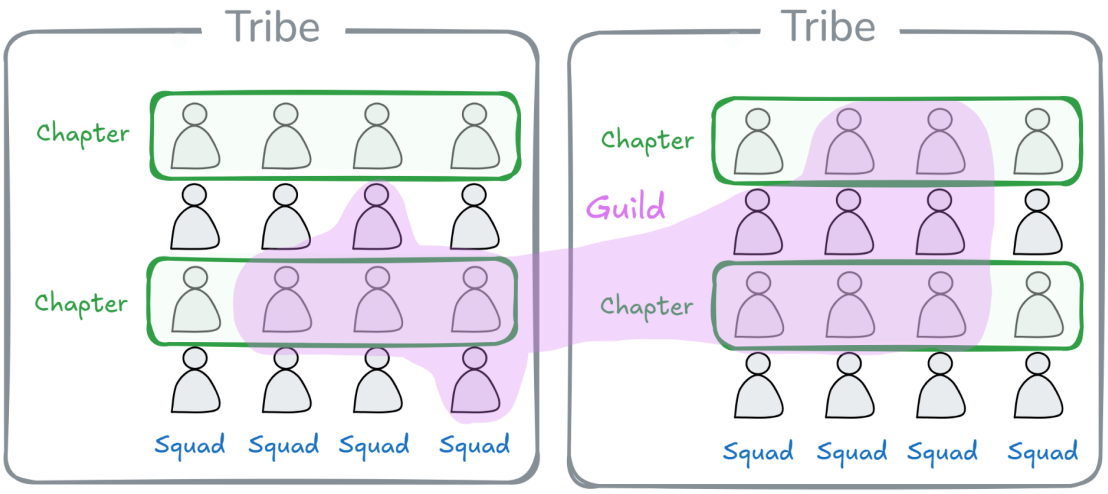

Figure 10.2 The Spotify Model

OceanofPDF.com

11. Brooks's Law

Adding manpower to a late software project makes it later.

Key Takeaways

- *Simply throwing more developers at a project that's running behind schedule usually slows it down initially.*
- *New people take time to get up to speed, consuming existing team members' time (for training and coordination), which reduces overall productivity.*
- *Instead of hoping that manpower will solve slippage, adjust the scope or timeline. Brooks's Law suggests managers to be wary of "just hire more coders" as a solution to late schedules.*

Overview

Brooks's Law conveys the belief that a software development effort is perfectly divisible among people. In reality, adding a person to a project incurs training costs and increases the number of communication paths, which can outweigh that person's contribution for a while. Anyone who's worked inside a real organization has probably seen this pattern.

This law especially holds true in late projects. If a project is already late, adding new developers can cause further delays as they learn the system and may introduce new bugs or mismatches in work. Brooks famously illustrated this with the following analogy: *"You*

cannot get a baby in one month by impregnating nine women.” Some tasks just take a certain amount of sequential time.

Also, when new joiners arrive, more experienced team members must leave their work to onboard them, which creates an additional dip in productivity.

The law doesn’t say adding people *never* helps; in case of long projects, adding people early on can increase total throughput after the initial cost. But as a response to lateness, it’s frequently counterproductive.

Origins

Frederick P. Brooks Jr., who managed the IBM OS/360 project, formulated this law based on painful experience. The *Mythical Man-Month* explores why software projects fail and where management assumptions break down.

Brooks’s Law has since been validated (and sometimes challenged) in various contexts, but it remains a cornerstone of software engineering theory.

Examples

A project is a month behind schedule, so management adds three new developers to a five-person team. Over the next few weeks, progress slows. The original developers spend much of their time explaining the design and code to newcomers, and merging their work causes many integration headaches. The project slips further, now 2 months late.

In another scenario, a complex bug required deep knowledge of the system. Adding more people didn't help because only the original dev understood it, and more debuggers only made things worse.

This was clear in our case, when we had an 8-person team which was constantly falling behind schedule. After some analysis, we understood that there was a lot of communication bottlenecks as well as overhead there. So, we decided to move two people from our team. This move helped us cut communication overhead from 28 lines to only 15. A smaller team picked up pace fast and got back on the track delivering much more than the previous team. So, the solution was removing people from the project, not adding new ones.

However, I should note that if there's a real parallel task (e.g., testing 10 independent modules), adding people might help. Still, Brooks's Law warns that, in most cases of lateness, especially in tightly interdependent systems, additional manpower is initially a liability.

This is why scaling a software team isn't linear. After a point, more members yield increasingly less output per person. If mismanaged, this can stall the project entirely.

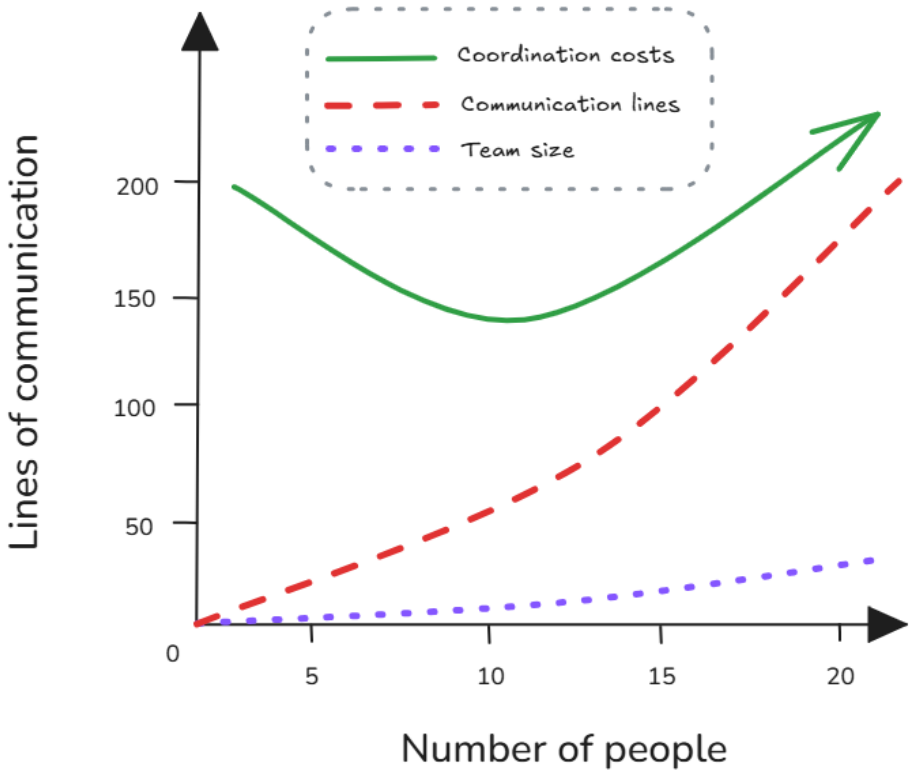

Figure 11.1 **Brooks's Law**

Related to

- Conway's Law
- The Second-System Effect
- The Law of Unintended Consequences

Further Reading

- Brooks's law. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Brooks%27s_law
- No Silver Bullet. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/No_Silver_Bullet

- Skelton, M., & Pais, M. (2025). *Team topologies: Organizing business and technology for fast flow of value* (2nd ed.). IT Revolution Press.

Sidenote: Little's Law Makes Brooks's Law Measurable

Little's Law is a formula from queueing theory that connects three variables: L = λ × W, where L is the number of items in a system (work in progress), λ is the throughput (items completed per unit of time), and W is the average time an item spends in the system (lead time):

Lead Time = Work in Progress / Throughput

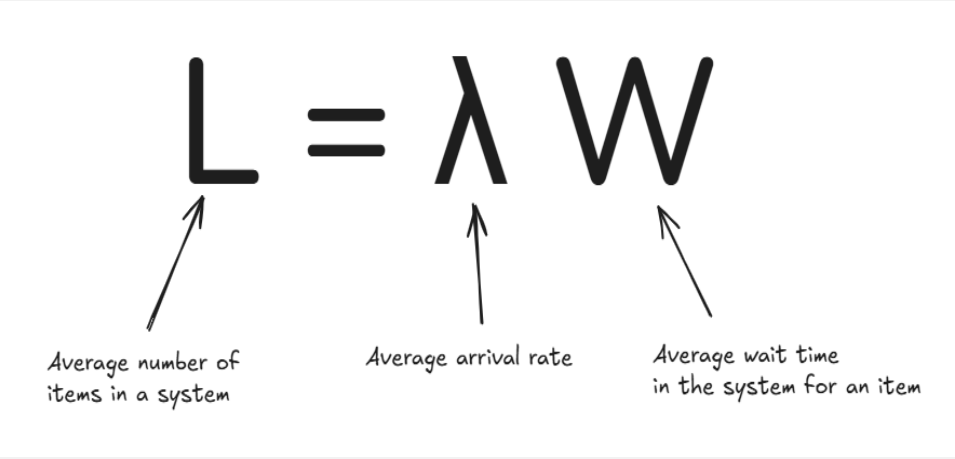

Figure 11.2 The Little's Law

This means that if your team completes 5 tasks per week and has 20 tasks in progress, average lead time is 4 weeks. Push WIP to 40 with the same throughput, and the lead time doubles to 8 weeks. No estimation is needed here.

This is what makes Brooks's Law measurable. When you add people to a late project, coordination overhead increases, WIP grows, but throughput stays flat or drops. Little's Law shows you the result: longer lead times, not shorter ones.

The practical takeaway is simple. If you want faster delivery, your first move should be reducing WIP, not adding people. Finish what's been started before starting new things.

OceanofPDF.com

12. Dunbar’s Number

There is a cognitive limit to the number of stable social relationships one person can maintain, which is commonly cited as around 150 people.

Key Takeaways

- *Dunbar’s Number (~150) is the size of a community in which everyone knows each other’s identities and roles. Beyond this, our brains struggle to track relationships.*
- *In software organizations, this principle suggests that teams larger than 100–150 will require more hierarchy, or splitting into subgroups, to function effectively. Coordination overhead rises sharply as the number of people increases.*
- *Smaller limits exist for closer relationships. For teams, effective working groups might be much smaller than 150, but this number is an upper limit for a community feeling.*
- *Strong collaboration happens in groups far below the 150 threshold (meaning that small teams win).*

Overview

Dunbar’s Number implicates that an engineering department of 150 might function informally, but as the number grows, more formal rules, communication channels, and management layers may become necessary. Informal knowledge (“I know who to talk to about X.”) begins to fail, and teams feel impersonal.

While this number is not particularly large, you will hardly recognize everyone at 150 people. At 300 or 500, it is already impossible.

The team size affects product teams directly. So, if your organization or open source community grows large, you must pay attention to communication structures (or heed Conway's Law to reorganize the teams) to avoid chaos.

The rule also encourages designing subteams or "teams of teams" that are small enough to work together effectively (such as Amazon's two-pizza team concept, although that's much smaller, at ~10).

Dunbar also proposed smaller social layers:

- Intimate relationships (~5)
- Trusted collaborators (~15)
- Close working relationships (~50)
- Stable social connections (~150)

For software teams, this implies that

- High-trust work happens in tiny groups.
- Teams larger than 10–15 lose cohesion.
- The 150 limit applies to departments or communities, not everyday collaboration.

Origins

Robin Dunbar introduced this number in a 1992 paper and popularized it in his book *Grooming, Gossip, and the Evolution of Language*. He found a correlation between primate neocortex size

and social group size, predicting human social group size at approximately 150.

Examples such as historical village sizes, military company units, and communal groups often hover around 100–200, lending credence to the figure.

Examples

Many startups report that the culture shifts around the 150-employee mark. You no longer know everyone's name, informal communication falters, and you start needing things like all-hands meetings and internal newsletters. That's Dunbar's number in action.

Another example is the *open-source Linux kernel community*. It has thousands of contributors, but within it are maintainers and subsystem teams that are relatively small (core team). Communication is structured into mailing lists by subsystem, so that any given person communicates only with a small number of peers (e.g., audit, dash, devicetree, etc).

Valve Corporation is a good example, as it runs its entire company on Dunbar's Number. They have no managers, no hierarchy, just self-assembling teams capped at 12 people. Why 12? Because once you hit 15, the trust that makes flat structures work starts to collapse. People can't track everyone's skills, responsibilities, and current work anymore. This makes consensus-based decisions impossible.

When a project requires more than 15 people, Valve doesn't grow the team, but they split it. The Steam client, backend infrastructure, and community features all run as separate units instead of one bloated

Steam team. At the broader level, 50 people mark their typical project division, while 150 trigger a new office instead of cramming more bodies into one location. This lets them ship products with more than 120 million users, while having the communication speed of a startup.

Also, scrum teams or Amazon's two-pizza teams (5–10 people) know that even within 150, effective working teams are smaller units.

Dunbar's Number explains why an engineering org of 1,000 can feel siloed and naturally breaks into subgroups: No one can connect with all 1,000 people.

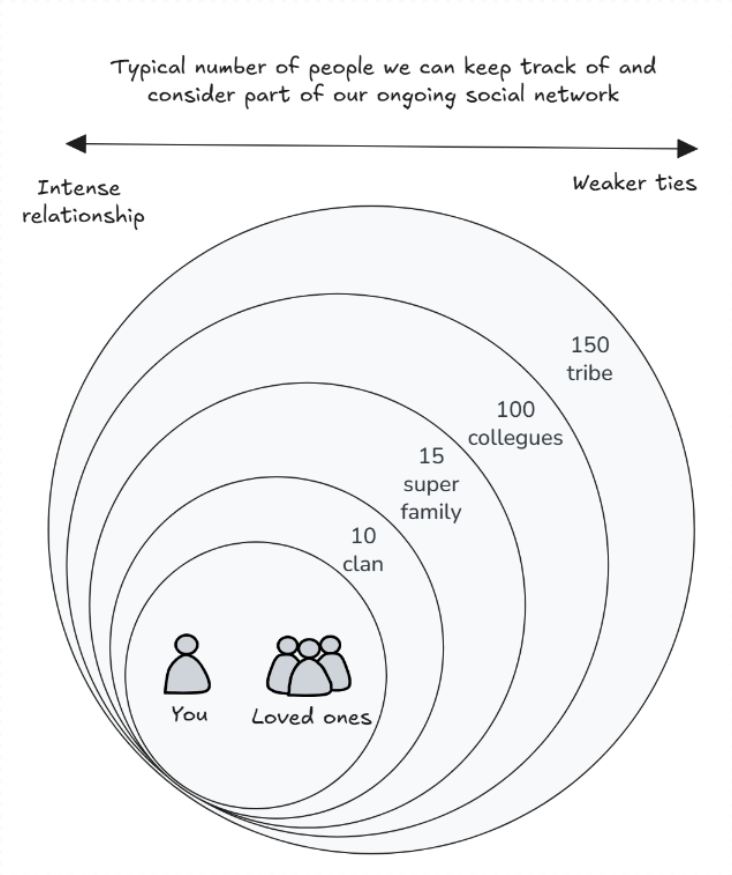

Figure 12.1 **Dunbar's Number: 150 People You Can Know**

Related to

- Conway's Law
- Brooks's Law

Further Reading

- Dunbar's number. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Dunbar%27s_number
- Dunbar, R. (2013). *Laughter, love and social cohesion* [Video]. TEDxTransmedia. https://www.youtube.com/watch?v=7ductvdAe5s

OceanofPDF.com

13. The Ringelmann Effect

Individual productivity decreases as group size increases.

Key Takeaways

- *In large teams, some people put in less effort because they assume others will pick up the slack or because their contributions are less visible.*
- *More people mean that more communication, meetings, and alignment are required, which can bog down progress. Time spent coordinating increases, leaving less time for actual work proportionally.*
- *There is a point at which adding people yields diminishing returns, or even negative returns per person. Small, focused teams often work better than poorly coordinated larger teams.*
- *Smaller teams or individuals feel greater ownership and responsibility.*

Overview

The Ringelmann Effect states that as more people pull on a rope, the total force doesn't scale linearly. Each person's effort decreases.

In software teams, this manifests as *team productivity per person often declining in larger teams*. Partly, it's due to coordination, and partly due to some individuals contributing less (consciously or not) when in a crowd. Here is the classic example: In a meeting of 20,

each person is less likely to speak up or feel less accountable than in a meeting of 3.

This law warns that adding more team members can make each existing member slightly less efficient. It also suggests why big projects often break teams into subteams: to keep each unit small enough to maintain high individual productivity and accountability.

Ringelmann, along with Brooks's Law, paints the picture that *teamwork doesn't scale linearly*. Teams must be managed and structured so as to mitigate these effects (defining clear roles, avoiding unnecessarily large-group tasks, etc.).

Origins

This principle was named after Max Ringelmann, who observed it in physical tasks. Later, it was termed "social loafing" in psychology and has been demonstrated in many group situations beyond rope pulling.

Examples

In a brainstorming session with 3 people, each might contribute many ideas. In a brainstorming session with 15 people, many participants will say nothing or very little, assuming others will or do not feel the need to speak.

In a software project, if one developer is working solo, they will fully tackle a feature. If 5 developers are on the same feature without a clear division, some may assume others will handle the tough parts, or duplication of effort and waiting on each other reduces efficiency.

In addition, there is the *coordination aspect*: If 10 developers work on one module, merging everyone's code, resolving conflicts, and communicating design decisions can consume significant time, so each person's net contribution is less than if only 2–3 were working on it.

A GitHub study of thousands of open source projects measured what happens to individual output as teams grow (Scholtes et al). Developers in 2–5 person teams averaged 1,850 lines of code monthly. At 10 people, that dropped to 1,200 lines, which is a 35% decline. When the team increased to over 50 developers, individual output cratered to 450 lines, a 75% drop from the smallest teams.

The villain here is coordination overhead. Small teams communicate directly, and conflicts can be resolved fast. If there are more than 10–12 developers, each new person adds more coordination overhead. Code reviews, merge conflicts, architecture discussions, sync meetings, it all stacks up exponentially. That's why large projects that succeed cap core teams at 10–15 people and use modular architectures. Small teams own different components, and coordination stays local, containing the Ringelmann Effect.

This effect is also a motivation behind Amazon's famous "two-pizza rule" (a team should be feedable by two pizzas, roughly 5–8 people) we saw in Conway's Law. Beyond that size, it's believed a team becomes less effective.

To tackle this problem, break large groups into *smaller, autonomous teams* where possible, establish clear individual responsibilities even

within large teams, and foster a culture in which everyone's contribution is visible and valued (to reduce loafing).

For instance, modern agile teams of ~7 people can often outperform a waterfall team of 20 because they aren't as affected by this effect.

This is something you have probably already witnessed: Teams with 3–9 members (ideally around 7) are the most efficient.

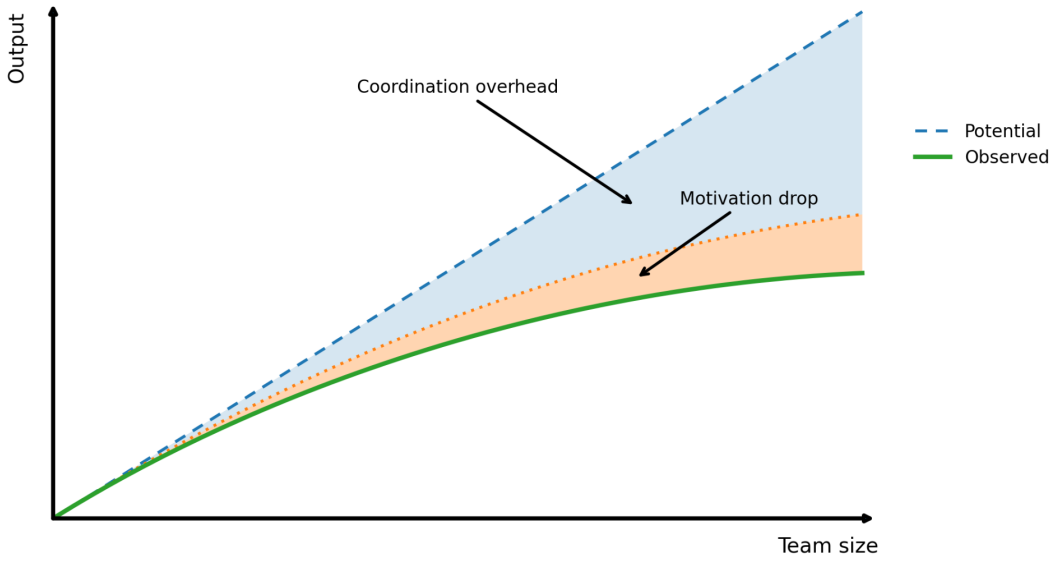

Figure 13.1 **The Ringelmann Effect**

Related to

- Brooks's Law
- Dunbar's Number
- Conway's Law

Further Reading

- Gladwell, M. (2002). *The tipping point.* Back Bay Books.

- Ringelmann effect. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Ringelmann_effect
- Scholtes, I., Mavrodiev, P. & Schweitzer, F. (2016). From Aristotle to Ringelmann: A large-scale analysis of team productivity and coordination in Open Source Software projects. *Empirical Software Engineering, 21,* 642–683.
- Social loafing. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Social_loafing

Sidenote: The Amazon Two-Pizza Rule

This rule says that any efficient team should be the size that can be fed by two pizzas, which is typically 5–8 people.

Small teams move faster because communication stays simple. In a team of 4, you have 6 relationships to manage. In a team of 6, that number jumps to 15. More people mean more meetings, slower decisions, and weak ownership. Keep teams small enough so that everyone knows what everyone else is doing, and you'll ship faster with less coordination overhead.

The number of links between people can be expressed as $n(n-1)/2$, where n = number of people.

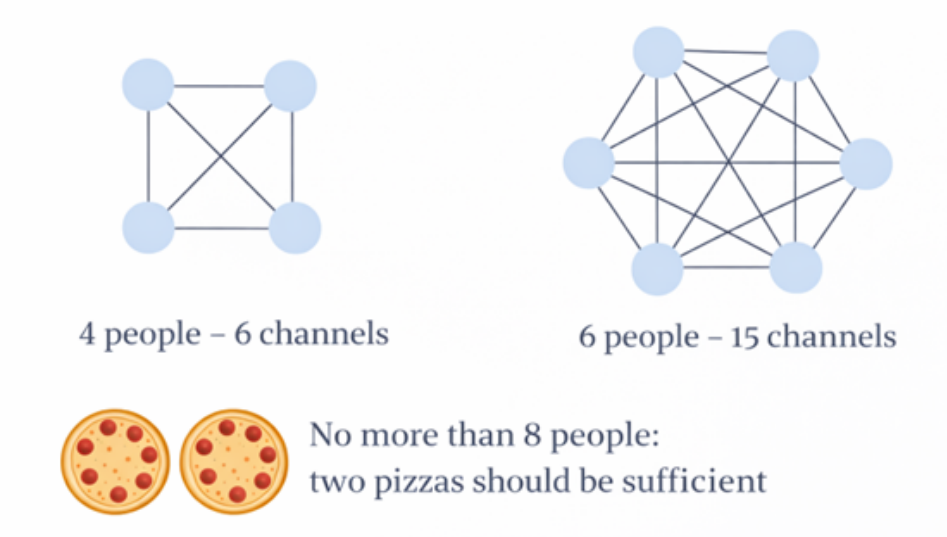

Figure 13.2 Two-Pizza Rule

"If you can't feed a team with two pizzas, it's too large"

—Jeff Bezos

Note that in the recent development of teams that use powerful AI tools, we often see teams that could become one-pizza team (3–4 people) across the business.

OceanofPDF.com

14. Price's Law

The square root of the total number of participants does 50% of the work.

Key Takeaways

- *A small fraction of people often contribute a significant fraction of the results. For example, in a 100-person engineering organization, about √100 (i.e., 10) people might produce half of the output.*
- *This law suggests there are a limited number of very high-productivity individuals (10x engineers, etc.) and many with lower output. Knowing this principle can help in team planning and facilitate understanding why losing specific individuals has a significant effect.*
- *As teams grow, productive output doesn't scale linearly. Adding more people increases total output, but many of those additional people will make smaller contributions than the core group.*
- *It's wise to rely on a small group of people when the knowledge is essential to the company.*

Overview

You have probably noticed that most of the (major) work depends on just a few people in your organization, and that is true. In software teams, it means *a relatively small group of engineers will deliver a disproportionately large part of the value*. This is similar to the Pareto Principle (80/20 Rule) but even more extreme for larger groups.

Price's Law says that simply hiring more developers won't necessarily scale output as expected. Beyond a point, many may contribute little or be working on peripheral tasks. It's also an argument for focusing on quality when hiring: A single excellent engineer can outperform several average ones.

However, we must be careful with this law: We don't want to make other people feel useless as they may be doing essential tasks that are given less importance. It highlights a risk. If those top $\sqrt{N}$ contributors leave, you lose a large chunk of productivity, so retention and preventing burnout for them are critical.

Origins

Derek de Solla Price, an information scientist, observed this distribution in scientific productivity. It's been generalized to various fields, and it sometimes surfaces in management functions, although not without controversy, as it can be demotivating or misused.

Examples

Take an open source project on GitHub with 30 contributors, for instance. Often, you'll see that maybe 5 contributors (roughly $\sqrt{30} \approx 5$) are responsible for about half the code commits or major features. The rest contribute smaller patches or documentation.

In a product development team of 16, a core group of 4 individuals are constantly solving the most complex problems and driving the architecture, whereas the others handle maintenance or minor

features. This isn't absolute, as it can change over time, and sometimes new members become part of the key group.

A recent example of this law was when Twitter cut staff hard right after Musk bought it, yet the product kept running. Before the takeover, Twitter had roughly 7,500 employees, which means $\sqrt{7{,}500} \approx 87$. Price's Law suggests that when the new Twitter leadership decided to lay off almost 50% of staff in early November 2022, the platform could still operate if the core 80–100 people stayed. But this holds only if the proper people are selected, of course.

Still, the law won't predict reliability. Layoffs strip redundancy in SRE, security, and moderation, and Twitter even asked some laid-off workers to return, a signal that it missed critical skills.

For managers, Price's Law implies that one should identify core productive members, ensure they're supported, and also not expect that doubling team size will double output. Beyond a point, one could add more marginal output.

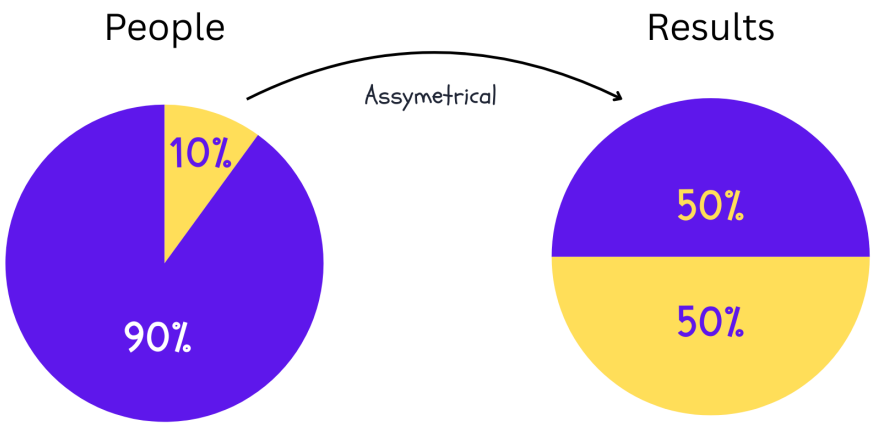

Figure 14.1 **Price's Law**

Related to

- The Ringelmann Effect
- Brooks's Law
- Dunbar's Number

Further Reading

- Price's law. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Price%27s_law

OceanofPDF.com

15. Putt's Law

Technology is dominated by two types of people: those who understand what they do not manage, and those who manage what they do not understand.

Key Takeaways

- *There is often a gap between deep technical understanding and management roles. Rarely does one person excel at both.*
- *Technical experts often aren't in management, and managers usually don't understand the technical details.*
- *This situation can lead to unrealistic expectations, such as when management sets unrealistic expectations because they don't understand the domain and its complexity. At the same time, engineers are feeling misunderstood.*
- *Successful organizations try to bridge this problem by training managers in tech basics or by promoting technically savvy leaders (or at least having a mix in leadership teams).*

Overview

Putt's Law explains a known concept: that the best engineers aren't in charge, and those *in charge aren't deeply knowledgeable about the technology*. While not universally true, this highlights a real challenge: Managing technologists requires some understanding of their work. But the problem is that managers often come from

different backgrounds or have been away from the code for a long time.

The law's corollary even says, "Every technical hierarchy, in time, develops a competence inversion," meaning the people at the top know less about the work than those below. The gap creates dysfunction: Managers set unrealistic deadlines because they don't understand technical subjects, while engineers build wrong solutions because they don't understand the business context.

For engineers, this law is often cited to explain why management makes bad decisions. For managers, it's a caution that not understanding the technology can lead to poor choices and dependence on subordinates. It overlaps with the Dilbert Principle as well (incompetent people end up in management).

The lesson from this law is clear: Technical leadership should maintain technical competence or risk mismanaging the technical work.

Origins

Putt's Law was revealed in the satirical book *Putt's Law and the Successful Technocrat* (1981). The author (`Archibald Putt`) used a pseudonym.

Examples

Think of a software manager who came from a sales or business background. They might push the team to add complex features at warp speed, but they fail to understand properly why quality or

testing is needed. They basically try to “manage what they do not understand.”

On the other side, a brilliant sysadmin in the company knows precisely how to optimize infrastructure but isn’t involved in management decisions about budgeting or project direction. They “understand what they do not manage.” This kind of dynamic can cause friction.

Perhaps the manager promises a client a particular feature by next week (not understanding the technical difficulty), and the engineering team is frustrated because the manager didn’t realize what’s involved. Many startups initially avoid this because founders often manage and deeply understand the tech, but as companies grow, specialization separates roles.

One remedy is to have CTOs or tech leads act as a bridge. They have one foot in management and one in technical detail, hopefully preventing total detachment.

Boeing 737 MAX aircraft crashes, in which 346 people have been killed, are an example of Putt’s law at work. During the development, engineers recognized the risks of MCAS (Maneuvering Characteristics Augmentation System), relying on one sensor only, and pilots having only seconds of reaction time if anything went wrong. Engineers offered proposals for safety improvements and better training. However, these were not implemented because of cost and schedule constraints (Herkert et al., 2020).

Putt’s Law states that this is exactly what will happen: A hierarchy of technological competence will develop, with people closest to the

work having deep competence but no decision-making authority, *which means they understand technology but cannot change it*. The next group, the managers who are no longer closely associated with the technology of modern flight control software, doesn't understand the technology. The engineers knew the failure modes but couldn't change the technology. The executives knew the technology but couldn't understand the failure modes. The final configuration was catastrophically dangerous. Putt's Law is essentially a cynical reminder to technical folks: If your boss doesn't seem to get what you do, that's normal in many places. And for managers, it means to rely on their experts and be humble about their knowledge gaps to avoid the pitfalls the law highlights.

Figure 15.1 **Putt's Law**

Related to

- The Peter Principle
- The Dilbert Principle

Further Reading

- Putt, A. (1981). *Putt's law and the successful technocrat*. Exposition Press of Florida.
- Herkert, J., Borenstein, J., & Miller, K. (2020). The Boeing 737 MAX: Lessons for engineering ethics. *Science and Engineering Ethics, 26*(6), 2957–2974.

OceanofPDF.com

16. The Peter Principle

In a hierarchy, every employee tends to rise to their level of incompetence.

Key Takeaways

- *Individuals get promoted based on success in their current role until they reach a role where they are no longer competent. They stop there, meaning many positions are filled by people who are not an ideal match.*
- *A great developer might be promoted to a team lead or manager position, only to perform poorly. Now you have a bad manager and have lost a good developer.*
- *Companies should offer dual career paths (technical vs. managerial) or carefully manage promotions to avoid stagnation.*

Overview

The Peter Principle explains why organizations are filled with mediocre managers. A familiar example is when a great engineer gets promoted to tech lead but does a poor job. Yet a company lost a great engineer with that move.

For a healthy organization, it's essential to mitigate this situation. For example, not every senior engineer should become a manager (some might advance as staff or principal engineers instead), and new managers need training to be competent in that role.

The Peter Principle warns that *promotion systems can create mediocrity at higher levels* if not designed carefully.

Origins

Described in the book *The Peter Principle* by Laurence J. Peter in 1969, which was meant as a humorous piece of sociology/business insight. Scott Adams's *Dilbert Principle* later riffed on it (inverting it cynically, that companies promote incompetents to get out of the workflow). Both laws highlight absurdities in corporate culture.

Examples

In a development team, Alice is the best programmer. After some time, management appoints her team manager, as she is the most experienced. She has no experience or desire for people management or project leadership. Consequently, the team's performance is in trouble. She either micromanages the team or misses essential management tasks, and in the meantime, she's coding less (her main strength). In this management position, Alice has reached her "level of incompetence," and according to the Peter Principle, she might remain in that role indefinitely.

Or consider a QA engineer who is fantastic at finding bugs and promoted to QA Lead, a role that involves lots of paperwork and strategy. If those aren't their strengths, the QA process might decline.

Many tech organizations now use *individual contributor (IC) career tracks* to combat this issue, allowing great engineers to remain engineers (with raises and titles) rather than forcing them into management to advance.

The principle can also encourage rotations or a try-before-you-promote approach, that is, acting in leadership roles to ensure someone is competent *before* permanently assigning them.

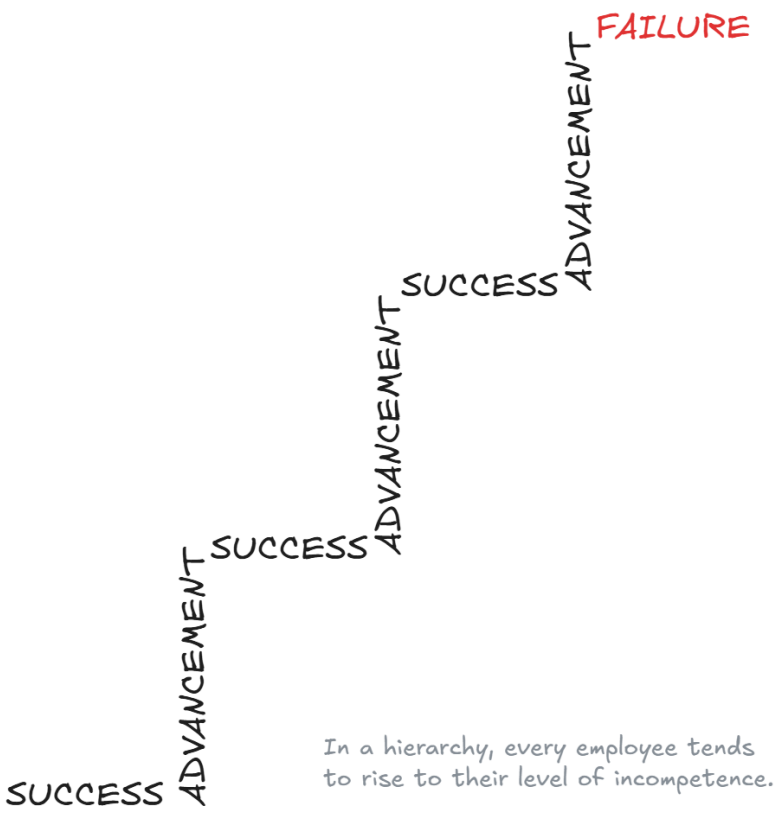

Figure 16.1 **The Peter Principle**

Related to

- The Dilbert Principle
- Putt's Law

Further Reading

- Peter, L. J., & Hull, R. (1969). *The Peter principle: Why things always go wrong.* William Morrow and Company.
- Peter principle. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Peter_principle

OceanofPDF.com

17. The Bus Factor

The Bus Factor is the number of team members who, if hypothetically hit by a bus, would cause the project to be in serious trouble.

Key Takeaways

- *A Bus Factor of 1 means one person holds critical knowledge; if they disappear, the project is essentially doomed or stalled. A higher bus factor (e.g., 5) means the project could lose any one of five specific people before stopping work.*
- *It's basically a measure of knowledge distribution and risk. A high Bus Factor is good (knowledge is shared among many), while a low one is bad (single points of failure in expertise).*
- *Teams should work to increase their Bus Factor by sharing knowledge, documenting critical systems, having code reviews, and rotating responsibilities.*

Overview

It is one of my favorite laws, which highlights the *human single point of failure in projects*. In many software teams, one or two people might understand the legacy system, or a crucial algorithm, or have all the deployment knowledge. If those people leave, others cannot easily pick up the work.

For example, in a typical codebase with 50 engineers working on a big system, the Bus Factor is usually low, at 2–3 people.

The concept encourages actively avoiding such dependency. For instance, open source projects evaluate their health partly by how many maintainers can independently drive it. A project with a Bus Factor of 1 (i.e., there is only 1 active maintainer) is fragile.

The Bus Factor is the size of the smallest set of people who collectively have all the critical knowledge.

Improving the Bus Factor overlaps with knowledge management practices. This means identifying areas that could be affected, implementing pair programming, using wikis, and providing mentorship.

Origins

The concept was popularized in the 1990s (in discussions of project risk). It's not attributed to a single person, but it emerged as slang in software teams. An early reference is when Michael McLay asked publicly in 1994, what would happen to the Python language if its author (Guido van Rossum) were to be hit by a bus.

The concept is also mentioned in the *Organizational Patterns* book (2004), by Coplien et al.

The idea likely existed informally even earlier ("what if so-and-so gets hit by a bus?" as a way to plan for continuity).

Examples

If Alice is a startup's only database expert, the Bus Factor for database knowledge is 1. If Alice quits, nobody else knows the backups, schema intricacies, or tuning, and it's very problematic.

A more positive example is when a team practices pair programming and collective code ownership. As a result, no single file or subsystem is understood by only one person. That project might have a Bus Factor of, say, 5 or 6, meaning up to five people could be lost before critical knowledge is lost.

In open source, the Linux kernel has a relatively high Bus Factor in many areas (many maintainers), but some subsystems might have just one maintainer (low bus factor).

There have been real cases: In the Left-pad incident, a single maintainer (Azer Koçulu) pulled a tiny NPM package (left-pad, that padded strings with characters on the left side), and thousands of builds broke. Babel, the JavaScript compiler, React, and many others could not be built. That reflects a Bus Factor of 1 for that utility on the broader ecosystem.

Another example I saw in my practice was when a company realized that only one engineer knew how an old billing system worked. But that person was soon on a longer vacation, so the problem emerged fast. They mitigated this by cross-training another engineer, raising the Bus Factor from 1 to 2 (still low, but better).

This law is often described best by asking, “If person X were out, could we still ship?” for each team member.

Figure 17.1 **The Bus Factor**

Related to

- Conway's Law

Further Reading

- Bus factor. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Bus_factor
- Coplien, J. O., & Harrison, N. B. (2004). *Organizational patterns of agile software development*. Wiley.
- McLay, M. (1994, June 29). *If Guido was hit by a bus? Python mailing list (python-1994q2)*. https://legacy.python.org/search/hypermail/python-1994q2/1040.html
- Npm left-pad incident. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Npm_left-pad_incident

- Webster, B. F. (2008, April 11). *The wetware crisis: The Dead Sea effect*. https://brucefwebster.com/2008/04/11/the-wetware-crisis-the-dead-sea-effect/

Sidenote: The Dead Sea Effect

When discussing the Bus Factor, it is essential to mention the Dead Sea Effect. This effect says that the best engineers leave companies first. And this is dangerous to a company.

Bruce F. Webster coined this pattern in 2008. In dysfunctional organizations, talent vanishes while mediocrity accumulates, just like the Dead Sea, where water escapes but salt remains. Skilled engineers have options and won't tolerate dysfunction for long. Those who stay are often the ones who can't easily find work elsewhere.

A team suffering from the Dead Sea Effect has knowledge concentrated in the wrong people. When these residue employees finally leave, the organization discovers its institutional knowledge has never been institutional at all.

OceanofPDF.com

18. The Dilbert Principle

Companies tend to promote incompetent employees to management to limit the damage they can do.

Key Takeaways

- *The Dilbert Principle humorously suggests that some organizations deal with underperforming employees by promoting them into management roles, effectively removing them from hands-on work where they could cause harm.*
- *It highlights a cynical view that lower-level employees do real productive work, while managers (especially those promoted under this principle) add little value to the actual output.*
- *It also emphasizes bad alignment between technical and management work.*

Overview

I once heard from my manager that his boss is "*promoted to a better position where he cannot do much damage to the organizatio*n," and that was exactly the Dilbert Principle.

Instead of addressing poor performance directly, companies often promote people who struggle in current roles into management roles, where their impact is perceived as less immediately harmful.

The Dilbert Principle satire sounds right, because it reflects a fundamental flaw: Many organizations treat management as the

default career path for engineers, or the upgrade of it, but they don't care about what they really want. Technical excellence and people leadership require different skills. Failing at one does not qualify someone for the other.

When promotion replaces accountability, management layers fill with people lacking either technical credibility or leadership ability. The result is many bad decisions, wrong priorities, and loss of trust between engineers and leadership.

This principle isn't an indictment of managers but a critique of systems that conflate seniority, tenure, and leadership capability.

Origins

Scott Adams's Dilbert Principle was created by the author in his 1996 publication, "The Dilbert Principle." While, of course, the approach is an exaggeration typical of much of the Dilbert series, the workplace commentary rings true for many engineers.

Examples

In practice, does the Dilbert Principle really happen? Many engineers and tech employees will share anecdotes that feel straight out of *Dilbert*.

Even when not done intentionally, the tech industry often provides *accidental examples* of the Dilbert Principle. Think of the scenarios where a mediocre programmer or a failing project lead is promoted to a management or architect role. Sometimes these moves are less about recognizing leadership talent and more about *getting a*

problematic person away from critical work. The result can be comically counterproductive: You now have an incompetent manager and one fewer developer.

In one of my previous companies, a person (let's call him Marcus), was a director of E-commerce operations. His job was sloppy, site uptime dropped, and his team missed a lot of deadlines. Leadership decided to promote him to VP of Digital Transformation, a role that doesn't have deliverables or technical requirements, with a job description "leveraging emerging technologies to revolutionize customer experience." Here, his incompetence could be safe.

In the next many months, Marcus created a lot of meetings, alignments, and studies of machine learning, even though he didn't understand it. Engineers from the team shipped features, while Marcus took credit for "strategic vision." When a new wave of restructuring killed his role, the company made him "special advisor," with no reports or specific responsibilities.

They never fired him but just kept inventing jobs where he couldn't cause any damage.

On a more positive note, awareness of the Dilbert Principle has arguably made some organizations more careful. Modern tech companies increasingly offer dual career tracks (technical vs. managerial) to avoid forcing great engineers into management roles.

This move helps prevent the Peter Principle scenario and also avoids using management as a dumping ground. When a truly underperforming employee can't hack it in engineering, *good*

companies will try coaching or reassigning them, and if all else fails, let them go, rather than make them the boss.

Figure 18.1 **The Dilbert Principle**

Related to

- The Peter Principle
- Putt's Law

Further Reading

- Adams, S. (1996). *The Dilbert principle: A cubicle's-eye view of bosses, meetings, management fads & other workplace afflictions*. HarperBusiness.
- Dilbert principle. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Dilbert_principle

OceanofPDF.com

Part III
Time, Estimation, and Planning

This set of laws describes optimism bias, schedule expansion, and the mechanics behind underestimation.

OceanofPDF.com

19. Hofstadter's Law

It always takes longer than you expect, even when you take into account Hofstadter's Law.

Key Takeaways

- *Humans are generally bad at estimating duration of tasks, especially in complex projects. Even when we know we're bad at something, we still underestimate.*
- *There are always unknowns and complexities that reveal themselves during implementation, extending the timeline.*
- *Because of this law, we should include contingency buffers in schedules. But even those buffers often get eaten up (it's recursive, as the quote jokes).*
- *Don't be overly optimistic in planning. Acknowledge that delays happen and factor that into timelines and expectations.*

Overview

This recursive law captures the paradox of estimation. No matter how much experience we have, *projects tend to run late*. Even if you say, "This might take 2x longer than I think," it might still take 3x. The concept highlights the inherent uncertainty in complex creative work, such as software development. There are often hidden tasks, unplanned integration issues, or requirement changes.

In practice, Hofstadter's Law explains why techniques such as padding estimates, awareness of Parkinson's Law, and the use of

historical data are essential, yet surprises still occur.

It pairs with Parkinson's Law as a caution: Don't pad too much (Parkinson will waste time), but also don't be naive (Hofstadter will bite you with delays).

My rule of thumb here is

- If it takes 2 minutes → do it now (don't estimate).
- If it takes "a few minutes" → call it 1 hour.
- If it takes "a few hours" → call it 1 day.

We are not working to prove speed, but to buy enough space to ship well. So stay focused, and be proud of the result.

Origins

Douglas Hofstadter created this law in 1979 in his book *Gödel, Escher, Bach*.

In the domain of software engineering, Fred Brooks's observation that *everything takes longer than it takes* (in *The Mythical Man-Month*) has a similar meaning.

The law has become common among developers who've all experienced projects that overshoot schedules.

Examples

Virtually, every software project can serve as an example, and I saw this law in almost all projects I worked on. Let's say a team plans a new feature, expecting it to take a month. They factor in potential delays and say 6 weeks to be safe. It ends up taking 3 months,

sometimes even 6 months later. Why? Integrating with an API was harder, or a key developer got sick, or the first approach failed and had to be redone. *The unexpected became the norm*.

One well known example is Berlin Brandenburg airport, due to be opened in 2011. The building itself included integration of 75,000 sensors, 50,000 light fittings, and fire safety measures. The team estimated that it would take around 18 months to integrate different software. The project management, assessed it, understood that it was unreal, and increased the estimation to 30 months.

In the end, the complete job took 7 years to complete. The airport opened in 2020, 9 years past the due date with €7 billion over budget (supposed to be €2.83 billion).

This shows clearly the Hofstadter's Law at work. The team was not only aware of the problem, but they failed to avoid it.

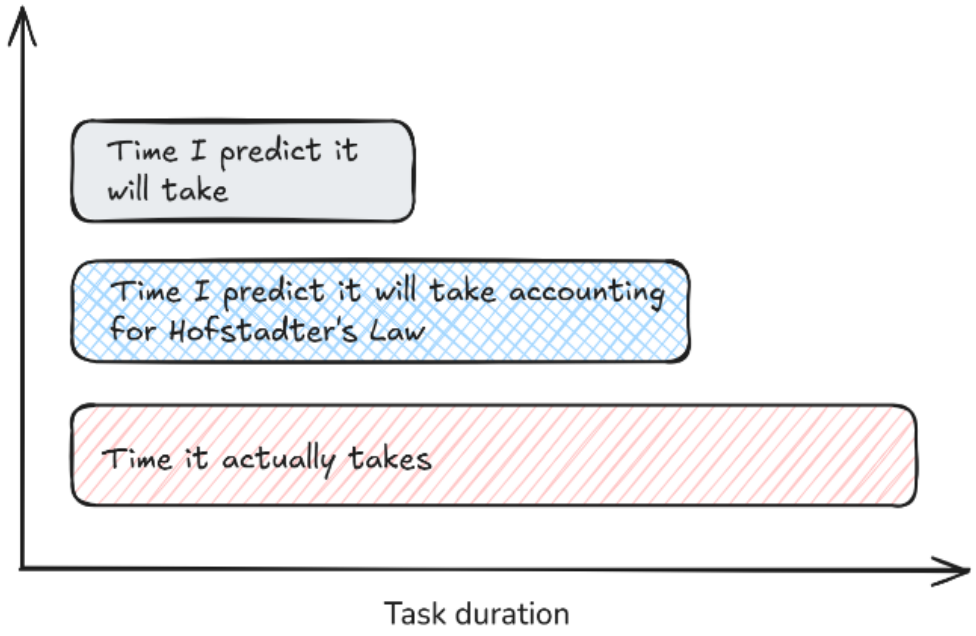

Figure 19.1 **Hofstadter's Law**

Related to

- Parkinson's Law

Further Reading

- Brooks, F. P., Jr. (1995). *The mythical man-month: Essays on software engineering* (Anniversary ed.). Addison-Wesley.
- Bernhardsson, E. (2019, April 15). *Why software projects take longer than you think: A statistical model.* https://erikbern.com/2019/04/15/why-software-projects-take-longer-than-you-think-a-statistical-model.html
- Hofstadter, D. R. (1979). *Gödel, Escher, Bach: An eternal golden braid*. Basic Books.

OceanofPDF.com

20. Parkinson's Law

Work expands to fill the time available for its completion.

Key Takeaways

- *If you give a task an overly long timeline, people tend to use all of it (or procrastinate until the last minute), so the task takes as long as the deadline allows.*
- *Giving more time, teams often do polishing (gold-plating) or add minor improvements that aren't strictly necessary, just to use the whole time allotted for the task.*
- *A bit stronger but still realistic deadlines can counteract Parkinson's Law by having a sense of urgency (called deadline-driven development).*

Overview

This law shows a common problem with time management among developers. If a developer is given 2 weeks to complete a task that could be done in 2 days, the work will usually slow down, consuming most of that time. Sometimes it may take even more. People might spend more time planning, bikeshedding minor details, or simply delaying the start of the work, consciously or not, but with the understanding that they have enough time given. This is a part of human nature.

This is a function of the Iron Triangle. Scope, time, and cost cannot be varied independently. If scope is allowed, time will be extended to accommodate it.

Parkinson's Law is often used to warn that loose deadlines reduce productivity, so teams should set clear, realistic time limits.

However, managers must use it judiciously, combining Parkinson's insight with realistic scheduling. If you compress timelines too much, you risk running into Hofstadter's Law, which reminds us that work often *still* takes longer than expected, even with buffers.

Origins

Parkinson's Law was first mentioned in an essay by Cyril Parkinson, published in *The Economist* in 1955, which initially described how bureaucracies expand over time.

The phrase has since entered the lexicon of project management. It's not specific to software, but software teams often see it in action during projects.

Examples

Let's say that a developer should write a module and is told, "Take as long as you need, maybe a month or two." In many cases, the developer will not rush to finish it, perhaps experimenting with multiple implementations or polishing noncritical items (bikeshedding). The same task, if given a 1-week deadline, might have been completed in a week.

Another example is with meetings. If an activity is scheduled to last all day, it often will. If the same activity were given a 2-hour window, participants would likely focus and complete it within that time.

Agile teams sometimes apply Parkinson's Law by *time-boxing tasks* (deadline-driven development). Those tasks should be visible and shared with the team. The fixed, short time box encourages everyone to focus on completing tasks before the sprint ends. Also, writing and sharing reports weekly can improve accountability and the overall process.

It's essential to remain aware of the counterpoint to Hofstadter's Law: We should not set *impossibly* short deadlines, as you will consistently be late or burn out your team.

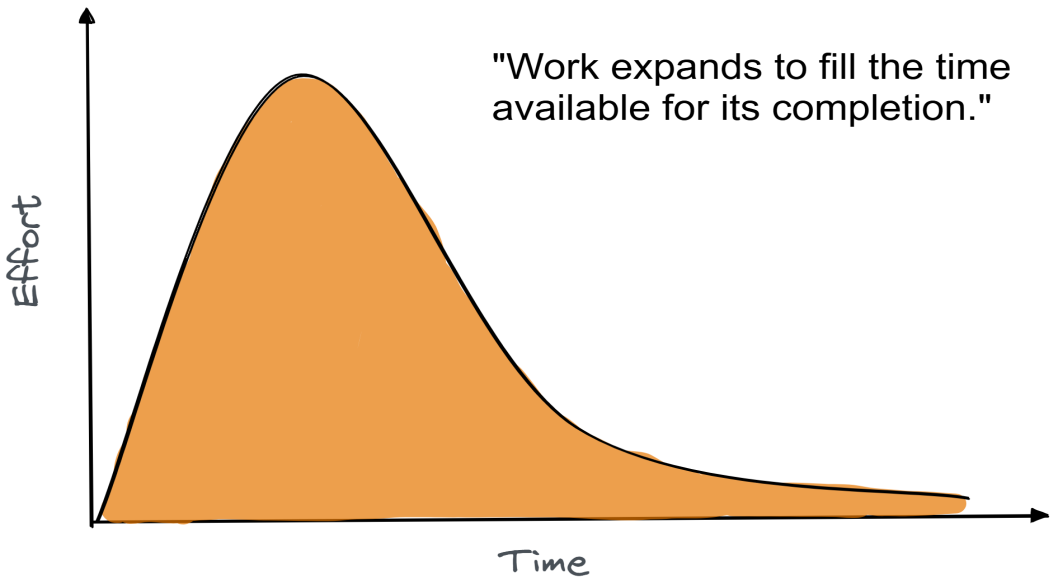

Figure 20.1 **Parkinson's Law**

Related to

- Hofstadter's Law

- Brooks's Law

Further Reading

- Parkinson, C. N. (1955, November 19). *Parkinson's law. The Economist*. https://www.economist.com/news/1955/11/19/parkinsons-law
- Parkinson, C. N. (1957). *Parkinson's law, or the pursuit of progress.* John Murray.
- Parkinson's law. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Parkinson%27s_law

OceanofPDF.com

21. The Ninety-Ninety Rule

The first 90% of the code accounts for the first 90% of development time. The remaining 10% of the code accounts for the other 90% of development time.

Key Takeaways

- *The final parts of a project (polishing, edge cases, integration, bug fixes) often take far more effort than anticipated, often as much as the initial development.*
- *What looks like the finishing touches involves many tricky problems and unknowns.*
- *Reaching a "mostly done" state can create optimism, but the rule warns that "almost done" often means only halfway in terms of time.*

Overview

This rule quantifies the extent to which projects get stuck in the final phase. Often, teams make good progress at the start, building core functionality, which leads to optimism. Then integration, corner cases, performance tuning, and bug fixing consume an *unexpectedly large amount of time*, often equal to or greater than what's already been spent.

In practical terms, this rule advises that *the last bit of a project is not a small bit*. You should plan for a significant effort in finishing, polishing, and delivering.

It's also related to Hofstadter's Law, as it's another way our estimates fall short. We don't realize how hard that last leg is.

Origins

This rule is credited to Tom Cargill of Bell Labs and was popularized by Jon Bentley's September 1985 "Bumper-Sticker Computer Science" column in *Communications of the ACM*. It was called the "Rule of Credibility" there, but the name has not stuck.

The rule became popular in programming circles and is frequently referenced in discussions of why software is usually delivered late or why "90% done" claims can be wrong.

Examples

Let's say that a team develops a new app. In 3 months, the core features will be done, and perhaps they will hit 90% of the functionality. Everyone expects to ship in 1 more month. Instead, integration testing reveals that multiple modules don't talk correctly, specific edge inputs crash the app, memory usage is too high, etc. That final 10% (bringing it to production quality) takes another 3 months.

Another example is when implementing a new website, you get a basic version up quickly (login, basic pages). The last bits, cross-browser fixes, responsive design adjustments, accessibility improvements, writing tests, and performance improvements, seem like 10% of the work. Still, they take as much time as the main coding.

In 2018, a DEV.to engineering team was creating their first iOS application (wrapper over web version), and 90% of the work went fine, but the final 10% was problematic. When navigating tabs, deep linking was a problem, as was scroll position retention, offline usage, pushes, coordination between the web views, basically, the native navigation. Nine days after project completion, the team lead wrote the following: "I'm seeing the last 10% of the existing approach as possibly resulting in another 90% of work." And this was the right decision. Instead of extending their development cycle for another cycle, they decided to keep it simple by creating a shell, with a perfectly optimized web view and native features such as push notifications and deep linking. The last 10% (finishing, putting the touches on, and making it production ready) would take longer than all of the work up to now.

The rule is an important reminder: Don't celebrate too early! So, we can say, "We're 90% done . . . and the other 90% is still to go."

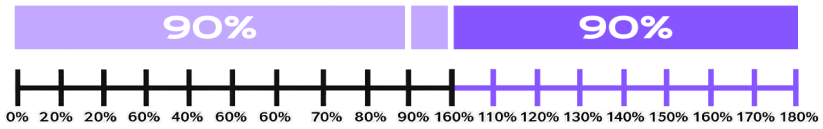

Figure 21.1 **The Ninety-Ninety Rule**

Related to

- Hofstadter's Law
- The Pareto Principle

Further Reading

- Ben. (2019, March 20). *Identifying and mitigating the ninety–ninety rule in software development. Dev.to.* https://dev.to/ben/identifying-and-mitigating-the-ninety-ninety-rule-in-software-development-4ap3
- Bentley, J. (n.d.). Programming Pearls: Bumper sticker computer science. https://moss.cs.iit.edu/cs100/Bentley_BumperSticker.pdf
- Bentley, J. (1985). Programming pearls: Bumper-sticker computer science. *Communications of the ACM, 28*(9), 896–901.

OceanofPDF.com

22. Goodhart's Law

When a measure becomes a target, it ceases to be a good measure.

Key Takeaways

- *If you set a particular metric as a goal (e.g., lines of code written, number of features closed), people will find ways to optimize for that metric.*
- *Metrics are proxies for what you value (productivity, quality, etc.). Once they're targets, people meet the metric even if it undermines the original intent.*
- *Metrics are useful for insight, but they must be used in context and balanced with qualitative judgment. Also, combine multiple metrics to avoid a singular focus.*

Overview

Goodhart's Law comes from economics and is very much relevant to software teams. For instance, a manager might set a target that "we must close 100 bug tickets this month." Developers, feeling pressure, might start closing tickets that aren't truly resolved or splitting a single bug into multiple tickets to inflate numbers.

The metric (tickets closed) goes up, but software quality might not.

Making the metric a goal distorts the process, so it loses its correlation with absolute success.

Origins

It is named after economist Charles Goodhart and originally about monetary policy metrics. In tech/management circles, Marilyn Strathern's phrasing is often cited: "When a measure becomes a target, it ceases to be a good measure."

This concept has been discussed in the context of education and business KPIs, and it certainly applies to software engineering metrics.

Examples

In my career, I saw a situation in which the software team was rewarded based on the lines of code it wrote (*direct output*). The management, who were not experts in software development thought that more code equaled more productivity. In addition to writing more code, developers also start copying and pasting it into multiple places (violating DRY) to increase the line count. The software becomes bloated and unmaintainable. I remember well one of the developers removed 1,500 lines of code making it faster, but this was considered unproductive, and management decided that this person was doing a bad job.

In today's world we see something similar with *AI tokens consumed*, where more tokens is considered better, and even become one of the goals that people need to fulfill for their performance reviews.

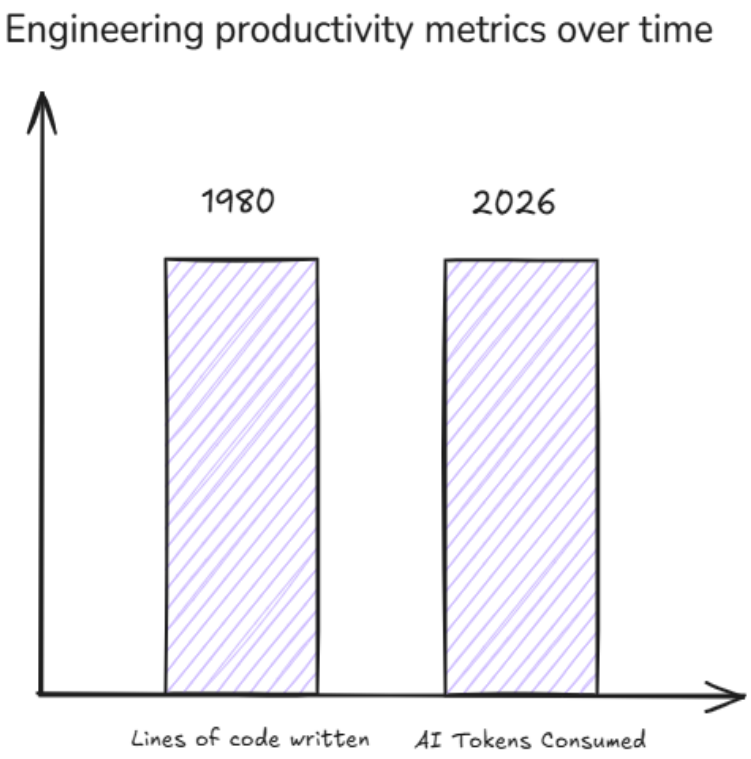

Figure 22.1 **Engineering productivity over time**

Another good example is a QA team that measured the number of test cases executed as its primary goal. They favored running many easy tests rather than doing proper testing to meet objectives. Yet, this approach misses many critical bugs.

Another common scenario we see in development teams is when teams focus on *code coverage percentage* as a primary target for code quality. Developers may write many trivial unit tests that don't assert meaningful behavior to raise coverage. On the other side, necessary integration tests (which are harder to write) are missed. Here, the metric (coverage %) increases, but neither confidence in the code nor bug counts improve.

This same process plays out at the level of the platform itself, which Cory Doctorow described as “enshittification.” The process begins with the optimization of user experience to drive adoption. However, once the platform achieves a large number of users, it changes focus to metrics such as user engagement, ad revenue per user, and so on. The optimization of the product is done at the expense of the very reason it became successful in the first place: the user experience. The feeds are filled with adverts, searches favor those who pay over those who are relevant, and features are paywalled. However, everything looks okay because users are more engaged and ad revenue per user is increasing. The product, however, becomes measurably worse.

This is Goodhart’s Law applied to an entire product and business model: When user engagement becomes a target, rather than simply a measure, the platform will immediately begin to do things that increase it yet make it measure something different.

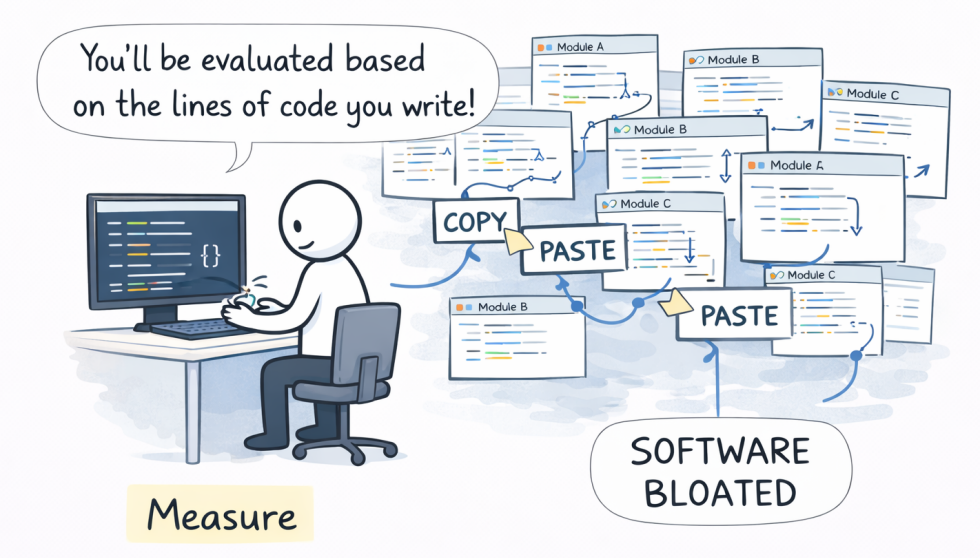

Figure 22.2 **Goodhart’s Law**

Related to

- The Dilbert Principle
- Parkinson's Law

Further Reading

- Chakraborty, A. (n.d.). *Goodhart's law: How measuring the wrong things drive immoral behaviour*. Coffee & Junk. https://coffeeandjunk.com/goodharts-campbells-law/
- Goodhart's law. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Goodhart%27s_law
- Enshittification. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Enshittification
- Muller, J. Z. (2019). *The tyranny of metrics.* Princeton University Press.

Sidenote: The Cobra Effect

British colonial India paid bounties for dead cobras to reduce the snake population. In turn, people started breeding them for the reward. When the program was cancelled, breeders released their worthless snakes into the wild. The population ended up larger than before.

This is Goodhart's Law at its most destructive. The incentive didn't just fail to help, but it actively created the problem it was designed to solve.

The difference might matter when you're setting targets for your team. Goodhart's Law warns that metrics lose meaning when they become goals. The Cobra Effect warns that bad incentives can multiply the very problem you're measuring. Before rolling out any metric-based reward, ask yourself: "Could gaming this make things worse than doing nothing at all?"

If so, you are but one policy change away from raising cobras.

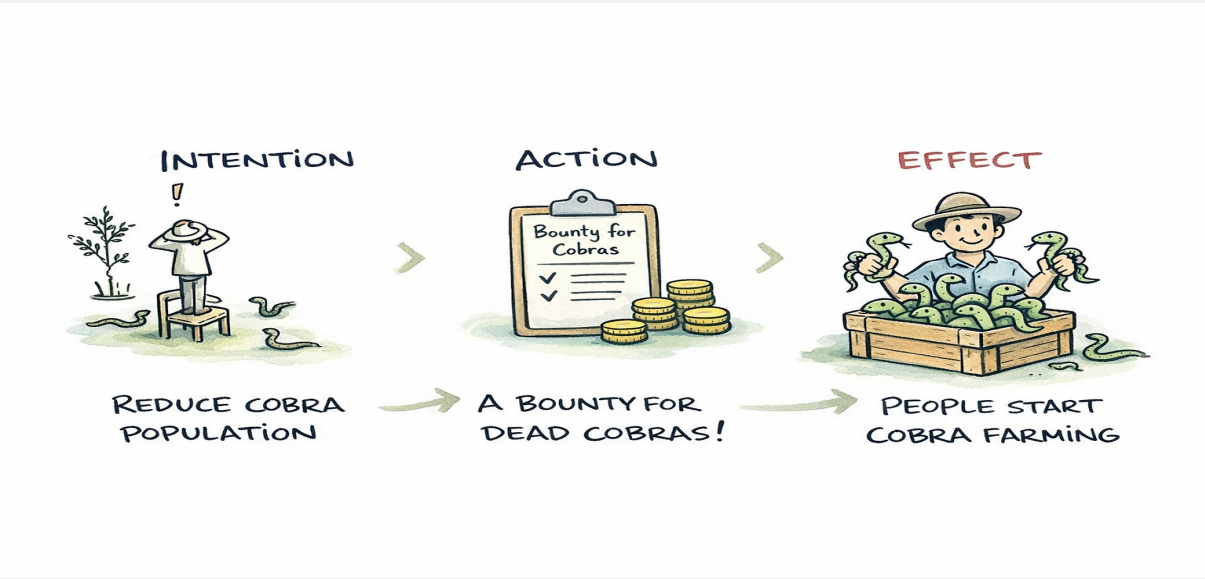

Figure 22.3 The Cobra Effect

OceanofPDF.com

23. Gilb's Law

Anything you need to quantify can be measured in some way that is superior to not measuring it at all.

Key Takeaways

- *It's better to have some data or metric on a phenomenon than to be completely blind, as long as you understand the metric's limitations.*
- *In contrast to Goodhart's Law, which warns about the misuse of metrics, Gilb's Law reminds us not to throw out metrics entirely. It is possible to find ways to gauge progress/quality meaningfully.*
- *Start with a basic measure and refine it over time. The act of measuring helps teams focus and identify trends.*

Overview

Gilb's Law responds to the paralysis that Goodhart's Law can cause. This law asserts that *even an approximate or indirect measurement is better than none*. When something is essential (performance, customer satisfaction, code maintainability), you should attempt to measure it, because otherwise you have no objective feedback.

Gilb's Law is a good reply to the statement "this aspect is unmeasurable so we won't try." For example, measuring code quality is difficult, but you can measure indicators such as cyclomatic

complexity, lint warnings, or defect rates as partial indicators. More such indicators can paint the bigger picture.

Those metrics won't be perfect, but Gilb's Law suggests that having them gives you some insight and a starting point for improvement, which is better than having no clue at all.

The law encourages a pragmatic approach to measurement: Find something to measure related to your goal and use it rather than giving up on measurement.

Origins

Tom Gilb, a consultant and author on software engineering, formulated this concept. It complements his other work on quantifying requirements and using metrics in planning (Planguage, etc.).

Examples

Measuring *developer productivity* is notoriously hard (lines of code are poor proxies, story points can be inconsistent, etc.). However, you might use *deployment frequency* or *change lead time* (as in the DORA metrics) as a proxy. They don't capture everything, but they give you some data to follow. For instance, if deployment frequency decreases, something might be wrong with the pipeline.

Another example could be *tracking Technical Debt*. No perfect measure of Technical Debt exists. But tracking things such as code complexity scores, incident rates, and developer surveys gives you visibility you wouldn't otherwise have.

One real-world use case is when the U.S. Army's Persincom personnel information system hit the wall during the First Gulf War. It happened when General Norman Schwarzkopf did not get the response he needed from the system. They were ready to spend years and millions of dollars on building an even better system, which is the typical reaction of the "large, plan-driven" solution approach, where the problem is vaguely defined (i.e., "build a better system"). Gilb's evolutionary approach came in when the Persincom information system's performance requirement was defined in quantified terms. What was the most critical measurable result? It was the response time needed for handling high-priority information requests. When this was done, it turned out that "shocking simplicity" characterized the solution that would fix the problems. They would reprogram the old system to recognize the concept of ranks and put the high-priority requestors at the head of the line. They estimated that 50% of what was needed would only take 1% of what was anticipated for building an even better system.

This realization was unclear until the team quit discussing just a "better system" and began to understand exactly what "better" means. Such requirements are tracked in this project (11 of them across 6 design strategies), such as availability requirements ranging from 90% up to 99.5% uptime, a decrease in user requests for support from 200 to 60, data integrity ranging from 88% to 97%, etc.

At its core, Gilb's Law shows that improvement depends on measurement.

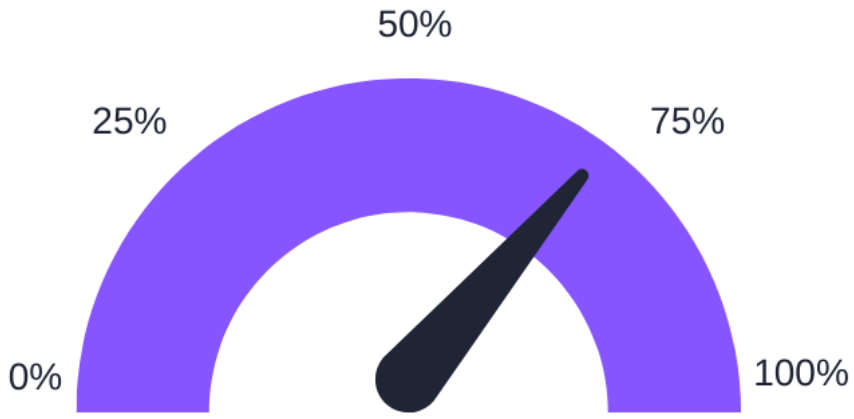

Figure 23.1 **Gilb's Law**

Related to

- Goodhart's Law
- Parkinson's Law

Further Reading

- Gilb, T. (2005). *Competitive engineering: A handbook for systems engineering, requirements engineering, and software engineering using Planguage.* Butterworth-Heinemann.
- Harvey, N. (2025, March 5). *DORA's software delivery metrics: The four keys.* DORA. https://dora.dev/guides/dora-metrics-four-keys/
- Tom Gilb. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Tom_Gilb

OceanofPDF.com

24. Knuth's Optimization Principle

Premature optimization is the root of all evil.

Key Takeaways

- *Most code doesn't run in performance-critical hotspots, so obsessing over micro-optimizations everywhere wastes time and makes code harder to read and maintain.*
- *According to Knuth, we should forget about small efficiencies about 97% of the time and focus on clean design and correct functionality.*
- *Optimized code is often more complex or less readable. If done prematurely, you incur this cost even when it's unnecessary.*
- *We should get it working correctly first, then make it fast, then make it pretty.*

Overview

Knuth's Optimization Principle captures a fundamental trade-off in software engineering: Performance improvements often increase complexity. Applying that trade-off before understanding where performance actually matters leads to unreadable systems.

Early in development, your focus should be on a clear design. If you optimize too soon, you might introduce bugs or inflexibility, all to speed up parts of the code that may not even be bottlenecks. It's often cited that *20% of the code may consume 80% of the execution*

time (a Pareto-like notion). Optimizing anything outside that critical 20% is wasted effort and adds risk.

The principle advises writing simple code, then profiling and improving only the parts that truly need it. Knuth said that there *are* cases where you know up front that something is critical to performance. In such cases, you might design that part carefully, but those are the rare 3% of cases, not the majority.

Origins

Donald Knuth, a legendary computer scientist, made this statement in his 1974 paper “Structured Programming with Go To Statements.” It has since been reprinted and quoted widely in programming literature.

It’s important to mention that Knuth didn’t literally mean all evil, but the quote caught on because it resonates with developers who have seen codebases twisted by needless optimizations. Over time, this quote became almost a proverb in software engineering. It’s taught to juniors that you need to first make it work, then make it right, then (if needed) make it fast.

The “97% of the time” suggests that, indeed, only a small fraction of code warrants intense optimization.

Examples

A classic example is when a developer writes a low-level C routine with complex bit-twiddling to squeeze out speed, spending 2 days on it, only to find that the function is called once at startup, taking

0.001% of runtime. The effort was wasted, and the code is now more complex to understand. Meanwhile, another part of the program (such as a sorting function on a large dataset) was the real bottleneck but remained unoptimized because time was spent on the wrong thing.

Another example is when prematurely choosing a complex data structure for theoretical efficiency (say, a custom tree for log(N) lookups) when the simpler approach (like a linear search) would have been acceptable for the data sizes involved. The complex structure introduces bugs and overhead.

In contrast, following Knuth's advice, you implement simple first, then measure, finding that, for example, *80% of time is spent in one loop*. Then you *optimize that loop or use a better algorithm there*. This optimization gives real improvements with minimal added complexity elsewhere.

Once, in a startup, we were developing a brand new web service. The startup was obsessed with optimizing the performance and scalability of the infrastructure before it even acquired any users (with Kubernetes, yes). We spent weeks designing for millions of users before we acquired even 10 of them. As one of the developers later quipped, "Before you worry about handling millions of users, you need to make sure that 100 users even like and want to use your product."

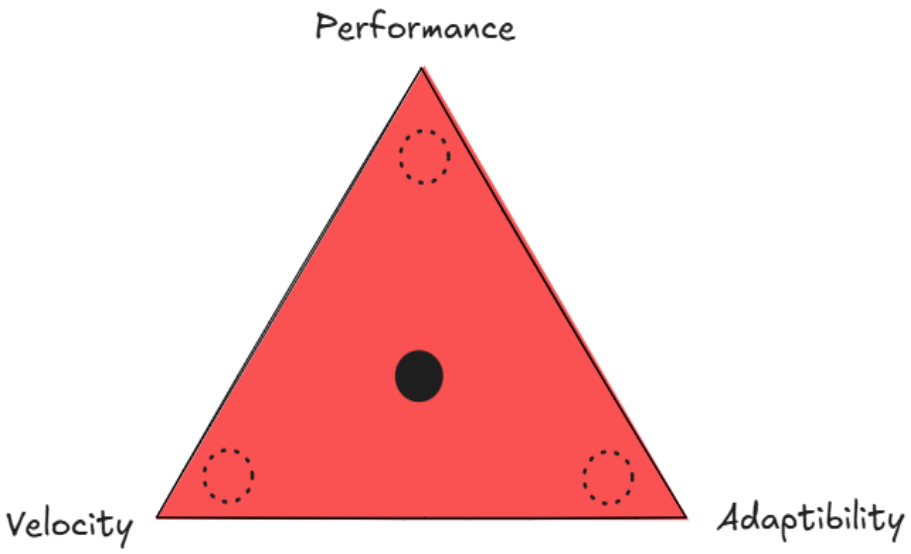

Figure 24.1 **Premature Optimization**

Related to

- Kernighan's Law
- YAGNI
- Amdahl's Law
- Hofstadter's Law

Further Reading

- Knuth, D. E. (1974). Structured programming with go to statements. *Computing Surveys, 6*(4), 261–301.
- McConnell, S. (2004). *Code complete* (2nd ed.). Microsoft Press.
- Program optimization. (n.d.). Wikipedia. https://en.wikipedia.org/wiki/Program_optimization

OceanofPDF.com

Part IV
Quality, Maintenance, and Evolution

These laws cover defect dynamics, maintainability, testing limits, and the trends followed by long-lived systems.

OceanofPDF.com

25. Murphy's/Sod's Law

Anything that can go wrong will go wrong.

Key Takeaways

- *If an error can happen, it will happen. Keep this in mind when planning and code defensively.*
- *Taking this law into account, we add error handling, backups, and checks. Treat any single point of failure as a guaranteed future failure.*
- *Edge cases will occur in production. Write tests for even unlikely scenarios.*

Overview

Murphy's Law is often used in software to explain bugs and production incidents: Whatever *can go wrong in code* (a null pointer, a race condition, a network outage) *will eventually manifest*, especially in large user bases or at the worst possible time.

In practice, this law encourages developers to write more *defensive code*. This means checking for nulls, handling exceptions, validating inputs, and failing gracefully when errors occur.

It also reminds DevOps teams to anticipate failures by implementing monitoring, enabling rollbacks, and maintaining contingency plans.

While it’s a pessimistic principle, it underlies many best practices, such as

- Back up your data, because if data loss is possible, someone will accidentally delete something someday.
- Don’t deploy on Friday, because if something bad happens, it’ll happen right before the weekend.

Origins

The law is attributed to Edward A. Murphy Jr., an engineer working on rocket sled experiments in 1949. It became popular in aerospace first and then spread everywhere. In software, it’s been around as long as bugs have, constantly reminding us that if there’s one untested scenario, one user will find it.

Examples

If a web form field can accept text, someone will enter a 10,000-character string of weird symbols to see what happens, unless you’ve explicitly handled it. If memory can run out, it will be when multiple processes align so.

A famous real-world instance was during a live demo (Who remembers Windows 98 presentation by Bill Gates?): If something can glitch, it likely will, that’s Murphy’s Law.

Here is one more recent example. On July 19, 2024, a small configuration update to the Falcon Sensor software had an error that produced out-of-bound memory read. The result was 8.5 million Windows operating systems worldwide showing a Blue Screen of

Death. This update was done automatically. What is worse, it happened on Friday morning with minimal IT staff and affected large infrastructure (airlines, hospitals, banks). Fixing it required deleting a single file, which required hands-on manual access to all 8.5 million systems, because the remote access was not possible (Windows didn't boot).

In coding, consider a function that assumes an input file exists. Murphy's Law says that one day, that file won't be there or will be corrupted, so your code should handle the file-not-found or bad-data scenario rather than crash. Along with handling it, these edge cases should have been properly tested.

Another typical case is that the server will crash on your only day off (because that's when it's most likely to cause trouble). For this reason, engineers build highly available systems and pager rotations to mitigate Murphy's Law.

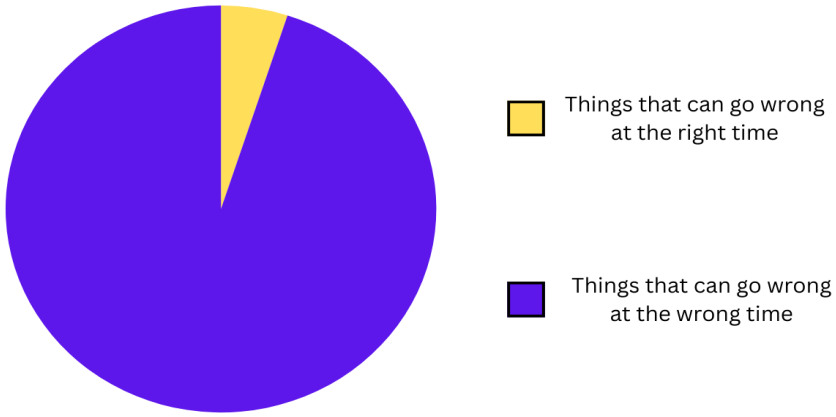

Figure 25.1 **Murphy's Law**

Related to

- Confirmation Bias
- Selection Bias

Further Reading

- CrowdStrike. (2024). *Executive summary: Root cause analysis—Channel File 291 incident*. CrowdStrike Holdings, Inc. https://www.crowdstrike.com/falcon-content-update-remediation-and-guidance-hub/
- Defensive programming. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Defensive_programming
- Murphy's law. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Murphy%27s_law
- Vuusteri. (2021). *Windows 98 presentation fail* (HQ) [Video]. *YouTube*. https://www.youtube.com/watch?v=yeUyxjLhAxU

OceanofPDF.com

26. Postel's Law (The Robustness Principle)

Be conservative in what you do, be liberal in what you accept from others.

Key Takeaways

- *When your system emits data or interacts with the outside world, adhere closely to protocols and standards.*
- *When receiving data, handle variations, minor errors, or deviations where possible. Don't crash or reject communication over minor issues.*
- *In modern times, overly liberal acceptance can sometimes mask errors, so this law is occasionally tempered.*

Overview

According to this law, if your server sends HTTP responses, it should format headers exactly per spec. But if your server receives an HTTP request with an uncommon header order or an unusual format, you should still process it rather than drop the connection, as long as you can interpret it safely.

This principle contributed to the Internet's resilience, meaning that different implementations can communicate because each side strives to be compatible.

In software in general, think of file readers: A robust reader can open not-quite-perfect files (e.g., a tolerant XML parser might recover from a minor error), whereas a strict one might refuse. Postel's Law would encourage the former.

However, it has caveats. Being too liberal can allow sloppy producers to proliferate, and in security contexts, accepting malformed input can be risky.

In fact, some argue that *overly tolerant software can cause long-term interoperability problems* (because producers never fix their bugs if everyone tolerates them). Still, as a principle of robustness, it's widely cited.

Origins

Jon Postel wrote the following in the specification of TCP in 1980 (RFC 760): "TCP implementations should follow a general principle of robustness: Be conservative in what you send, be liberal in what you accept."

It became known as Postel's Law or the Robustness Principle and has influenced many protocol designs.

Examples

In web browsers, HTML on websites is often malformed. Browsers, following Postel's spirit, perform extensive error correction and forgive parsing (they'll still render the page even if a tag isn't closed correctly, etc.). If browsers were strict, half the web pages might not display. They are liberal in what they accept (any messy HTML), but

when they send requests or updates, they strictly adhere to HTTP/HTML standards.

Another example is an *email client* that might receive a slightly nonconforming email (e.g., missing a MIME boundary or incorrect newline encodings). A robust client will still attempt to show the email, whereas a strict one might just throw an error. Postel's Law would favor showing the email.

In APIs, say your service expects a timestamp. If it receives a timestamp without a time zone, instead of rejecting, maybe you assume UTC or try to parse it anyway, being liberal in acceptance. But when your service returns data, you always include the time zone (to be conservative/precise in output).

Finally, some *programming languages/compilers* are very strict about syntax (Hello, strongly typed languages!). Others might auto-correct certain things or have implicit conversions (Who said JavaScript?). Postel's Law is usually more about system communication than internal code style.

Note that there's a modern reconsideration: For example, the HTML5 spec codified much of the error handling that browsers already do, arguing that all that liberal acceptance should itself be standardized to avoid ambiguity.

And in security, if an API is too liberal (accepts slightly malformed JSON by guessing what it means), an attacker might exploit differences in how different layers interpret it.

So while applying Postel's Law, one must not compromise security or clarity. But in general, following this law can reduce friction. If your system handles small input mistakes and produces clean output, it'll work smoothly with other systems. It's a good guideline, especially when designing public-facing protocols (read APIs) or file formats.

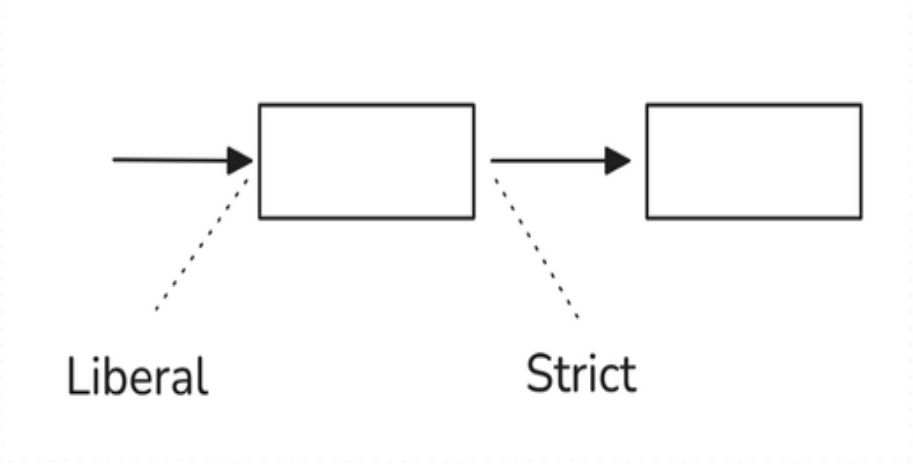

Figure 26.1 **Postel's Law**

Related to

- Hyrum's Law
- The Law of Leaky Abstractions
- Fallacies of Distributed Computing

Further Reading

- Robustness principle. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Robustness_principle
- Thomson, M. (2015). *The harmful consequences of Postel's maxim* (Internet-Draft, draft-thomson-postel-was-wrong-00).

IETF. https://datatracker.ietf.org/doc/html/draft-thomson-postel-was-wrong-00

OceanofPDF.com

27. The Broken Windows Theory

Don't leave "broken windows" (bad designs, wrong decisions, or poor code) unrepaired.

Key Takeaways

- *If you let minor bugs and bad style slide, people assume quality doesn't matter, and they'll ship messier code.*
- *A clean, well-maintained codebase encourages engineers to keep it clean, whereas a chaotic one encourages corner-cutting and further degradation.*
- *Fix problems while they're small. Refactor destructive code, update outdated docs, and so forth to prevent a downward spiral of code health.*

Overview

The Broken Windows Theory in software was popularized by the book *The Pragmatic Programmer* (1999).

In a city, a broken window on a broken car left unfixed signals neglect and invites more vandalism. Also in code, an apparent bug or messy section left untreated can lead to more developers bypassing the process or introducing more mess.

The idea is that *quality problems snowball if left unaddressed*. For example, if a team routinely ignores failing tests or lets linters/errors slide during builds, developers get the message that it's okay to ship

sloppy work. This accelerates code decay (also known as *software entropy*).

Conversely, if a team quickly fixes minor issues and maintains high standards, it creates a *culture of quality*. Over time, the difference is enormous: One path leads to an unmanageable legacy system, while the other leads to a system that can evolve cleanly.

Origins

The Broken Windows Theory comes from criminology (James Q. Wilson and George Kelling). In software, Andy Hunt and Dave Thomas applied it as a metaphor in *The Pragmatic Programmer* (1999). They argued for fixing minor problems promptly.

Jeff Atwood and others in blogs have also discussed this principle in the context of software project maintenance.

Examples

Consider a codebase with a few obvious kludges, such as functions with `// TODO: fix this hack` and comments that never get fixed. If newcomers see that, they may be more inclined to add their own hacks (they will say, “This project is full of hacks anyway…”).

Or if there’s a module with no tests (a “broken window” in a test-coverage sense), other modules might start losing tests, too, as discipline erodes.

Here is a real case from one of our client’s projects I worked on. It was a fintech startup, where we launched MVP, as a team of four members. Everything we did was high quality: clean code, high test

coverage, proper PR reviews. Then, under the pressure, we approved a “temporary” shortcut: We hardcoded a client’s pricing logic into the billing module (with `// TODO: refactor later` comment). This was the first broken window. After a few months, other developers saw this and did similar hardcoding in other modules. Test coverage dropped from 80% to less than 50%. By the time we needed to scale the system, our problems piled up so much that we needed to refactor across the entire codebase. And this took months to do.

In addition, I’ve worked on projects where maintainers aggressively fixed style issues and simplified code whenever they spotted a problem. By contributing to such a project, developers quickly learn to match that style.

Many teams have turned projects around by paying off technical debt and cleaning up code. Once things are polished, they find fewer new bugs are introduced because everyone is more careful.

In summary, clean code tends to stay clean, and messy code tends to get dirtier, so don’t let the first broken window linger.

Figure 27.1 **Broken Windows Theory**

Related to

- Technical Debt
- The Boy Scout Rule
- Gall's Law

Further Reading

- Atwood, J. (2005, June 23). *The broken window theory. Coding Horror.* https://blog.codinghorror.com/the-broken-window-theory/
- Broken windows theory. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Broken_windows_theory
- Thomas, D., & Hunt, A. (1999). *The pragmatic programmer: From journeyman to master.* Addison-Wesley.
- Wilson, J. Q., & Kelling, G. L. (1982). Broken windows: The police and neighborhood safety. *The Atlantic Monthly, 249*(3), 29–38.

OceanofPDF.com

28. The Boy Scout Rule

Leave the code better than you found it.

Key Takeaways

- *Don't leave bad things rot in a project. You should take a moment to refactor or fix at least one small thing in the area you're working on. It can include anything from variable names to simplifying logic.*
- *A cleaner codebase is easier to read, work with, and extend. It also improves project maintainability and reduces technical debt.*
- *When every developer follows this rule, a culture of ownership is created in the code. In addition, teams feel responsible collectively for code quality.*

Overview

The Boy Scout Rule basically means *leaving the code better than you found it*. In practice, it's not about doing big-bang rewrites or achieving perfection in one go. Instead, it's about *constant and incremental improvements*.

Whenever a developer encounters a piece of code, whether adding a new feature or hunting down a bug, they take the opportunity to polish something nearby.

For example, if there's a confusing function name, you rename it clearly. If you notice some duplicated code, you refactor it into a single helper. If a test case is missing for a critical method, you add it. These are often small changes that might take only a few extra minutes but compound over time.

By adopting the Boy Scout Rule, teams also avoid the "broken windows" scenario in software.

As I like to put it, "If you touch it, you own it." Take responsibility for any code you work on, and leave your mark by making it cleaner.

Origins

The mantra "leave the campsite cleaner than you found it" comes from the Boy Scouts of America and Scouting organizations worldwide. This rule helps us understand why taking responsibility for our surroundings is essential.

In the domain of software development, the Boy Scout Rule was popularized by Robert C. Martin (Uncle Bob). He introduced this rule in *Clean Code: A Handbook of Agile Software Craftsmanship* (2008).

Examples

Let's say you're working on an old module in an application to add a new feature. The code works, but you notice a couple of things: There's a function `processData()` that has over 200 lines, and there are some `//TODO` comments, as well as some commented-out code blocks. You could ignore these, add only your feature, and move on. But following the rule, you improve what you can. It means splitting

the `processData()` function into smaller, better-named functions to improve readability. Also, you can delete those commented-out sections that are no longer needed.

On a larger scale, let's take Google, for example. Google's engineers often say, "If you touch it, you own it." This means that when you modify code, you take ownership of its quality, which leads to fixing even minor issues as they arise during unrelated tasks. Over the lifespan of a product, thousands of such small clean-ups occur, preventing major problems.

It's worth noting that the Boy Scout Rule isn't about polishing code endlessly (which is called Yak shaving), but about *responsibility*. If every engineer leaves the code a bit better, the overall system keeps improving. If someone ignores the rule and dumps sloppy code "just to get their task done," they are effectively making the codebase worse for everyone else.

Each minor cleanup is an investment in the team's future productivity, which makes it a proper step in professional software engineering practice.

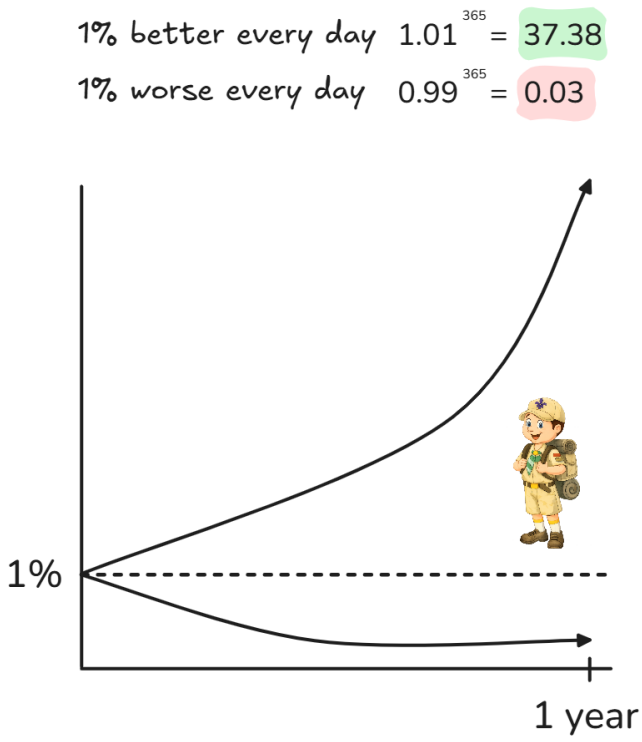

Figure 28.1 **The Boy Scout Rule**

Related to

- The Broken Windows Theory
- Technical Debt
- YAGNI

Further Reading

- Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2nd ed.). Addison-Wesley.
- Martin, R. C. (2008). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

- Martin, R. C. (2010). The Boy Scout rule. In K. Henney (Ed.), *97 things every programmer should know: Collective wisdom from the experts* (pp. 15–17). O'Reilly Media.

OceanofPDF.com

29. Technical Debt

Technical Debt is everything that slows us down when developing software.

Key Takeaways

- *When we take a shortcut in code, we borrow time from the future. This gives an immediate benefit (we ship faster), but you owe principal (the work to fix) plus interest.*
- *If the technical debt is not repaid, it accrues interest. Each minute spent on dirty code, bugs, and workarounds due to a lack of refactoring is an interest in the debt.*
- *Not all technical debt is inherently flawed. It is sometimes necessary for you to assume debt, and this is a perfectly valid strategy, for instance, for market timing or for prototyping. You have to manage this debt by planning refactoring, which helps you pay it off.*
- *The way to pay down technical debt is to refactor code, add missing tests, improve design, and so forth. Good teams track debt and prioritize paying it down regularly.*

Overview

Technical debt highlights a fundamental tension in software engineering: *speed vs. quality*.

The term helps explain to both developers and nontechnical stakeholders why seemingly *done* software still needs ongoing work.

When you write clumsy code or postpone cleanup, you've essentially taken out a loan: You get short-term feature delivery, but the complexity you introduce is the interest you'll keep paying.

We want to produce code with the *right speed, quality, and complexity*, not in a quick-and-dirty way. Over time, if you never refactor (repay principal), the interest payments (extra effort) become so high that progress grinds to a halt. Developers will spend most of their time wrestling with the consequences of earlier shortcuts, which is a classic "pay me now or pay me *more* later" scenario.

Properly managing technical debt means being conscious of when you're adding a quick workaround or a "temporary" hack, and having a plan to revisit it. Teams often use tools to log technical debt items (e.g., comments, issue tracker tickets, or dedicated debt backlogs).

These items should be addressed regularly, much like paying down chunks of debt to avoid bankruptcy.

When managed, technical debt can be an effective tool; you *intentionally* take on a bit of messiness to get feedback or meet a deadline, then clean it up. But if ignored, debt becomes a problem.

The rule teaches that *software design integrity is crucial for long-term velocity*: Investing in code quality actually enables faster progress in the future.

Origins

The term “technical debt” was coined by Ward Cunningham (one of the authors of the *Agile Manifesto*) at the OOPSLA conference in 1992. He first used the financial metaphor, while working on a finance application to explain to his boss why the team needed time to refactor code.

Ward compared developing software to borrowing money: It can be wise to take shortcuts (borrow) to achieve something sooner, *provided* you pay back the loan by cleaning up the code later.

Examples

A typical example of technical debt is skipping automated tests. Suppose a team under deadline pressure releases a new feature without writing tests for it. The release is successful (debt incurred). But later, making changes or ensuring that the feature’s stability becomes harder, and every change risks unforeseen bugs because there’s no safety net of tests (interest payments).

Eventually, a problem appears in that area due to a bug, and the team simply cannot continue. They must stop adding features on top and fix the debt by adding the missing tests and refactoring parts of the code. If they continue adding features without tests, they could face a quality crisis because the interest is being accumulated.

Another example is a startup that wants to deliver a quick prototyping effort to onboard the first customer. Developers do rapid but messy coding, with hardcoded variables and copy-and-paste code. This approach is made with technical debt in mind. The

endeavor helps the startup win a customer; however, the customer's codebase is now full of technical debt.

If the team just continues to add to it, they will face bugs and slower performance due to the messy code (interest on debt). They realize this and therefore set up a rehab sprint to refactor the code, thereby resolving the most troublesome bits (paying down the principal). While they choose to sacrifice speed for flexibility, this is what the application of technical debt highlights.

One recent example is Skype. It was launched in 2003 as a peer-to-peer (P2P) architecture. Microsoft acquired it for $8.5 billion in 2011, but it never fully modernized it from the legacy code base for mobile and cloud. They managed to retrofit P2P protocols for Azure infrastructure, but this led to performance issues, dropped calls, and memory issues in clients. When the pandemic hit in 2020, Skype held 32% of the video-calling market, but by 2021, it fell to just 6%. Even though it was one of the biggest remote-work booms in history, their user base declined sharply.

The problem was an *architectural debt*. The original P2P protocol caused two separate global network outages from a single client bug. Also, it couldn't support always-on mobile devices properly. Microsoft eventually decided to shut it down on May 5, 2025, migrating users to Teams. The fall of Skype was in large part due to its architecture design, intended for a world of always-on desktops with broadband, not for mobile and cloud-native era.

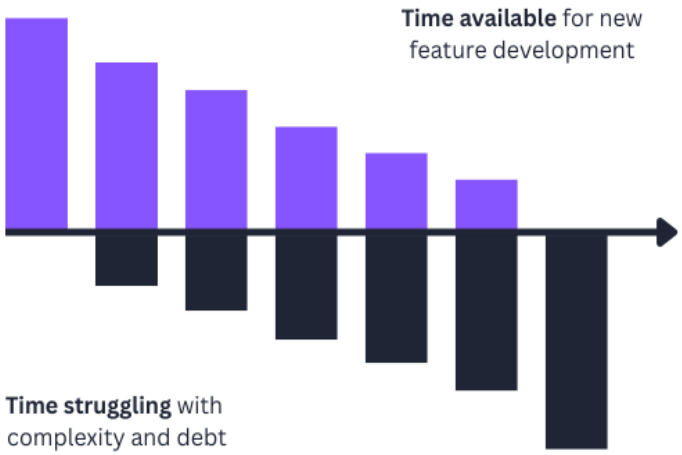

Figure 29.1 **Technical Debt**

Related to

- The Broken Windows Theory
- Hofstadter's Law
- The Boy Scout Rule

Further Reading

- Cunningham, W. (2009). *Debt metaphor* [Video]. *YouTube*. https://www.youtube.com/watch?v=pqeJFYwnkjE
- Fowler, M. (2019, May 21). *Technical debt*. MartinFowler.com. https://martinfowler.com/bliki/TechnicalDebt.html
- Technical debt. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Technical_debt
- Whelan, D. (2016). *Cutting corners and paying for it*. Agile Alliance. https://agilealliance.org/

OceanofPDF.com

30. Linus's Law

Given enough eyeballs, all bugs are shallow.

Key Takeaways

- *Linus's Law highlights the strength of peer review and community in software development. If a codebase is accessible to many developers (as in open source projects), someone will eventually have the expertise to identify and fix a given bug.*
- *The rule is often cited as a key advantage of open source software. When source code is widely available, you accumulate a large pool of contributors. This leads to bugs being discovered and fixed by volunteers.*
- *Linus's Law assumes those eyeballs are indeed looking, that is, there is an active community. However, being open source doesn't magically fix bugs.*

Overview

Linux's open development model, releasing code early and often to the public, results in rapid bug finding. When many people use and review a piece of software, problems become apparent to *someone*.

What is a perplexing bug to you might be trivial to another programmer who spots it. Or among thousands of users, one will stumble on the exact reproduction steps, and another might submit

a fix. The bug that might have taken a small team weeks to diagnose can be found in hours by the crowd.

This law shows the core of open source: Transparency and collaboration lead to more robust, reliable software. It encourages projects to leverage the community's wisdom.

However, it's not absolute. Coordination and quality control are still needed (not every community contribution is correct). But as a guiding principle, Linus's Law captures the self-correcting nature of a large developer ecosystem.

Well-known examples include open source databases or frameworks, where soon after a bug is reported on forums or issue trackers, someone from the community provides a patch.

The "many eyeballs" principle is also why internal code reviews and pair programming can be effective: More eyes on the code mean more chances to catch mistakes early.

Origins

The law is named after Linus Torvalds, the creator of Linux, but Eric S. Raymond formulated it in the late 1990s. In *The Cathedral and the Bazaar* (1999), Raymond writes, *"Given enough eyeballs, all bugs are shallow"* and calls this Linus's Law in honor of Torvalds's open development approach.

The idea was inspired by the Linux community, where hundreds of developers worldwide were debugging the kernel simultaneously.

Raymond noted that issues in Linux got solved faster precisely because so many people were checking the code.

The essay containing Linus's Law was first presented in 1997 and published as a book in 1999.

Examples

Consider the Apache HTTP Server, an open source web server used by millions. Because its code is open and it has a vast user base, many developers have at some point debugged or improved it. If there's a security flaw or bug, odds are high that someone in the global community will discover it and report it.

In many instances, patches that fix these issues are contributed by volunteers. In late 2021, a suspicious character named "Jia Tan" began making innocuous contributions to XZ Utils, a data compression package included with (almost) every Linux install. Over 3 years, Jia Tan gained trust, assisted by sock puppets intimidating a single maintainer into relinquishing control, before being given full commit access. In February 2024, this person added a sophisticated backdoor (CVE 2024-3094, CVSS 10.0) that used SSH authentication to allow remote code execution with root privileges. The backdoor was detected by one loner, Andres Freund, a Microsoft engineer, who found that SSH was taking 500 milliseconds longer than usual on his testing box. He traced the slowdown to obfuscated code hidden inside liblzma and reported it (in Openwall security mailing list on March 29, 2024) just days before the compromised versions could reach stable production distributions.

These examples demonstrate how wide-ranging reporting results in rapid discovery and remedying of the bug, which is the key to Linus's Law.

For instance, consider an enterprise software product that is proprietary and has only a few clients. If a subtle bug appears, only the vendor's small team and a few client implementers are likely to notice it. It might take a long time to surface and debug, because of "limited eyeballs." In fact, the bug might remain hidden until a customer reports it under specific conditions.

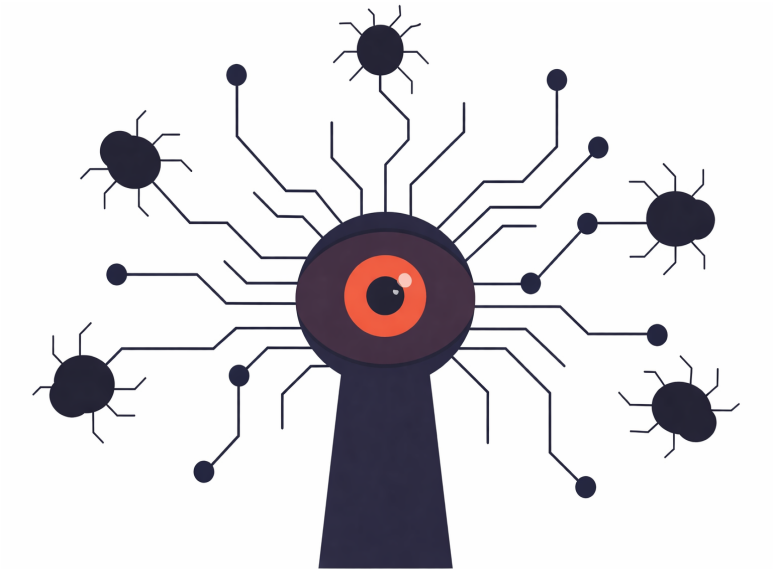

***Figure* 30.1 Linus's Law**

Related to

- Brooks's Law
- Sturgeon's Law
- The Bus Factor

Further Reading

- Cox, R. (2024, April 3). *Timeline of the xz open source attack. research!rsc.* https://research.swtch.com/xz-timeline
- Linus's law. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Linus%27s_law
- Merrill, S. (2012, February 23). *With many eyeballs, all bugs are shallow. TechCrunch*. https://techcrunch.com/2012/02/23/with-many-eyeballs-all-bugs-are-shallow/
- Raymond, E. S. (1999). *The cathedral and the bazaar: Musings on Linux and open source by an accidental revolutionary*. O'Reilly Media.

OceanofPDF.com

31. Kernighan's Law

Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it.

Key Takeaways

- *A consensus in the software development world is that bug detection and removal, or debugging, is more complex than programming. This is because debugging involves understanding both code and the reason why the code doesn't work, while programming requires understanding code only.*
- *If you write code at the limit of your intelligence (clever or overly optimized), you won't be able to understand or troubleshoot it later quickly.*
- *This rule promotes clean coding practices. Simple code with good structure and documentation is easy to debug. This is because coding with clarity in mind saves time in the long run.*
- *It's a reminder that even if your code runs successfully when you write it, it's not necessarily complete. Systems become increasingly fragile as their complexity grows.*

Overview

Kernighan's Law says that debugging requires understanding what the code actually does, which can be twice as hard as writing it. When coding, you operate with a specific mental model and full

context in mind. But when debugging, you might be dealing with someone else's code or your own code after that context has gone.

Writing clever code is essentially self-sabotaging. It may take you some time to fix it, and you may forget the nuances of this clever coding trick, so you will not be able to understand it.

This does not mean you should never apply optimizations or advanced methods, but you should do so with caution and with a clear rationale.

Quite often, a maintainable version is superior to an optimized version, which is difficult to understand, as the optimization could become nullified at a later stage when resolving a particular issue with the program.

As Kernighan implies, if you make your code too tricky, you've essentially outsmarted yourself.

Origins

Brian Kernighan first expressed this idea in *The Elements of Programming Style* (1974, second edition 1978) with P.J. Plauger. Famous for co-authoring *The C Programming Language* and *The Elements of Programming Style*, Kernighan wrote about simplicity in an era defined by scarce computing resources.

Examples

Imagine a developer writing a function in a compressed style, chaining multiple operations in one line, employing bitwise tricks and side effects that save a few lines of code:

```
public string GetUserDisplay(User u) =>
     u?.IsActive == true ? (u.Name ?? "").Trim() is var n &&
n.Length > 0
     ? n + (u.Role > 0 ? $" ({(Role)u.Role})" : "") : "Unknown" :
"Inactive";
```

It works for the initial need. However, a month later, a bug is reported in that function's output.

The proper, full-fledged version should look like:

```
public string GetUserDisplay(User user)

{

    if (user is null || !user.IsActive)

        return "Inactive";

    var name = user.Name?.Trim();

    if (string.IsNullOrEmpty(name))

        return "Unknown";

    if (user.Role > 0)

        return $"{name} ({(Role)user.Role})";

    return name;

}
```

This scenario demonstrates Kernighan's Law clearly. The initial writing might have taken 30 minutes of clever thinking, but debugging the complex solution took 3 hours. Had the code been written simply and clearly, it might have taken 45 minutes to write, but only 30 minutes to debug later, which is a net win.

Another example is in optimization. A programmer prematurely optimizes a function with complex caching logic and microoptimized loops to make it run faster. It's very clever and elegant, but when a bug or a new requirement emerges, the team finds the optimized code almost unreadable. They struggle to modify it without breaking something else, and each change introduces new bugs. The performance gains from the clever version didn't outweigh the difficulty in maintenance.

Also, have you ever seen a developer write recursive code? Recursive code shows up far less often in production than textbooks suggest, and when it does, it frequently brings along hard-to-diagnose edge-case issues. The logic was unintuitive, so understanding it was so hard. In the end, we rewrote the whole module in a simpler way so that everyone can understand and maintain it.

As Kernighan's Law suggests, cleverness becomes a liability. Nobody can debug or adapt such code easily, proving the danger of writing code at the edge of one's cleverness.

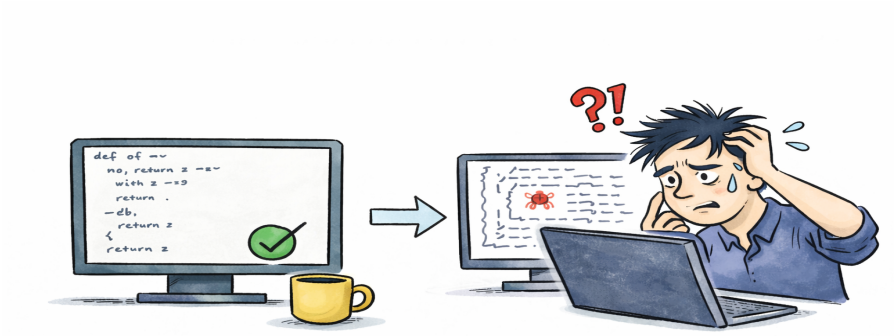

Figure 31.1 **Kernighan's Law**

Related to

- The KISS Principle
- Knuth’s Optimization Principle

Further Reading

- Kernighan, B. W., & Plauger, P. J. (1978). *The elements of programming style* (2nd ed.). McGraw-Hill.
- McConnell, S. (2004). Code complete (2nd ed.). Microsoft Press.

OceanofPDF.com

32. The Testing Pyramid

A project should have many fast unit tests, fewer integration tests, and only a small number of UI tests.

Key Takeaways

- *Unit testing is where you start. Unit tests are run as separate functions and execute quickly. That means you can afford to write lots of unit tests.*
- *After the unit tests, there must be an integration test layer. These tests verify the integration of modules. The number of integration tests is much smaller than that of unit tests.*
- *At the top of the pyramid are the end-to-end tests, which simulate real-world scenarios based on how users will use the system. These are essential for ensuring everything is working correctly, but they are slow and expensive to maintain.*
- *By organizing your tests in this manner, you receive quick feedback (most tests are quick unit tests), and when your UI test fails, there's a better chance it's due to a real problem.*

Overview

The pyramid is a visual metaphor: Picture a triangle divided into levels. The most significant level (at the bottom) is unit tests, which are fast and the most numerous. The middle layer includes integration tests (service or API tests), fewer in number. The top

point of the triangle represents UI tests, which are the least numerous.

The idea is that *as you go up the levels, tests become more expensive* (in terms of time, effort, fragility), so you want proportionally fewer of them. By following the pyramid, a team ensures that most bugs are caught at the cheapest level: unit tests.

For example, an error in a calculation should ideally be caught by a unit test for that function, rather than by a GUI test that clicks through the application and then finds the calculation off. That would be a slow and indirect way to catch it.

If the pyramid is inverted (lots of end-to-end tests, few unit tests), the test suite tends to be slow (because end-to-end tests take time to set up environments, UIs, etc.) and fragile (one small change can break many high-level tests).

Thus, the Testing Pyramid also aligns with agile and DevOps practices that emphasize quick feedback and continuous integration: A solid unit test suite can run in seconds or minutes in a CI pipeline, alerting developers to problems immediately.

Modern discussions sometimes add layers or modify terms (for instance, calling the middle layer “service” or “API” tests, including a layer for integration vs. component tests, etc.). Still, the core principle remains: *Test more at the unit level than at the UI level.*

By doing so, teams gain confidence in their code with minimal slowdown. They then supplement that with a right-sized set of higher-level tests to verify the overall system.

Origins

Mike Cohn is credited with popularizing the Test Pyramid. He described it in his book *Succeeding with Agile* and blog posts around 2009, coining the term and the concept. The idea itself builds on earlier testing theory (such as the test levels in the ISTQB foundation or the general practice in TDD of writing many unit tests).

Examples

Consider a web e-commerce application. Following the test pyramid, the development team writes extensive unit tests for functions such as price calculation, discount logic, and validation functions (hundreds of unit tests covering all edge cases of those functions). They also write a set of API integration tests, for example, testing that the order placement API works correctly when connecting the inventory, payment, and notification services (maybe a few dozen of these tests).

Finally, the team has a few end-to-end tests that simulate a user journey, such as browsing products, adding to cart, checking out, and confirming an order. These end-to-end tests run using a headless browser, clicking through the UI.

Because of a high number of unit tests, if something breaks in the business logic, it's likely caught long before a complete end-to-end test is run. End-to-end tests are essentially the last sanity check the overall system receives. This kind of test, which intentionally focuses on fast unit tests, takes little time in CI and allows the development team to proceed with the knowledge that they can deploy without

impediment. A failure in any of the UI tests (perhaps the ID of the checkout button changed, and the script can't find the button) would be a trivial matter.

Instead, think about a team that did not follow the pyramid structure. They have a few unit tests and instead use 50 end-to-end GUI tests. They perform these tests once a night because it takes them a few hours. They often find several failing tests each night, but it's hard to tell whether it's due to genuine bugs or just test flakiness or environment issues. Developers get feedback with a day's delay and spend a lot of time diagnosing failures.

This is the *anti-pattern* the pyramid warns against. Such a team might incorporate the pyramid approach. Increasing unit test coverage (to get faster feedback on logic) and reducing reliance on slow UI tests improves build times and reliability.

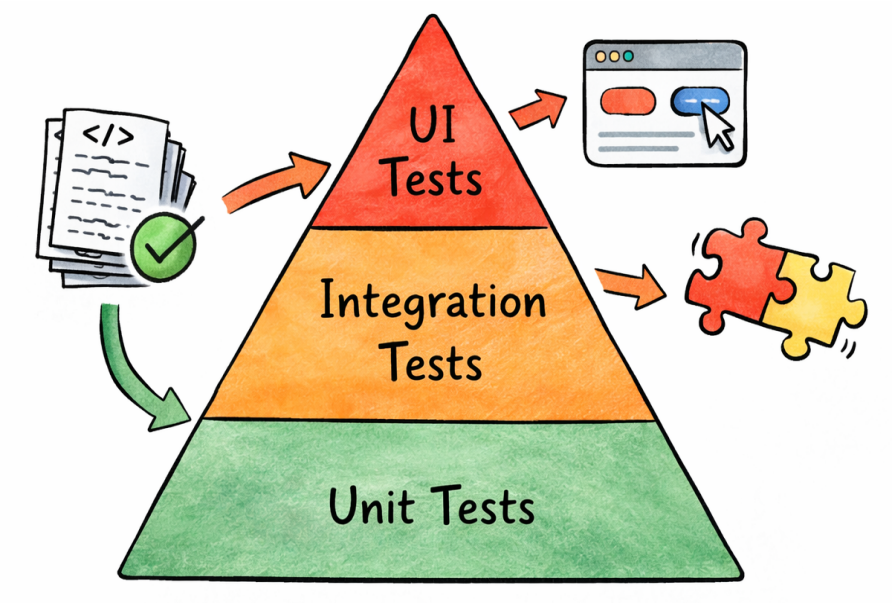

Figure 32.1 **Testing Pyramid**

Related to

- Shift-Left Testing
- Lehman's Laws

Further Reading

- Bender, A. (2020). Chapter 11: Testing overview. In T. Winters, T. Manshreck, & H. Wright (Eds.), *Software engineering at Google: Lessons learned from programming over time.* O'Reilly Media. https://abseil.io/resources/swe-book/html/ch11.html
- Cohn, M. (2009). *Succeeding with agile: Software development using Scrum*. Addison-Wesley (chapter on automation and the Test Pyramid).
- The practical test pyramid. MartinFowler.com. https://martinfowler.com/articles/practical-test-pyramid.html

Sidenote: The Beyoncé Rule

When talking about test coverage, we simply must mention the Beyoncé Rule, which states, "If you liked it, you should put a test on it." This is how Google programmers determine which code to test (Bender, 2020). The lesson originated from Google infrastructure teams who make changes to large codebases. If they land a change successfully on your tests but your product is broken anyway, it is your problem. It gets fixed. The missing tests are your job to write.

This turns the old blame game on its head. You can't place the blame on someone else for a thing you never tested to ensure that it actually works. This test suite is your contract with the world.

Everything outside is deemed “fine to break.” The principle applies to more than happy path testing. “Performance, accessibility, security, error handling, edge cases: If you care about it, test it.” A team that doesn’t learns something painful: “The absence of a test is not neutral; it’s a silent permission slip to ship broken.”

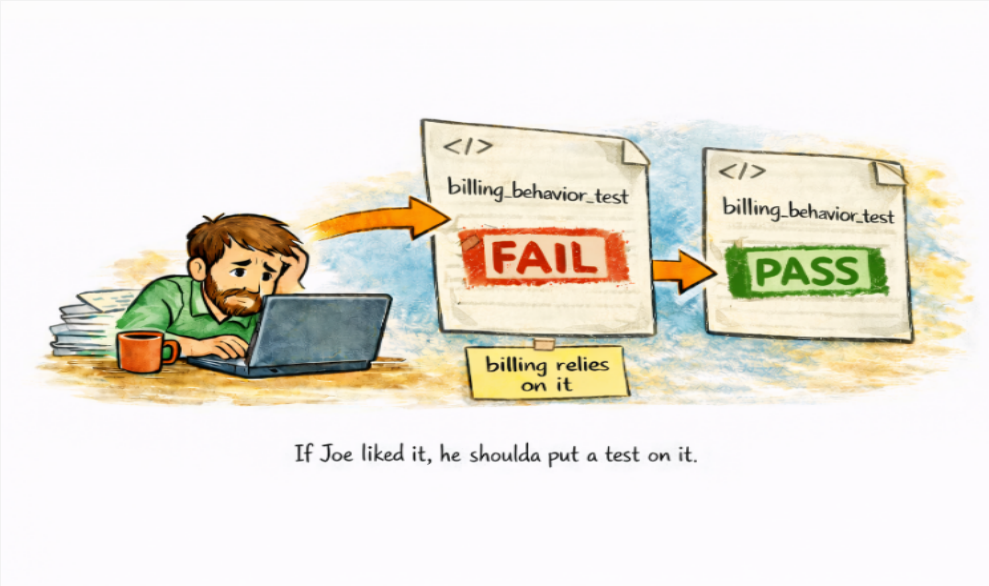

Figure 32.2 The Beyoncé Rule

OceanofPDF.com

33. The Pesticide Paradox

Repeatedly running the same tests becomes less effective over time.

Key Takeaways

- *Over time, the outdated test suite will identify fewer bugs. The tests are still helpful, even when they are no longer effective at detecting defects.*
- *This paradox illustrates the need to infuse new test data continuously. It is essential for testers not to become complacent with their test scripts, thinking that these alone will be sufficient in the future.*
- *By periodically creating new tests (by altering existing ones), you effectively "refresh the pesticide," thereby getting an opportunity to catch new bugs. This involves extending tests for new functionality, trying new input combinations, or perhaps just investigating new boundary cases.*

Overview

The pesticide paradox analogy comes from the agricultural domain. When the same pesticide is used repeatedly by farmers, the pest develops resistance to it.

In software, the first few runs of a new set of test cases might find a bunch of bugs. Those bugs get fixed. If you keep running the same

tests, and no changes occur in the tested areas, you shouldn't find more bugs there; the software has become immune to those specific tests.

However, if new functionality is introduced or bug fixes occur in untested regions of the application, your existing tests will not detect these issues, just as new pests may not be vulnerable to last year's pesticides.

The point is that the testing process cannot be stagnant. Test cases must be maintained regularly. For example, after every release cycle, testers can assess which bugs made it through to production and which remained problematic, and enhance the test suite afterward by adding test cases that address those vulnerabilities.

Also, every time requirements evolve or a new feature is introduced, test designs for these must be created, or existing tests may need to be updated to reflect a behavior change.

This paradox also provides an argument for exploratory testing. Exploratory tests conducted by people testing the application in new ways with each iteration may uncover problems that a static regression suite would otherwise overlook.

To be clear, the Pesticide Paradox doesn't imply regression testing is of no use. Those old tests still serve to catch regressions (bugs reintroduced into previously working areas). In fact, a lack of failures in a stable regression suite is a good thing; it means no old bugs have resurfaced. But to catch *new* bugs, one must design *new* tests.

Mature testing strategies, therefore, balance regression testing with exploratory and new-scenario testing.

Origins

The concept was articulated by Dr Boris Beizer, a software testing expert, and it became widely known as one of the “seven principles of testing” in ISTQB (International Software Testing Qualifications Board) materials.

Beizer basically stated that every method of testing will yield a Pesticide Paradox as bugs adapt to tests, leaving more subtle bugs behind.

The principle has its roots in practical observations by testers that, after a point, continually rerunning the same tests find nothing new. By the 2000s, it was widely taught and understood in quality assurance circles. The paradox captures the need for *ongoing test innovation* to keep testing effective.

Examples

Let’s consider a mobile app where, in the first release, testers wrote extensive tests for the login and user profile features, finding and getting 10 bugs fixed in those areas. In the subsequent releases, those test cases all pass (no new bugs in login or profile). Meanwhile, the app has added a messaging feature, but the test suite didn’t add many cases for it. However, within a couple of releases, complaints started trickling in regarding the messaging functionality crashing. Unfortunately, the test suite failed to identify

these because it was not updated to test messaging, which is predominantly centered on login and profile.

This is the Pesticide Paradox, which states that as testing became green (no defects remained in the code tested), new defects began to emerge, requiring new tests. After the group included their messaging tests, they began to uncover these issues before they become releases. The same process continues until those tests no longer find new issues, and the pattern repeats with the following features that emerge.

Another typical example is a set of 50 regression test scripts that click through the UI. They found many problems initially (broken links, pages routed incorrectly, etc.). These problems are taken care of. These scripts now run every night and, for the most part, are passing. The UI has been redesigned over time, with the addition of a new module, resulting in underlying logical problems (e.g., the calculation is incorrect on a report page). The old UI tests don't check the calculation; thus, they don't catch this problem. The QA team is seeing fewer and fewer problems being uncovered via the automation toolset.

Once the Pesticide Paradox is acknowledged, the team can decide to add new test steps to cover the report calculations, as well as to do an exploratory test session on the new module. This approach leads to discovering the calculation bug and a couple of edge cases.

The automation suite is then expanded with those test scenarios. In essence, the team "changed the pesticide" to start finding bugs again by updating their tests.

Figure 33.1 **The Pesticide Paradox in Testing**

Related to

- Lehman’s Law of Continuing Change
- The Testing Pyramid

Further Reading

- Beizer, B. (1990). *Software testing techniques* (2nd ed.). Van Nostrand Reinhold.
- International Software Testing Qualifications Board. (2024). ISTQB® Certified Tester—Foundation Level syllabus (v4.0.1). https://istqb.org/wp-content/uploads/2024/11/ISTQB_CTFL_Syllabus_v4.0.1.pdf
- Kaner, C., Bach, J., & Pettichord, B. (2001). *Lessons learned in software testing: A context-driven approach.* Wiley.

OceanofPDF.com

34. Lehman's Laws of Software Evolution

Software that reflects the real world must evolve, and that evolution has predictable limits.

Key Takeaways

- *The first law, Continuing Change, states that if a system isn't continuously updated to meet new needs, it becomes less satisfactory to use. In practice, this means that successful software is never truly finished; to remain useful, it undergoes ongoing enhancements (or else, users abandon it for something that keeps up).*
- *Over time, changes accumulate, making the system more complex (the second law). Unless developers invest in refactoring or restructuring (i.e., paying down technical debt), the system's internal complexity will increase, slowing further development.*
- *Even if users demand more, a development organization can only do so much in a given time. They can't exponentially accelerate delivery forever. Factors such as team knowledge (familiarity) and process overhead limit it.*
- *The perceived quality of the system will decline (perhaps due to rising user expectations, environmental changes, or the accumulation of minor issues).*

Overview

Lehman's Laws describe an unavoidable reality of long-lived software systems that operate in the real world. Such systems, business software, operating systems, and platforms must continuously change to remain useful. That change is neither optional nor free.

As software evolves, complexity accumulates. Each change slightly increases internal disorder unless effort is spent counteracting it. Over time, this complexity reduces development speed, increases risk, and raises the cost of further change. Teams feel this as "everything taking longer than it used to."

At the same time, organizations face hard limits. Knowledge, coordination, and familiarity constrain how much change can be absorbed in a single release.

Adding people does not remove these limits. It often reinforces them. As a result, the delivery rate stabilizes even as demand grows.

Lehman's Laws explain why mature systems slow down, why maintenance dominates cost, and why continuous investment is required simply to stand still.

Initially, there were three laws (in 1974), then five, and eventually eight by the 1990s as more data became available.

Here are the eight laws:

1. **Continuing Change:** Software that interacts with the real world (E-type) must be continually changed, or it becomes obsolete. For

example, tax accounting software must be updated whenever tax laws change. If it isn't, it quickly becomes useless as laws outpace it.

2. **Increasing Complexity:** As an E-type system evolves, its complexity increases unless active efforts, such as refactoring and redesign, are made to control it. This means older software tends to get more convoluted, making changes harder, a well-known phenomenon in aging codebases.
3. **Self-Regulation:** The software evolution process is self-regulating. The measures, such as the frequency of releases and the number of changes, remain within fixed bounds. There's a balance in the evolution of software, possibly constrained by business reasons.
4. **Conservation of Organizational Stability:** During the life of a system, the rate of organizational efficiency, or the number of people effectively working on the system, remains relatively stable. This is true regardless of the number of people working, as there are other factors, such as efficiency and communications, that can't be infinitely improved.
5. **Conservation of Familiarity:** There is a point beyond which the amount of growth that can happen in one release is limited by the need to preserve the team's familiarity/control with the system. Significant, dramatic changes are not very common here. Instead, incremental changes that the team can manage well, while still understanding the system are required.
6. **Continuing Growth:** To remain useful, a system generally needs to continue delivering new functionality over time. User expectations rise, and competitive pressure means software must do more or better to stay relevant.

7. **Quality Degradation:** If the system is left to degrade, its quality will appear to decline over time. Quality, in such a context, could refer to either code quality or user experience. This quality could degrade due to various changes that cause issues, or because standards change (e.g., something that passed as quality UI design 10 years ago just won't cut it anymore).
8. **Feedback System:** Software evolution is a multiloop feedback system. That is, software evolution as a process involves feedback from users, developers, stakeholders, and the software itself through metrics, and this feedback must be addressed to evolve software effectively. That is, managing software evolution is complex, and this process involves attending to inputs at various levels, such as user satisfaction, velocity, or code, among others.

These laws work together to explain why software projects show slowing trends and why continued investment is necessary. They also indicate how software is not a finished artifact but rather a living thing within its ecosystem of users and requirements.

Origins

Lehman's early laws were formulated in 1974, based on his study of various software, including OS/360, which was developed by IBM. Lehman and Belady observed version histories, growth metrics, and other data, which led to the initial laws.

Over the years, Lehman continued to refine the laws through the 1980s and 1990s as more projects (and later, open source projects) provided data.

Examples

Let’s consider a big enterprise resource planning (ERP) system deployed in 2000. Over 25 years, the company has made changes, including updates to new business rules and integrations with the latest technologies, UI refreshes, and similar.

According to the first Lehman’s Law (Continuing Change), this is necessary. If they had not continued updating the ERP, it would no longer meet the business’s needs today. Over those 25 years, the system’s complexity grew enormously (hundreds of modules, millions of lines of code). The company observes that adding functionality in 2025 takes longer than it did in 2005. This reflects the effects of the second (Increasing Complexity) and the seventh law (Diminished Quality, if Not Maintained).

Periodically, there was a need to invest in cleanup projects (refactoring, restructuring some aspects of the system) to keep the system maintainable. Still, they found they could only deliver so much per year (Laws 4 and 5; output per year stabilized and big jumps in scope were rarely feasible because the team could handle only incremental changes without breaking things).

The system also required continued growth (Law 6): Each year, new capabilities were added (support for mobile, then cloud, etc.), or users would become dissatisfied and consider alternatives.

This real-life story aligns with Lehman’s Laws, demonstrating why long-lived software demands constant effort and why that effort often yields diminishing returns if complexity isn’t addressed.

Also, if we take the Linux kernel as another example, it has undergone continuous change (releases constantly, adapting to new hardware and requirements). If it stopped adapting, it would become irrelevant as new tech emerges. Its functionality has grown massively (from a simple terminal OS to supporting billions of devices and countless features). Complexity has increased. The kernel is large and complex, and it requires subsystem maintainers to keep it organized.

The “conservation of familiarity” can be seen in how changes are managed: Big rewrites are rare. Most development is iterative, and maintainers enforce a pace that the community can keep up with (you can’t overhaul everything at once).

Interestingly, studies noted that Linux’s growth was super-linear for a while (violating the strict interpretation of some laws). Eventually, even Linux saw growth taper off linearly as the project matured.

This example shows Lehman’s Laws at play and also how they might evolve under modern collaborative development.

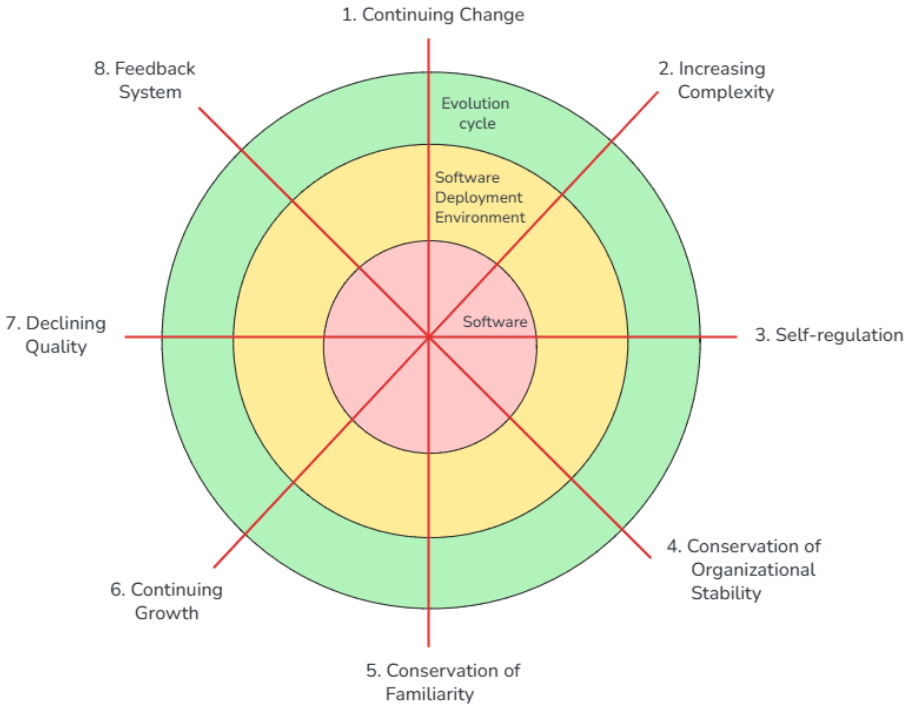

Figure 34.1 **Lehman's Laws of Software Evolution**

Related to

- Brooks's Law
- Conway's Law
- Technical Debt

Further Reading

- Herraiz, I., González-Barahona, J. M., & Robles, G. (2013). *The evolution of the laws of software evolution*. ACM.
- Lehman, M. M., & Belady, L. A. (1980). *Programs, life cycles, and laws of software evolution*. IEEE Computer Society Press.
- Lehman, M. M., Ramil, J. F., Wernick, P. D., Perry, D. E., & Turski, W. M. (1997). *Laws of software evolution: The nineties view*.

IEEE Computer Society Press.

OceanofPDF.com

35. Sturgeon's Law

90% of everything is crap.

Key Takeaways

- *It's like an extreme form of the Pareto Principle (80/20 Rule). The bulk of what's being put out just isn't good, and the good stuff is the exception.*
- *In software, there are often many features or code paths that add very little value, and the critical challenge is to find a way to maximize the high impact 10%.*
- *Most new concepts or technologies fail to deliver, while the exceptional ones stand out.*

Overview

In the tech sphere, this law implies that most of the code or features are not necessary. Perhaps 90% of experiments or features don't pan out, and only 10% drive real value.

You have probably already seen this. From an engineer's standpoint, not every line of code you write is gold; from a product manager's standpoint, many features don't delight users nor bring much value (Who said dark mode?).

It pairs with the idea of the *10x engineer* not being someone who writes 10x more code, but someone who identifies the 10% of work that delivers 10x value.

The risk is not in the lower quality of the 90%. The risk is in pretending that it isn't. When a team considers all the work to have the same value, this introduces more complexity and slows down.

In practice, Sturgeon's Law encourages continuous refinement, assumes there's a lot of noise, and focuses on finding the signal.

Origins

Theodore Sturgeon, a science fiction author, said this in 1957, responding to criticism that "90% of science fiction is crap." His response was that 90% of everything was crap.

It was first published as "Sturgeon's Revelation" in a 1957 issue of *Venture* magazine.

Examples

Consider an app with 100 features. User analytics might show that a user is using 10 of these heavily and the rest not at all, which is what Sturgeon's Law says: most things are crap. 90% of this stuff doesn't provide significant value.

Another example is that the organization may have many project ideas to develop each quarter. So, according to Sturgeon's Law, most of these ideas wouldn't amount to anything, but a few could be game-changers.

In the area of code reviews, a significant amount of code has been written in the past that is neither elegant nor required. This supports the hypothesis that less code is high quality.

One good example of Sturgeon's Law is WordPress.org plugin repository. Data indicates that from an overall tally of 57,000 or so plugins listed within the library, more than 34,000 (59.3%) have not seen any kind of update for more than 2 years. Of these abandoned plugins, 78.99% (or 26,940) have fewer than 100 active installs, and very nearly 19% of the total plugin library boasts "0+ installs," or essentially zero users. Meanwhile, the small number of them that do get regular maintenance is what drives the vast majority of the 40%+ of the web running WordPress.

Sturgeon's Law can be empowering if taken positively: Don't be disheartened by the prevalence of crud. Instead, strive to make something in the top 10%.

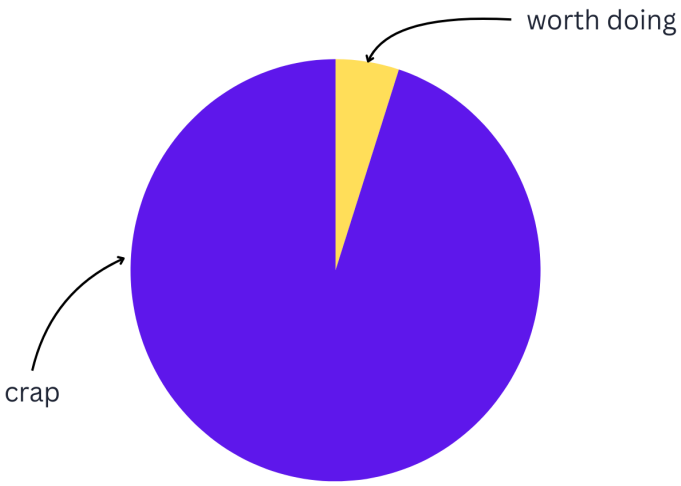

Figure 35.1 **Sturgeon's Law**

Related to

- Goodhart's Law
- Gall's Law

Further Reading

- Sturgeon's law. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Sturgeon%27s_law
- u/OP_username. (2025). *Over 50% of plugins in the WordPress repository haven't been updated in 2+ years* [Online forum post]. Reddit. https://www.reddit.com/r/Wordpress/comments/1n0lnas/over_50_of_plugins_in_the_wordpress_repository/

OceanofPDF.com

Part V
Scale, Performance, and Growth

These laws describe bottlenecks, limits on parallelism, queueing effects, and why scale has hard math under it.

OceanofPDF.com

36. Amdahl's Law

The speedup from parallelization is limited by the fraction of work that cannot be parallelized.

Key Takeaways

- *Sequential work sets the ceiling, and no amount of parallelism can overcome it.*
- *Scaling exposes bottlenecks. More resources make limits visible, not disappear.*
- *Fix before you scale. It is necessary to reduce sequential paths first. Parallelism comes second.*
- *This law applies to people, too. Decision bottlenecks can dominate at the team scale.*

Overview

Suppose you have a process that has three steps: one takes 1 minute, another takes 10 minutes, and the last one takes 2 minutes (13 minutes in total).

If you double the speed of Step 3, you save 1 minute. If you double Step 2, you save 5 minutes. The biggest gains come from speeding up the part that takes the most time. That's the core idea behind Amdahl's Law.

Now, let's put this into the context of parallel computing. As you add CPU cores, only the fraction of your code that can be parallelized will

actually accelerate. The sequential fraction, the work that has to run one step at a time, remains unchanged.

That is, as you throw more cores at the problem, the parallel portion completes faster. However, the sequential portion doesn't budge. Eventually, it becomes the bottleneck that dominates your total execution time. You can never go any faster than the sequential portion allows, regardless of how many cores you add.

Amdahl's Law provides the following formula:

- If s is the fraction of work that must be sequential, the maximum speedup with infinite parallel resources is 1/s.
- If 10% is sequential, the maximum speedup is 10x.
- If 50% is sequential, the maximum speedup is 2x. No additional parallelism beyond that limit helps.

This law has a direct effect on horizontal scaling. If your system has a database that can't be parallelized, adding more application servers hits a wall. You must identify and reduce sequential components before scaling further.

The same principle applies to organizations. If a single person or committee handles architectural decisions, reviews, or conflict resolution, adding engineers increases coordination costs without increasing throughput. Parallel effort accumulates behind sequential gates.

Origins

Gene Amdahl, a computer architect known for his work on IBM mainframes, introduced the law in 1967 at the AFIPS Spring Joint

Computer Conference. It was originally framed around processor performance but has since proven universally applicable to systems and organizations.

Examples

A good example is that adding application servers doesn't help if all requests hit a single database instance. One database becomes the limit.

Another example is that breaking a monolith into microservices won't improve performance if all requests ultimately serialize through a shared dependency, such as an authentication or billing service.

Let's say that you need to optimize a web page that renders in 1,000ms. 90% of this was parallelizable work (database queries, inventory checks, price computation), while 10% was sequential (authentication and rendering out the template). Let's say we added 16 threads and got 6x speedup from the process. Our total time would come down from 1,000ms to 150ms, but 100ms of that was due to the sequential portion of the work. At any time, regardless of the number of threads available, our program was never going to come under 100ms. The real gain here is to stop adding new threads and start reducing the sequential component. For example, it can add a caching layer that could speed up rendering. If we reduced the sequential part from 10% to 2.5%, Amdahl's Law now predicts a maximum speedup of 14.5× with 16 threads.

This law is current in the era of AI. Even if AI speeds up large chunks of work, you still have to do things yourself, such as thinking,

deciding what to ask, checking results, coordinating, and acting as a sequential bottleneck that limits overall gain. AI can facilitate the actual coding, writing, or analysis, but time outside of AI assistance still dominates total throughput, so the jump from 1× to 10× productivity simply can't happen unless those non-AI pieces shrink, too. In practice, that means that the value derived from AI is throttled by human tasks (human-in-the-loop; HITL) and interactions that AI doesn't absorb (yet). That's why some people see huge gains, while others see modest impact.

As shown in Figure 36.1, with 50% parallel work, speedup flattens near 2× even as processors grow into the thousands. At 75%, the ceiling sits near 4×. At 90%, gains approach 10×, then stall. Only workloads near 95% parallel continue to scale meaningfully, and even those show signs of diminishing returns.

The lesson stays simple: Remove sequential bottlenecks before adding hardware, nodes, or people.

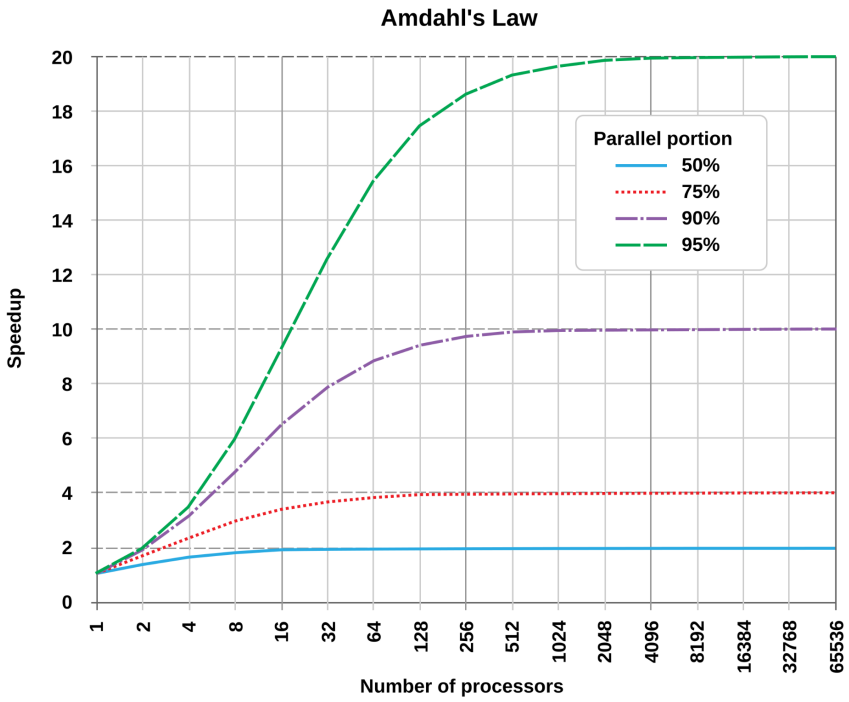

Figure 36.1 **Amdahl's Law in Action (Source: Wikipedia)**

Related to

- Gustafson's Law
- Metcalfe's Law

Further Reading

- Amdahl, G. M. (1967). Validity of the single-processor approach to achieving large-scale computing capabilities. In *AFIPS Conference Proceedings* (Vol. 30, pp. 483–485). AFIPS Press.
- Brooks, F. P., Jr. (1995). *The mythical man-month: Essays on software engineering* (Anniversary ed.). Addison-Wesley.

OceanofPDF.com

37. Gustafson’s Law

It is possible to achieve significant speedup in parallel processing by increasing the problem size.

Key Takeaways

- *As computing resources grow, it is possible to compute more problems in a given time, rather than solving the same issues faster.*
- *It opposes the pessimism of Amdahl’s Law by assuming the size of the problem to be solved will increase proportionally with computing power, so that parallel processors remain busy.*
- *Practically, Gustafson’s Law promotes the use of more resources in computation to achieve more in terms of the scope of tasks, rather than obtaining diminishing returns.*
- *The law teaches that software and algorithms should be designed to “scale out.” That means that as available processor cores or machines increase, the amount of work that can be done can be scaled up accordingly.*

Overview

Gustafson’s Law is a principle in parallel computing that offers an optimistic view of scalability. While Amdahl’s Law assumes a fixed problem size and concludes that speedup is limited by the fraction of work that must be done serially, Gustafson’s Law changes the perspective.

It observes that when more processors are available, developers tend to **i***ncrease the problem size* to use that extra power. For example, if you have a cluster twice as powerful, you might process twice as much data (or simulate greater detail) in the same time.

By doing so, the *parallel portion of the work grows with N processors*, while the serial portion remains about the same in absolute terms. This leads to a “scaled speedup” that can be almost linear with the number of processors because the workload has been scaled up to keep all processors busy.

Gustafson’s insight is that developers naturally find ways to use more computing power by asking bigger questions: Instead of finishing yesterday’s computation in half the time, we often choose to answer a more complex question in the same amount of time.

This means parallel computing can be far more rewarding than Amdahl’s Law predicted, as long as we *adapt our software to tackle bigger challenges when hardware expands.*

Origins

Gustafson’s Law was formulated by computer scientist John L. Gustafson (with Edwin Barsis) in 1988, in a paper titled “Reevaluating Amdahl’s Law.” Gustafson was working at Sandia National Laboratories on high-performance computing. At that time, Amdahl‘s 1967 result cast doubt on the usefulness of massively parallel supercomputers, since it suggested diminishing returns as more CPUs were added.

Gustafson observed from practical workloads that this held only if the workload was kept constant, which didn't reflect how scientists used new machines. His 1988 argument demonstrated that *by increasing problem size, a 1024-processor system could achieve a near-1024× speedup on suitably scaled tasks*.

Examples

In practice, Gustafson's Law is evident in many domains of computing. High-performance computing (HPC) applications, such as climate modeling, molecular simulations, or astrophysics, routinely increase the model resolution or simulation time span when more processors are available.

For instance, if a weather simulation runs on 1 CPU for a particular region, a team using 1000 CPUs won't just get the forecast 1000 times faster. Instead, they might run a far more detailed global model in the same amount of time. The result is a better forecast (a larger problem) completed in the original timeframe.

Here, the *serial setup* (reading input and initializing) remains about the same. Simultaneously, the *parallel work* (crunching numbers for each grid cell) scales with the number of CPUs, yielding nearly linear overall speedup.

Another example can be seen in big data processing. Suppose analyzing a dataset of 1 million records takes an hour on 1 machine. If you have a cluster of 10 machines, rather than simply finishing in 6 minutes, you might analyze 10 million records in an hour. You leverage the cluster to handle *more data* at once. Effectively, you've

achieved a 10× scaled speedup by solving a problem 10 times larger in the same time window.

Gustafson's Law is also evident in modern distributed systems such as MapReduce and Spark. When adding more nodes, these systems encourage splitting the dataset into more partitions so each node works on a portion, keeping them all busy.

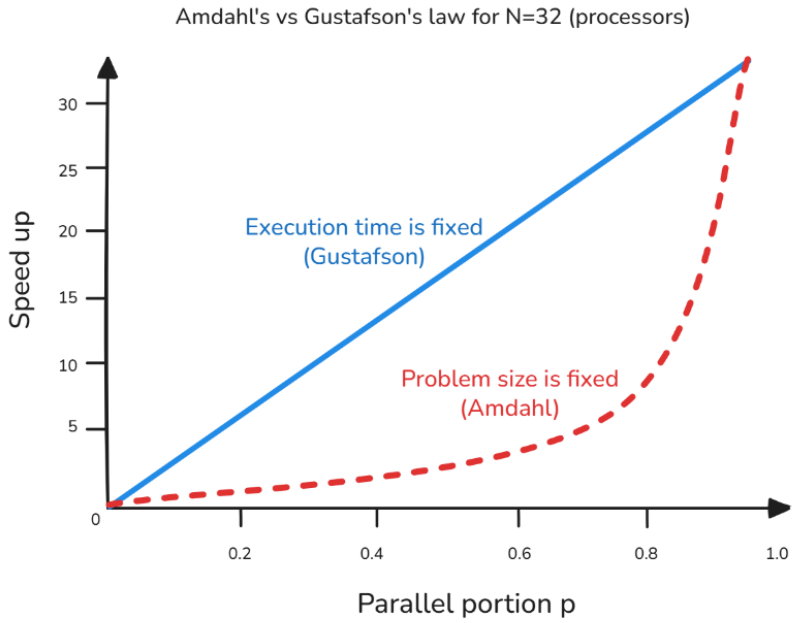

Figure 37.1 **Amdahl's vs. Gustafson's Law**

Related to

- Amdahl's Law
- Metcalfe's Law

Further Reading

- Gustafson, J. L. (1988). Reevaluating Amdahl's law. *Communications of the ACM, 31*(5), 532–533.
- Gustafson's law. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Gustafson%27s_law

OceanofPDF.com

38. Metcalfe's Law

The value of a network is proportional to the square of the number of users.

Key Takeaways

- *If you double the number of users, the potential connections (and interactions) more than double; they roughly quadruple.*
- *This law highlights the importance of network effects: Each new user adds value to existing users by creating new opportunities for interaction. A product or platform becomes exponentially more useful as its user base grows.*
- *Metcalfe's Law helps explain why these social media and messaging platforms can grow explosively in value once a tipping point is reached.*

Overview

Metcalfe's Law is an observation about *communications networks* that has been generalized to many technology ecosystems. It basically says that *the value of a network is proportional to the square of the number of connected users*.

Why square? With 5 nodes, up to 10 pairwise connections are possible, and with 12 nodes, 66 are possible. As you add nodes, each one can connect to all the others, so the number of potential pairwise connections grows roughly like the square of the network size.

For social networks or any platform focused on user-to-user interaction, Metcalfe's Law explains their scaling power. The more users there are on Facebook, the more connections and content become available to each user, making the platform more valuable to everyone.

Network effects feed on themselves, a rich get richer phenomenon. Initially, when a network is small, adding one user might not change much (if you only have 3 users, going to 4 adds only a few connections). But later, adding the 10-millionth user to a platform potentially enables millions of new interactions. This creates an incentive for even more users to join (because "that's where everyone is"), often leading to winner-takes-all dynamics in tech.

It's important to note that Metcalfe's Law is a simplified model. In practice, not every user connects with every other (social networks form clusters, and inactive users add less value). As networks grow large, the incremental value of additional users can diminish.

Origins

Metcalfe's Law is named after Robert Metcalfe, who proposed the concept around 1980 while discussing Ethernet and network adoption. Metcalfe was quantifying the informal understanding of network effects (e.g., a telephone is useless with one user, but incredibly useful when everyone has one).

In 1983, he presented the idea to 3Com (his company) in terms of the number of "compatible communicating devices," arguing that the network's value grows as the square of that number.

The term “Metcalfe’s Law” itself was popularized later, in a 1993 *Forbes* magazine article by George Gilder. Gilder used Metcalfe’s diagram of networks to coin the law’s name and draw attention to the explosive value of networking (at the time, to explain the rise of the Internet).

Examples

A classic example of Metcalfe’s Law comes from telecommunication: telephone networks. One telephone by itself is worthless, as there’s nobody to call. With 2 telephones, you have one connection. If a company gets 10 phones, suddenly, dozens of internal calls are possible.

In the domain of social media, with platforms such as Facebook, early on, if only a few of your friends were members, Facebook’s value for users was low. But as soon as *everyone you know* was on Facebook, it became hugely valuable. You could share updates, photos, and events with your entire social circle in one place.

The number of possible social connections (friend pairs, group conversations) grew roughly in proportion to the square of the number of users. With 500 users, the network had ~125,000 possible connections. With 1,000, that number jumped to ~500,000. This is why Facebook’s value skyrocketed with user growth.

Researchers at Tsinghua University checked Metcalfe’s Law against Facebook’s actual data and found the fit to be remarkably accurate (Zhang et al., 2015). In 2006, it had 12 million users and $52 million in revenue. By 2009, users grew 25× (to 300 million), but revenue grew 600× (to $777 million). That’s quadratic scaling, not linear. A

75× increase in users from 2006 to IPO corresponded to roughly 5,625× in squared terms, and revenue tracked accordingly.

The study also revealed something Metcalfe didn't predict: Costs scale quadratically, too. More connections mean more storage, computation, and moderation. But the key insight is that value grew faster than cost. Once Facebook crossed that break point, the surplus compounded. That's what made the network so hard to compete with.

The same pattern holds for *WhatsApp* or *WeChat*. Each new user can open chats with many others, and large group chats become possible as the user base grows. This network effect locked users in: It's hard to leave a platform where "everyone already is," because no alternative can offer the same web of connections unless it too achieves similar scale.

We also see Metcalfe's Law in modern collaboration platforms or developer ecosystems. Take GitHub as an example. Its value to developers is that *everyone is there*. Millions of developers (nodes) means there are countless project collaborations, code libraries, and knowledge-sharing links forming between them.

It's worth noting negative examples, too: Metcalfe's Law can work *against* a network in a death spiral. If users start leaving a social network (say, it becomes uncool or trust in it erodes), the value drops faster than linearly. Each departure reduces the potential connections for all who remain.

Metcalfe's Law works both ways: If a network hits critical mass and users stay connected, it grows. If not, it shrinks.

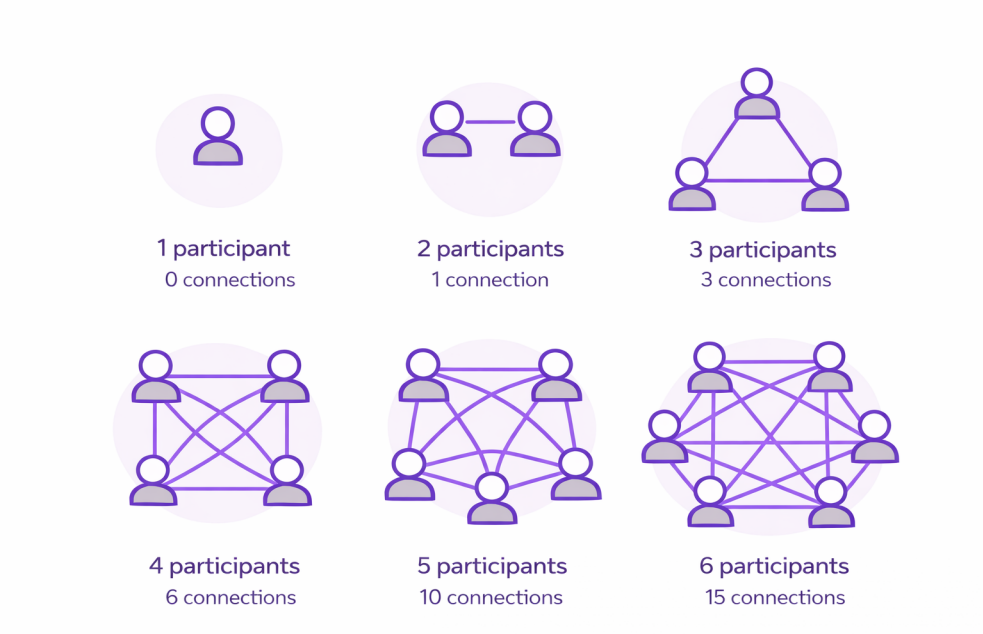

Figure 38.1 **Lines of Communication and Team Size**

Related to

- Amdahl's Law
- Gustafson's Law

Further Reading

- Gilder, G. (1993, September 13). Metcalfe's Law and legacy. Forbes ASAP. https://www.discovery.org/a/41/
- Metcalfe's law. (n.d.). Wikipedia. https://en.wikipedia.org/wiki/Metcalfe%27s_law
- Simeonov, S. (2006, July 26). *Metcalfe's Law: More misunderstood than wrong?* HighContrast Blog. https://blog.simeonov.com/2006/07/26/metcalfes-law-more-misunderstood-than-wrong/

- Zhang, X. Z., Liu, J. J., & Xu, Z. W. (2015). Tencent and Facebook data validate Metcalfe's law. *Journal of Computer Science and Technology, 30,* 246–251.

Sidenote: Network effects concept (Sarnoff's Law and Reed's Law)

Sarnoff's Law models broadcast networks, where value scales linearly with audience size ($V \propto n$), and each new follower adds reach, not interaction.

Reed's Law was proposed by David P. Reed (MIT), and it models group-forming networks. With this law, potential value can scale exponentially with the member count, since the number of possible subgroups grows rapidly (2^n). Any new channel, server, or team creates many overlapping communities for members.

These two laws separate two product modes: distribution and collaboration. Newsletters, podcasts, docs, and status pages fit Sarnoff. On the other side, Slack, Discord, and collaborative tools fit Reed once group creation and coordination sit at the core.

The following issues come from mixing models and metrics:

- Group fragmentation and abandoned spaces
- Moderation and governance debt
- Weak discovery, where users miss the groups worth joining
- Vanity growth, where member count rises activity per group falls

Healthy products have the correct structure. For Sarnoff, this includes track consumption and return: open rate, watch time, repeat visits, unsubscribe, and user retention. For Reed, it includes track group health: active groups, retention per group, posts per active group, join-to-first-contribution time, and moderation load.

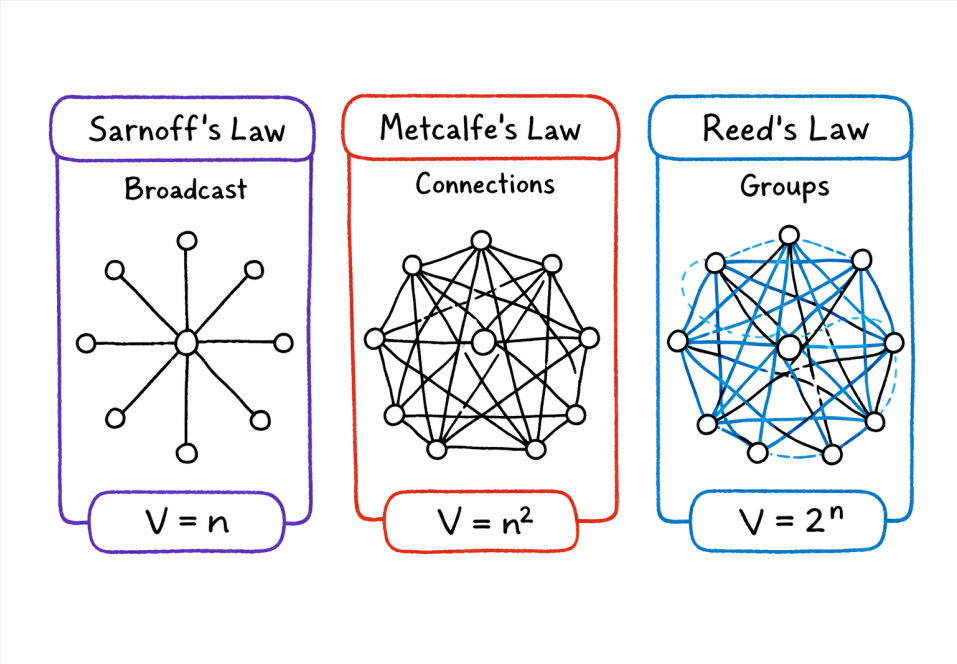

Figure 38.2 Sarnoff, Metcalfe, and Reed's Laws

OceanofPDF.com

Part VI
Coding and Design Principles

These principles push toward small units with clear boundaries and avoid unnecessary work and abstraction.

OceanofPDF.com

39. DRY (Don’t Repeat Yourself)

Don’t Repeat Yourself (DRY) says that every piece of knowledge must have a single, unambiguous, authoritative representation.

Key Takeaways

- *The DRY principle is all about avoiding duplication of knowledge in code. If the same idea or logic is located in multiple places, it’s a signal to refactor. There should be one source of truth for each piece of information.*
- *When requirements change, you should update logic in only one place. Repeated code increases the risk of inconsistencies and bugs (you might fix a bug in one copy and forget about the others).*
- *Apply DRY wisely. It’s about knowledge repetition, not just duplication. Sometimes two pieces of code look alike but do different jobs; merging them could overcomplicate things.*

Overview

The idea of the DRY principle is simple: *Each fact or piece of logic in your system should be expressed once and only once*. If you find the same code, formula, or rule located in multiple modules, you’re violating DRY.

Such repetition may seem harmless at first (copy-paste is easy), but it creates a hidden maintenance cost. When the business rule behind

that code changes, you have to hunt down every duplicate instance and update it.

By following the DRY principle, developers strive for a single and unique implementation of any given behavior. This often means *abstracting standard code into a function or class*, or consolidating configuration data into a single file instead of duplicating it.

For example, if the tax calculation logic is used in three different places, DRY suggests putting it in a single module and having all parts call it. This way, when tax rules change, you edit one module instead of three.

DRY also applies to other forms of knowledge, such as database schemas, tests, and documentation. If you generate documentation from code, you adhere to DRY (writing things once). If you have to update a comment in multiple places to reflect a code change, that's a DRY violation.

A DRY approach promotes even more modular code that works well together. This gives each piece of code a specific role, ensuring that every piece of functionality has a single source. This approach fits well with other design guidelines, such as Single Responsibility (every piece of code deals with one thing, leading to little overlap with other code). This approach also yields a positive effect: Less code means fewer errors.

However, it is essential to remember that the DRY principle is not concerned only with microoptimizing similar pieces of code. The DRY principle is based on duplicated intent. Two pieces of code can look identical, but they have different purposes in an application.

Hence, the application of DRY requires good judgment, meaning apply it when you have the same knowledge twice.

> “Most people take DRY to mean you shouldn’t duplicate code. That’s not its intention. The idea behind DRY is far greater than that.”
>
> —Dave Thomas

Origins

The term “DRY” was coined in 1999 by Andy Hunt and Dave Thomas in their book *The Pragmatic Programmer*. They phrased it as the “DRY principle,” defining it as follows: “Every piece of knowledge must have a single, unambiguous, authoritative representation within a system.”

This crystallized a long-standing ethos in programming. Even earlier, the idea of avoiding code duplication was part of structured programming and the Unix philosophy (“store data in one place,” etc.).

In the Extreme Programming community, a similar idea was known as *Once and Only Once*. Hunt and Thomas gave it a catchy name and expanded its scope beyond just code to all information.

Examples

Let’s say that an application has a database connection URL that appears in five different source files. If the database host changes, you’d have to edit all five places. Until you do, some parts of your app break. The DRY approach is to keep that URL in a single

configuration variable or file and have all modules read it. That way, a single edit updates the URL across the whole system.

In coding practice, duplications often occur when two developers tackle similar problems in different modules, or when you copy and paste to quickly reuse code. For instance, if two parts of an app both format dates in a specific way, you might see two functions doing the same string manipulation. A DRY fix would be to create a single utility function, such as `formatDate()`, and call it from both places. Now, if the date format needs to change (say from `MM/DD/YYYY` to ISO `YYYY-MM-DD`), there is precisely one function to modify. All code that uses it will immediately receive the new behavior.

Let's say you have a tax calculation formula replicated in three locations: order processing, invoice generation, and cart display. The three locations used a tax calculation formula, but with a fixed rate of 10%. The company decided to move to Canada, and a different tax rate was applied in different provinces. One developer changed tax calculation in the order processing, while another changed tax calculation in invoices a week later, leaving the cart unchanged. It took 3 days to identify the bug, as no one knew that tax calculation was replicated in three locations.

The fix was to pull out the `TaxCalculator` class that these three consumers called. When the company later expanded to the EU with VAT rules, the change only took place in one place. Otherwise, it would have meant finding every place where the computation of tax happens, something which would have been impossible because of the duplication of logic in unexpected places.

Large codebases especially benefit from DRY through techniques such as *libraries and frameworks*. For example, in an enterprise, if multiple applications need to implement the same security logic or business rule, the organization might create a shared library instead of copying it across applications. Each app then calls into the library.

Finally, we can see it in the documentation, too. Let's say that you maintain an architecture diagram in a PDF and also in a README. If you update one and not the other, your documentation is inconsistent. The DRY mindset suggests keeping a single source (perhaps the diagram is embedded or referenced, or the README generates it from code metadata).

Figure 39.1 **The DRY Concept**

Related to

- SOLID Principles
- The Law of Demeter

Further Reading

- Don’t repeat yourself. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Don%27t_repeat_yourself
- McConnell, S. (2004). *Code complete* (2nd ed.). Microsoft Press.
- Thomas, D., & Hunt, A. (1999). *The pragmatic programmer: From journeyman to master.* Addison-Wesley.

OceanofPDF.com

40. KISS (Keep It Simple, Stupid)

Designs and systems should be as simple as possible.

Key Takeaways

- *KISS (Keep It Simple, Stupid) primarily refers to avoiding complexity in solution designs. A solution that satisfies the requirement and is simple in design is better than a complex one.*
- *Code that is simple is easier and faster to understand and debug. It's even easier to understand and fix if you're your own future self. Smarter code may look impressive at first, but it often masks problems.*
- *Keep the code simple and easy to monitor. The code solution for the problem at hand should be as simple as possible. Anything that adds complexity unnecessarily is counter to the KISS Principle.*

Overview

The KISS principle is basically a reminder that *simplicity should be a key goal*.

For instance, if you can solve a problem with a 50-line script versus a complex 500-line solution, KISS tends toward the 50-line solution (all else being equal) because each line of code has the potential to cause an error.

Why is simplicity so important? Because software has to be understood by humans. A simple design is *much easier to maintain*, where new team members can get up to speed faster, bugs are easier to localize, and modifications cause fewer ripple effects.

Simplicity also often correlates with robustness. The more special cases you have, the higher the chance that something goes wrong in an unanticipated way.

There's also a productivity angle in here. Simple solutions can often be developed and tested faster.

The KISS principle encourages developers to resist the temptation to write clever code that tries to do too much at once, or to architect solutions that address future problems at the cost of current complexity. Instead, *focus on clarity and the essentials*: Solve today's problem as straightforwardly as possible.

Origins

The phrase "Keep It Simple, Stupid" comes from the U.S. military. It's attributed to Kelly Johnson, a lead engineer at Lockheed's Skunk Works in the 1960s.

Johnson reportedly challenged his design team: Design an aircraft that could be repaired with a basic set of tools by an average mechanic in the field. This meant the design had to be *robust and simple*.

Influential programmers such as Edsger Dijkstra and C.A.R. Hoare often discussed complexity. Hoare (1981) said, "There are two ways

of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies. The first method is far more difficult."

This quote captures the essence of the KISS Principle.

Examples

Let's say that we have a web application that needs to generate some reports. A KISS approach would be to write a simple script or use an existing library to query the database and output a CSV or HTML. A non-KISS approach would be to design a complete plug-in architecture or a domain-specific language for reporting, just in case you need it more generically later. If the requirements are currently small, the latter is overkill and will slow down development and debugging.

Here is another scenario. Let's say you're writing a function to parse a file. A KISS implementation might use simple logic: open the file, read line by line, split fields, and handle errors. A more complex approach might implement a fully generic parser–combinator framework (with a design pattern) to handle any possible format, with layers of abstraction. Unless you truly need that level of generality, the simpler approach will be easier to get right and maintain.

A startup needed a feature flag solution to toggle features on/off in production. What they got was a 6-week project to implement a separate microservice that included a database, a cache, admin tools, WebSocket-based notifications, and A/B testing integration.

The result was over 7,000 lines of code. One Saturday night, the service failed, and all features were turned off.

What they actually needed was a JSON config file with `{"new_checkout": true, "dark_mode": false},` 40 lines of code. The 6 weeks they spent building overengineered infrastructure could have gone toward more valuable work for users.

It's telling that many experienced engineers will advise, "First make it work, then make it right, then make it fast (if needed)." The "make it work" part often implies a KISS solution first, something clear and basic, before even considering fanciness.

In algorithm design interviews or coding, candidates sometimes try overly complex solutions to impress, but the best answer is often the simplest correct one. KISS would guide you in choosing the simplest data structure and approach that meets the requirements.

Figure 40.1 **The KISS Principle**

Related to

- YAGNI
- The Principle of Least Astonishment
- Gall's Law

Further Reading

- Hoare, C. A. R. (1981). *The emperor's old clothes* [1980 ACM Turing Award lecture]. Communications of the ACM, 24(2), 75–83.
- McConnell, S. (2004). *Code complete* (2nd ed.). Microsoft Press.
- Ousterhout, J. K. (2018). *A philosophy of software design*. Yaknyam Press.

OceanofPDF.com

41. YAGNI (You Aren’t Gonna Need It)

Don’t add functionality until it is necessary.

Key Takeaways

- *YAGNI (You Aren’t Gonna Need It) reminds developers to focus on current tasks rather than implementing features that aren’t needed at the moment.*
- *Anticipating future needs often leads to overengineering, which adds complexity and maintenance issues.*
- *YAGNI encourages iterative development. You implement minimal solutions and refine or extend them when required.*

Overview

YAGNI captures a core philosophy of agile development: Don’t write code for features that haven’t been requested or aren’t immediately needed.

This means that, for example, if you’re implementing Module A and you think, “Hmm, in the future we might need Module B to do X, so I’ll code some hooks for that now,” YAGNI advises avoiding that, because that future feature may never come, or it may change drastically by the time it’s needed.

This principle directly fights *overengineering*. Many developers (especially enthusiastic or experienced ones) tend to generalize solutions “just in case.”

To successfully apply YAGNI, teams rely on *confidence in refactoring*. You defer a feature only if you trust you can add it later when needed at low cost. Agile methods provide that safety net via good test coverage, refactoring tools, and continuous integration.

In essence, YAGNI pushes the problem of complexity to when it's actually needed.

Origins

YAGNI was part of the Extreme Programming (XP) paradigm that arose in the late 1990s. It was promoted by people like Ron Jeffries (1998), one of the founders of Extreme Programming, himself, who wrote, "Always implement things when you actually need them, not when you just foresee that you need them."

It has been found that this slogan was adopted in XP literature, such as *Extreme Programming Installed* by Jeffries, Anderson, and Hendrickson (2001).

Examples

Imagine you're building a feature with specific behavior. You think, "Maybe in the future someone will want to toggle this behavior on/off, so I'll add a configuration flag for it now." YAGNI would challenge that. So, unless there's a current requirement for configurability, implement it with only expected behavior, nothing less, nothing more. You can always introduce a config option later if a real need arises.

Skipping the config flag avoids extra code, unnecessary test branches, and confusion about a setting no one has asked for.

For example, you're working on a library function, which currently performs exactly one thing. You realize this might be useful in a more general way, so you might add parameters or abstraction tiers to support it. YAGNI would say you should implement it for the original task you need it for. For example, your app needs to output JSON, so you implement a simple JSON export module. You might decide to implement a full-blown serialization library for exporting, say, XML, YAML, or whatever, but based on YAGNI, you shouldn't do so until necessary. Your app only has to do JSON.

I remember when, in one project, I used the Abstract Factory Pattern as an abstraction layer so that our app could easily switch between SQL Server and Oracle database (one day). However, we never used Oracle. We just introduced more complexity, with no real benefits. Note here that when you have in your head "one day," but you're unsure when that day will come, you probably need to adhere to YAGNI.

Also, once I was called by our client to create an employee directory (with names, departments, phone numbers) that was to be imported from a spreadsheet 20–30 times a month. During planning, one person asked, "What if HR wants to edit profiles via the web later?" So, they went ahead and built the entire CMS (role-based access controls, WYSIWYG, image uploads, drafts, publishes, revisions, approval, etc.). Their database expanded from 4 to 17 tables. But in reality, HR didn't use any of it. They just kept modifying their spreadsheet. The import every month, every 10 minutes, worked just

fine. After 2 months, ~120,000 dollars in engineering hours were all spent on features that nobody needed. The CMS was ripped out entirely. It's amazing how much easier a codebase can be 60% smaller, all because of something that should never have been implemented.

A positive effect of YAGNI is seen with *refactoring*. Teams that have adopted the YAGNI principle are often very comfortable refactoring because they rely on it. For example, you have a bit of duplication or an imperfect design now, but you don't generalize early; you trust that when the feature set grows, you will refactor those two similar bits into one. And because you waited, you had more information about the real use cases, which led to a better abstraction than you would have designed initially.

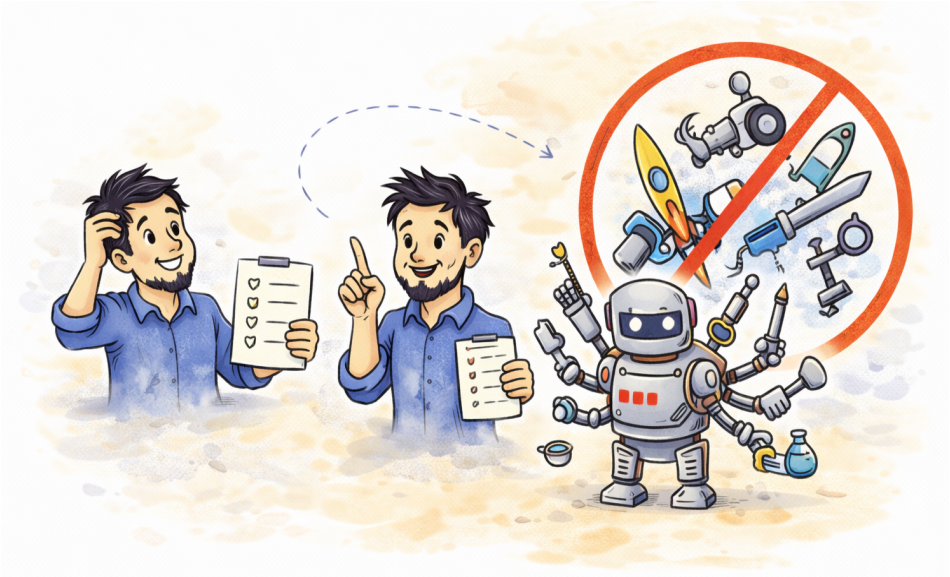

Figure 41.1 **The YAGNI Principle**

Related to

- The KISS Principle
- DRY

Further Reading

- Fowler, M. (2015, May 26). *Yagni*. https://martinfowler.com/bliki/Yagni.html
- Jeffries, R. (1998). *You're NOT gonna need it!* https://ronjeffries.com/xprog/articles/practices/pracnotneed/
- Jeffries, R., Anderson, A., & Hendrickson, C. (2001). *Extreme programming installed.* Addison-Wesley.

OceanofPDF.com

42. The SOLID Principles

The SOLID principles are five main guidelines that enhance software design, making code more maintainable and scalable.

Key Takeaways

- *Following SOLID helps produce software that is easier to extend, test, and refactor without breaking existing functionality. Each principle addresses a different aspect of good OO design.*
- *A class with a single responsibility (SRP) that depends on well-defined interfaces (ISP, DIP) and uses inheritance appropriately (LSP) and polymorphism for extensions (OCP) will likely be easy to maintain.*
- *Changes to one part of the system won't cascade into breakages elsewhere, because the code is loosely coupled and encapsulated well.*
- *SOLID does not guarantee a perfect design, but it provides proven guidelines.*

Overview

The SOLID principles are five high-level guidelines for object-oriented design, created by Robert C. Martin in the early 2000s. They have since become a cornerstone of good OO programming practice.

Here is how each principle contributes to overall software quality:

- **Single-Responsibility Principle (SRP):** *Each class or module should focus on a single concern*. If a class is handling multiple, unrelated jobs, it tends to grow large and complex (often called a “god class”), and any change to one responsibility can affect the others.
- **Open/Closed Principle (OCP):** This principle was formulated by Bertrand Meyer. It says that software intended to be maintainable must be open for extension but closed for modification, which implies that one can extend the code without modifying it, and it can be implemented using abstraction techniques such as interfaces and abstract classes, as well as polymorphism.
- **Liskov Substitution Principle (LSP):** LSP was proposed by Barbara Liskov. It focuses on the proper use of inheritance. According to LSP, a subclass should support the same operations that a user of the superclass expects.
- **Interface Segregation Principle (ISP):** It is related to the level of detail of the interface. It says, “Clients should not be forced to depend on interfaces that they don’t use,” implying that you should break a general interface into specific ones for a particular role.
- **Dependency Inversion Principle (DIP):** “Depend on abstractions, not on concretions.” The purpose is for high-level modules (business logic) not to depend on low-level modules (details), while they both rely on abstractions (interfaces or abstract classes).

When applied cumulatively, the SOLID principles are designed to provide a system that is modular, extensible, and robust under

change.

Origins

The five SOLID principles emerged over the years and were collected and popularized by Robert C. Martin (Uncle Bob). The acronym SOLID itself was coined around 2004 by Michael Feathers, who noticed that the initial letters of these five principles (as named by Martin) spelled out a catchy word.

Uncle Bob described SRP, OCP, LSP, ISP, and DIP in various articles from the late 1990s and early 2000s (for example, in his “Principles of Object-Oriented Design” papers and his 2002 book *Agile Software Development: Principles, Patterns, and Practices*).

These principles were drawn from earlier thinkers: OCP comes from Bertrand Meyer (1988), LSP from Barbara Liskov (1987), ISP from work at Xerox PARC (the interface separation idea), and DIP and SRP were Martin’s formulations (inspired by common sense and earlier practices about layering and cohesion).

Examples

For the *Single-Responsibility Principle (SRP)*, consider a simple web application that manages user accounts. An anti-SRP design might have a single class, `UserManager`, handling everything, such as validating user input, saving to the database, sending confirmation emails, logging activities, and similar. According to SRP, that class has multiple reasons to change (business-logic, database, email-content, and logging-policy changes).

A better design would separate these: for example, `UserValidator` (validation logic), `UserRepository` (database operations), `EmailService` (email sending), and a higher-level `UserRegistrationService` that orchestrates those smaller pieces. Each class has one focus. If the email format changes, you only touch `EmailService`.

This modular approach is also more straightforward to test (you can test `UserValidator` independently of the others). SRP violations often manifest as giant classes (often with "Manager" or "Utility" names); breaking them down makes the system cleaner.

Another example is the *Dependency Inversion Principle (DIP)*. Let's say we have a notification system in which a `NotificationService` sends alerts to users. Then, without DIP, `NotificationService` could directly use the `EmailSender` or `SMSSender` classes. Or, if you were to introduce push notifications tomorrow, you'd be forced to change `NotificationService` to use a `PushSender`. This inflexibility is where DIP helps.

For DIP, you would define an interface `INotificationChannel` with a method `send(message, recipient)`. `EmailSender`, `SMSSender`, and `PushSender` all implement this interface. The class `NotificationService` will depend on the `INotificationChannel`. It might be configured with a list of `INotificationChannel` objects. When it's time to send, it just calls the `channel.send(msg, user)`. To add a new channel, you create a new class that implements `INotificationChannel` and,

perhaps, inject it via configuration; you don't touch the `NotificationService` code at all.

Moreover, in tests, you could make a `TestNotificationChannel` that just records messages instead of actually sending, and inject that into the `NotificationService` to verify logic without sending real emails.

DIP often involves frameworks or patterns: for example, using an IoC (Inversion of Control) container to bind interfaces to implementations at runtime. It decouples the wiring of things from the business logic.

For the Liskov Substitution Principle (LSP), there is a good example in Java. Let's consider `java.sql.Timestamp` extending `java.util.Date`. You would imagine, given this relationship, that you could use Timestamp and Date interchangeably. Clearly, the nanosecond-level precision of Timestamp is important to sql databases. But the behavior of the `equals()` method of Timestamp is certainly not symmetric. Consider `date.equals(timestamp);` it returns true, while `timestamp.equals(date)` returns false for the exact same moment. Clearly, we violate the contract of the method signature. This is, of course, the contract of the method signature of `Object.equals()`, which the Date version complies to but Timestamp does not. So, we get no exception, no warning, just incorrect results, which hides the fact that our logic is faulty. As the Java documentation says, "*It is recommended that code not view Timestamp values generically as an instance of java.util.Date*." Sun

Microsystems admitted, “We broke the LSP principle on this method signature.”

Note that SOLID can sometimes be overdone. For example, too many tiny interfaces or abstract layers can lead to complexity (the infamous “AbstractFactoryFactory” problem).

When used correctly, though, these principles are genuinely helpful in managing object-oriented system complexity, which is why they’ve become so popular to teach.

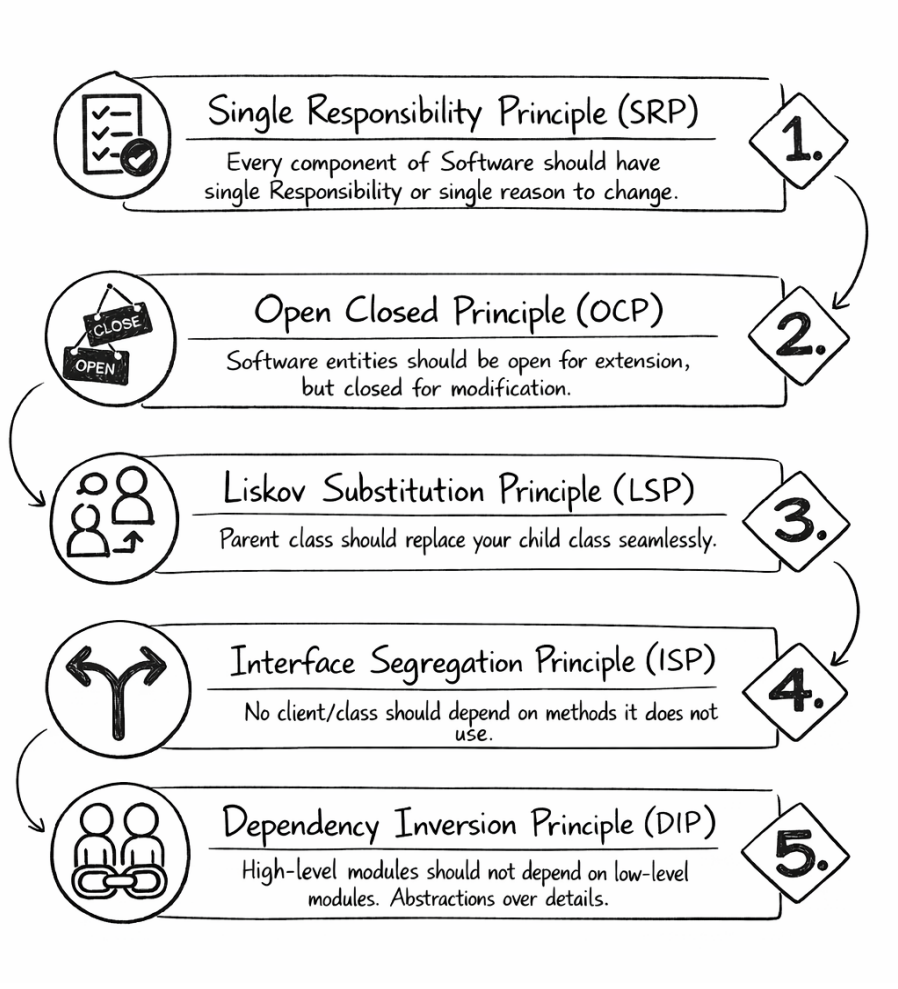

Figure 42.1 **The SOLID Principles**

Related to

- The Law of Demeter
- Hyrum's Law
- The Law of Leaky Abstractions
- DRY
- The KISS Principle
- YAGNI

Further Reading

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Martin, R. C. (2000). *Design principles and design patterns.* Object Mentor. (Originally published as a white paper.)
- Martin, R. C. (2002). *Agile software development: Principles, patterns, and practices.* Prentice Hall.
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design.* Prentice Hall.

OceanofPDF.com

43. The Law of Demeter

An object should only interact with its immediate “friends.”

Key Takeaways

- *An object should only call methods of itself, its direct components, its function parameters, or objects it creates. It should not navigate through one object to reach another.*
- *If Object A only calls B (its immediate friend) and doesn’t reach into B’s internals (like C), then changes to C or removal of C does not affect A. Each class knows as little as possible about others, which reduces the effect of changes.*
- *This principle often leads to adding wrapper methods or delegations. While that might increase the number of methods, interactions are cleaner.*

Overview

The Law of Demeter is sometimes mentioned as “don’t talk to strangers” or “principle of least knowledge.” It was formulated in the context of object-oriented programming to minimize the knowledge that any given object has about the overall system structure.

In practice, this law says that if Object A has a reference to Object B, and B has a reference to C, A should not call methods on C through B (like `a.b.getC().doSomething()`). A can talk to B (its friend), and B can talk to C. Still, A shouldn’t directly get involved with C. By

doing so, *each object only communicates with its immediate collaborators* and is unaware of any deeper connections.

This principle is good because it fosters loose coupling. If you break LoD, your code assumes the structure of the world extends beyond its immediate neighbors.

For example, if `OrderProcessor` does `order.getCustomer().getAddress().getZipCode()`, it knows that an `Order` has a `Customer` that the `Customer` has an `Address`, and so on. If the `Customer` class is changed to have multiple addresses or no address (structure changes), `OrderProcessor` breaks, although logically, one might argue order shouldn't worry about how customer addresses are structured.

If, instead, `Order` had a method `getShippingZip()` that internally navigates its customer/address, `OrderProcessor` could just call that and be oblivious to any structural changes behind the scenes.

This law also aligns with the idea of *information hiding*. Each object should hide its internals and only expose necessary interfaces. If others aren't reaching through it to subobjects, that hiding is respected.

However, as pointed out by some, strictly following this law can lead to an explosion of small methods (such as many forwarding methods just to avoid one extra dot in a call). There's a concept called *Demeter transmogrifiers* (lots of trivial delegations), which implies that classes can sometimes be bloated. However, this also often indicates that lessons can be structured differently so that

navigation isn't needed. If you find yourself writing a lot of delegators, maybe those behaviors belong in the original class.

Origins

The Law of Demeter was first described around 1987 by Ian Holland and colleagues at Northeastern University, as part of the *Demeter Project* on adaptive and aspect-oriented software development. It was named "Demeter" after the project, which itself was named after the Greek goddess of agriculture, thus signifying a growing approach to software.

The researchers (including Karl Lieberherr) sought ways to make software more maintainable and developed this style rule. They summarized it in a motto, "Only talk to your friends."

Examples

A good example would be UI element interactions. Suppose a `Presenter` object has a reference to a UI View, which in turn contains some `TextField` object. Without LoD, the presenter might do something like `view.getTextField().setText("Hello")`. LoD would encourage that View provides a method such as `setUserMessage(String)`, and internally, it would do `textField.setText`.

This way, the Presenter doesn't need to know there's a TextField. The detail is encapsulated in the View. If the `View` later changes to use a `Label` instead of a `TextField`, the `Presenter` code doesn't change. Had it been reaching in, it would have broken or required change.

Avoiding long chains also improves readability. It is often possible to identify a violation simply by counting the dots in a call: `a.getB().getC().doSomething()` has two dots. Typically, the Law of Demeter would say that's one too many. A single dot `a.doSomething()` or at most `a.getB().doSomething()`, where B is a direct friend, is preferable. Fluent APIs are designed with that chaining in mind, so one could argue the Law of Demeter is not violated in spirit because each call returns a new context object, which you are directly calling next.

Let's say that a team had to create an order processing system, and they made `OrderService` dependent on a customer's country code. A programmer wrote, `order.getCustomer().getAddress().getCountry().getCode();`

This train wreck implied that `OrderService` knows the internal structures of four objects. `OrderService` had to reach inside `Order`, `Customer`, `Address`, and `Country` just to obtain a string.

Here is a full example:

```
public double getShippingCost(Order order) {
    String countryCode = order.getCustomer()
                              .getAddress()
                              .getCountry()
                              .getCode();
```

```
                                                    return
shippingRateService.getRateForCountry(countryCode)
;

}
```

The cost was felt immediately. When the `Customer` refactoring changed to separate billing and shipping addresses, the absence of the original `getAddress()` method caused 23 methods across 14 classes to fail because they all shared the same deep chain. A second change, which replaced the string of `Country`'s code with an enum, broke every call that chained to `.getCode()`. A couple of internal refactors rippled through the entire codebase.

The fix? Add `getShippingCountryCode()` method as a direct method on `Order`. `OrderService` calls one method on one object. Hence, when `Address` or `Country` changes internally, we only need to change the `Order` class, just one time in one place, instead of 23 changes over 14 locations.

Here is a full solution example:

```
// On Order class

public String getShippingCountryCode() {

                                                    return
customer.getShippingAddress().getCountry().getCode
();

}


// In OrderService
```

```
public double getShippingCost(Order order) {
    return shippingRateService.getRateForCountry(
        order.getShippingCountryCode()
    );
}
```

Overall, by adhering to the Law of Demeter, *each component remains more self-contained*. Systems become a network of friends rather than a tangle of strangers reaching through each other.

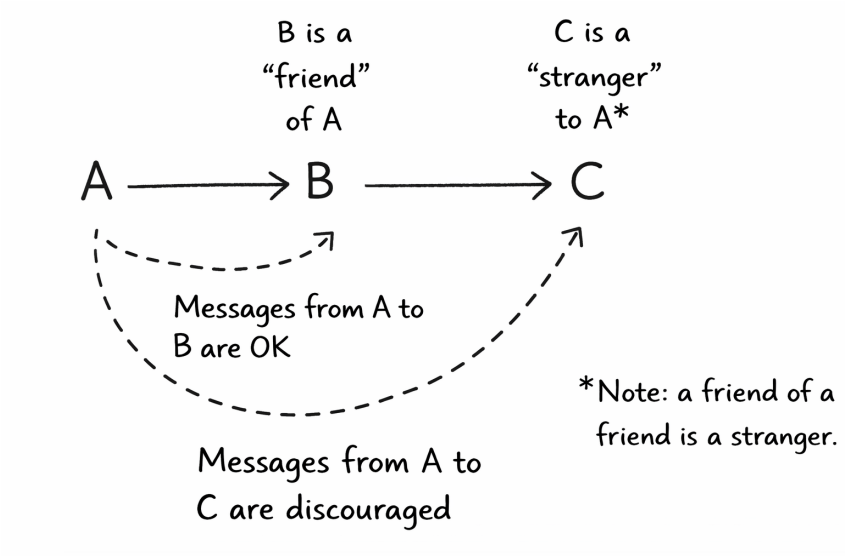

Figure 43.1 **The Law of Demeter**

Related to

- SOLID Principles
- The Law of Leaky Abstractions
- Hyrum's Law
- The KISS Principle

Further Reading

- Bock, D. (n.d.). *The paperboy, the wallet, and the Law of Demeter.* https://www2.ccs.neu.edu/research/demeter/demeter-method/LawOfDemeter/paper-boy/demeter.pdf
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software.* Addison-Wesley.
- Lieberherr, K. J. (1996). *Adaptive object-oriented software: The Demeter method.* PWS Publishing.
- Lieberherr, K. J., Holland, I. M., & Riel, A. J. (1988). *Object-oriented programming: An objective sense of style.* Northeastern University.

OceanofPDF.com

44. The Principle of Least Astonishment

Software (and interfaces) should behave in a way that least surprises users and other developers.

Key Takeaways

- *Design decisions should align with user expectations. When someone uses a component (a UI element, API, etc.), its behavior should not be surprising or counterintuitive.*
- *Following platform norms or standard conventions yields the least astonishment.*
- *In practice, this principle improves usability and developer experience. Users can predict how to interact with the software, and developers can predict how to use an API by analogy with similar ones. Surprises often lead to mistakes.*

Overview

The Principle of Least Astonishment (POLA) is a general design principle in user interface design, software API design, coding, documentation, and more.

The idea is simple: *Don't surprise the user*. Or better, it means that the system should behave in a way that *matches the user's mental model* of how it ought to work. When the user or another developer using your code makes an assumption, the software should validate

that assumption if possible. Violating this principle doesn't necessarily break functionality, but it does break trust and ease of use.

In UI/UX, this principle results in *intuitive interfaces*. For example, clickable buttons should look like buttons. Links are usually underlined and blue; if you style a link as plain text without any indication, users might not realize it's clickable.

In coding, this principle means your code should behave as the developer expects when name, type, and context are taken into account. We should choose defaults and behaviors that match standard conventions and avoid surprise side effects.

One often-quoted consequence of the principle is: "If a necessary feature has a high astonishment factor, it may be necessary to redesign the feature." This statement implies that if something is inherently complex or unexpected, it is necessary to consider a different approach or add clear affordances to set expectations.

Origins

The concept of least astonishment has roots in early human–computer interaction and even earlier in general system design. An early mention by that name appears in the *PL/I programming language documentation around 1967*, complaining that certain behaviors of PL/I violated the Law of Least Astonishment.

The principle was explicitly stated in a 1972 publication on the design of programming languages, which recommended that

language constructs behave as their syntax suggests and follow widely accepted conventions.

So, initially, it was in the context of language design (make languages consistent and unsurprising).

Since then, it has been independently discovered or emphasized across various domains. It's also often cited in the Unix community (e.g., Eric Raymond's *Art of Unix Programming* references it as the "Rule of Least Surprise").

Examples

In software APIs or libraries, POLA manifests as designing functions, classes, and behaviors that *meet standard expectations*. For instance, if you have a method `deleteFile()`, one would expect it to remove the file. It should not, say, archive it somewhere secretly or throw an error if the file doesn't exist (silently doing nothing might be more expected in some contexts).

Good API design also employs this concept. If your method works similarly to a well-known one in another library, keep naming and behavior aligned. For example, in many languages, the `toString()` method returns a human-readable representation. If one class's `toString()` returned a complex binary blob of data, that would violate expectations.

JavaScript's `sort()` method is a good example. If you use `[1, 10, 2, 21].sort()`, you might expect to get `[1, 2, 10, 21]`, but what you'll actually get is `[1, 10, 2, 21]`. The method is converting all the values to strings and sorting by that. So `"10"`

comes before `"2"` because `"1"` comes before `"2."` It's a reasonably well-known behavior, but every developer who sees this for the first time is surprised by it. An API that is called "sort" and is being called on an array of numbers really should be sorting numbers, not performing a sort that depends on character values. That silent failure is a big part of why this is an anti-pattern. The correct behavior is what a normal person would expect, not what makes the code easy to program. The solution is simple: `[1, 10, 2, 21].sort((a, b) => a - b)`. But the very fact that it is needed is the problem. Every developer will spend time debugging this, which simply shouldn't take any time in the first place.

POLA covers topics such as *error handling and defaults*. Sensible defaults cause the least surprise. If a field is optional and you omit it, the system should default to the most commonly expected behavior. Surprising behavior often arises from edge cases or overly clever design.

For developers, *code surprises* can be just as problematic. If a function is named wrongly or a module has side effects not clear from its name, it violates least astonishment. For a function `parseDate(str)`, we could expect it to return a `Date` object or throw if unable. If instead it modifies a global date format setting internally, that's surprising.

In development teams, POLA can apply to code readability, for example, when following standard naming conventions (a variable named `isReady` should be a boolean, a function named `compute` should likely return something rather than modify global state, etc.). Surprising naming or patterns cause developer confusion.

The net effect of following POLA is a better user experience and less user errors. When users aren't fighting unexpected behaviors, they can use the software more effectively.

Basically, a delighted user (or developer) uses something and it "just works" as they expected. POLA is the guiding principle to achieve that outcome.

Figure 44.1 **Code Quality Measurement**

Related to

- Hyrum's Law
- The KISS Principle
- Postel's Law
- The Law of Demeter

Further Reading

- Bergeron, R. D., Gannon, J. D., Shecter, D. P., Tompa, F. W., & Van Dam, A. (1972). *Systems programming languages*. In M. Rubinoff & M. C. Yovits (Eds.), *Advances in computers* (Vol. 13). Academic Press. https://linkinghub.elsevier.com/retrieve/pii/S0065245808605100
- Bloch, J. (2006). *Bumper-sticker API desig*n. Sun Microsystems.
- Raymond, E. S. (2003). *The art of Unix programming*. Addison-Wesley.
- Southworth, R. N. (1967). Proposal for PL/I pseudo-name (R. N. Southworth, Ed.). IBM Technical Report.

OceanofPDF.com

Part VII
Decision-Making and Cognitive Biases

These laws and biases explain bad judgment, story-driven reasoning, and how to choose simpler, safer moves.

OceanofPDF.com

45. The Dunning–Kruger Effect

The less you know about something, the more confident you tend to be.

Key Takeaways

- *Confidence without context is unreliable. Early confidence often signals ignorance, not mastery.*
- *Awareness grows faster than skill. Learning initially reduces confidence, only to rebuild it.*
- *Real experts speak in ranges, trade-offs, and probabilities, not with confidence about every topic.*

Overview

The Dunning–Kruger Effect explains *a gap between confidence and competence*. When people know little about a domain, they lack the awareness required to judge their own ability. Consequently, they overestimate how well they understand the problem.

As learning progresses, people discover the depth and complexity of the domain.

Awareness of unknowns grows faster than capability, leading to a drop in confidence, often called the “valley of despair.” Only after experience does confidence rise again, now with fundamental understanding.

And this may be a familiar concept to all of us. When you started learning about something, after some time, you may have thought, “I know it all.” But shortly after, you may have hit the wall where you did not know some basic stuff. Finally, you realized that you in fact do not know anything.

Origins

The effect was described by psychologists David Dunning and Justin Kruger in a 1999 study at Cornell University. Their experiments showed that people with low performance consistently overestimated their ability on logic, grammar, and humor tests, while high performers underestimated their ability.

The proposed mechanism is simple. The skills required to perform well are often the same skills needed to assess performance accurately.

Examples

A good example is when new developers give confident, precise estimates, while experienced developers give ranges (the famous “*it depends*” answer). The juniors aren’t wrong to be convinced, they simply don’t yet know what they don’t know (i.e., *unknown-unknowns*).

Here is a personal story. Once I worked with a developer (let’s call him Pat), who wrote not so great code, with a lot of bad naming, messiness, frequent code duplication, and so on. He was not able to understand his own code after a few months. Yet when he was rated poorly by his manager, he said, “I’m one of the best programmers in

this department. Your performance review form is broken." This was not just a defensive statement, but his belief. And because he didn't know what good looked like, he brushed off every suggestion as a waste of time from people who didn't get "real coding." The irony was, when our company finally hired an experienced senior engineer, one who knew many, many things, he would frequently comment, "I'm not sure, let me check." And it was not that he was humble, but he was acknowledging all he didn't know. Pat knew very few things and was convinced he knew everything. That, of course, is part of the problem: Incompetence is not only about failure, but also about blindness regarding it.

Another example is tech enthusiasm. Peak enthusiasm for a new technology often comes from those who have used it least (Who said managers?). Those with deep experience are more careful. This is something we see today about AI trends. Most of the people who believe AI can do anything are people who don't work directly with those technologies.

We should also note that *impostor syndrome* reflects miscalibration in the opposite direction. Skilled individuals underestimate their competence because they are deeply aware of complexity and edge cases.

What usually happens is that

- juniors overestimate themselves, and
- seniors underestimate themselves.

Healthy teams try to deal with both by grounding decisions in evidence, reviews, and shared learning rather than self-perception.

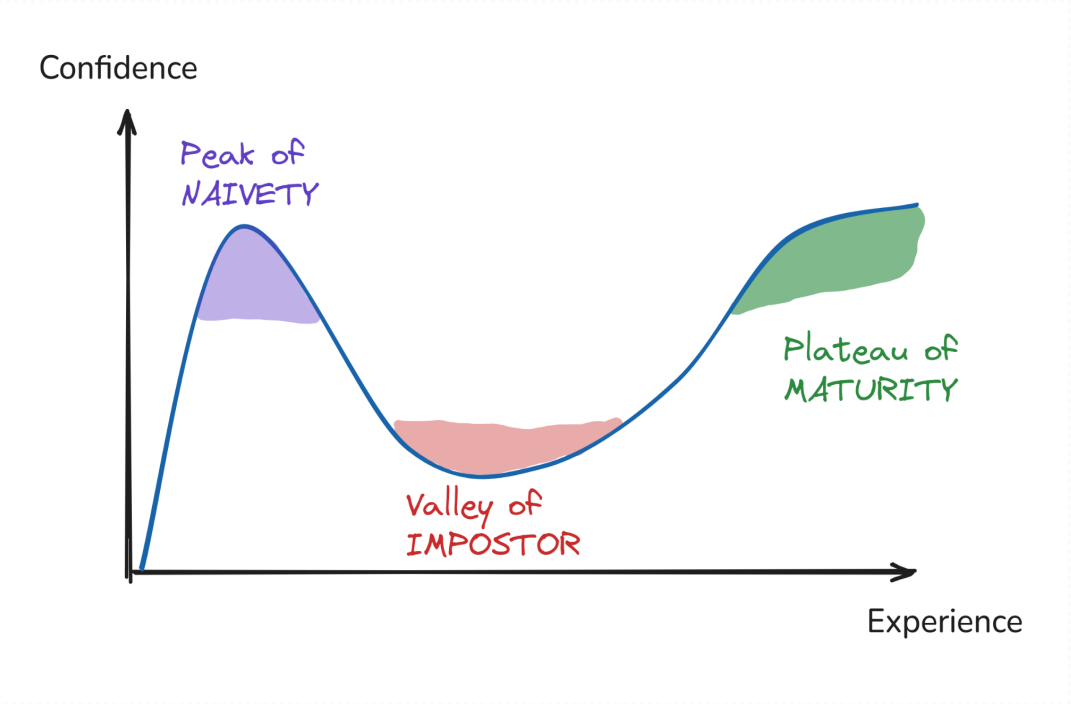

Figure 45.1 **The Dunning–Kruger Effect**

Related to

- Brooks's Law
- Hanlon's Razor

Further Reading

- Dunning, D. (2014, October 27). *We are all confident idiots. Pacific Standard.* https://psmag.com/social-justice/confident-idiots-92793/
- Dunning, D., & Kruger, J. (1999). Unskilled and unaware of it: How difficulties in recognizing one's own incompetence lead to inflated self-assessments. *Journal of Personality and Social Psychology, 77*(6), 1121–1134.

- Dunning–Kruger effect. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Dunning%E2%80%93Kruger_effect

Sidenote: Impostor Syndrome

Impostor syndrome is the tendency of competent people to believe they are inadequate and attribute their success to pure luck or even fraud.

In software engineering, this phenomenon is most evident in top performers. As engineers gain experience, they develop a better sense of complexity, edge cases, and how things can fail. But greater knowledge can bias self-evaluation because the more you know, the more you realize you don't know.

Contrary to the Dunning–Kruger Effect, impostor syndrome arises out of expertise. Impostor engineers experience a self-perceived incompetence relative to others' perceived skill, believing they are getting left behind, whereas they are actually well ahead.

The problem is not self-doubt, but what might follow from it:

- Hesitation to speak up in design discussions
- Lack of leadership or ownership
- Overpreparation and burnout

Healthy teams are aware of the concept and incorporate it in their structure. They tend to base their confidence on peer reviews and outcomes rather than self-assessment.

OceanofPDF.com

46. Hanlon's Razor

Never attribute to malice that which is adequately explained by stupidity (or carelessness).

Key Takeaways

- *When something goes wrong (a bug, a coworker's action, a user's input), it's likely not malicious, but rather an error or misunderstanding.*
- *Don't jump to "the system is hacked" or "someone intentionally broke this." First, consider simpler explanations (e.g., a configuration file was missed or there was a typographical error).*
- *If a colleague's code is problematic, it's probably not sabotage. It may be due to being rushed or unaware of something. Try to approach the issue at hand with questions, not accusations.*

Overview

Hanlon's Razor is a principle that advises against assuming bad faith when an outcome may be due to human error or ignorance. In software, it's a sound guiding principle both technically and in team culture.

For instance, if a commit introduces a security hole, it's probably a mistake, not that the developer wanted to introduce a vulnerability. For this reason, it is necessary to respond with help and fixes, not with blame or malice.

In debugging production issues, if data is corrupted, it could be a bug rather than “a hacker broke in,” unless evidence points that way. This principle encourages rational, evidence-based thinking, focusing on the likely cause (which is often simpler).

However, this doesn’t mean ignoring the possibility of malice (security does consider malicious actors). But for everyday development and operations, Hanlon’s Razor keeps you grounded: The build failed likely due to a misconfiguration, not because someone deliberately sabotaged it.

Origins

Robert J. Hanlon submitted the concept to a 1980 compilation of Murphy’s Law variations. It echoes similar statements by Napoleon and Goethe about not attributing malice where incompetence can explain it.

Examples

A customer reports that your software “deleted all my data!” It’s easy to panic and think that a disgruntled employee might have put a logic bomb inside. But Hanlon’s Razor would have you first check for a simple bug (e.g., perhaps there is a boundary condition in a delete function that wipes more than intended, i.e., a logic error). Indeed, you find a bug where, when a user’s account name was empty, the cleanup script deleted all records. No malice, just a bug.

Another common scenario is when code review feedback comes off as harsh. You might feel the reviewer is being mean. But maybe they were just in a hurry or not good at phrasing (not malicious, just

clumsy in communication). So, instead of firing off an angry email, you talk and clarify. And then you really find out they meant well.

Once, in one of our teams, a junior developer deployed a database migration on a Friday afternoon. This caused the API to stop working for thousands of users. What he did was drop one column that production still needed. The reaction of the team was to say, “Who deploys on a Friday? He just does not care.” But instead of jumping to conclusions like most of his team did, the manager took the time to speak to him. What she found was that he had actually followed an outdated wiki page that did not have one mention of the team’s rule not to deploy on Fridays. He tested it on staging, and it all passed, but what Marcus did not know was that production ran an older version of the service that still needed that column.

However, the real problem wasn’t this junior developer. The real problem was that there was a lack of docs and that the CI failed to test against production by default. What was needed wasn’t corrective action against him, but better team documentation or a CI pipeline that defaults to testing against production. So, the problem was not the developer or his experience, but the bad process.

In security, consider a sudden spike in server traffic. Before screaming “it’s a DDoS attack!” check first whether it’s a misconfigured client spamming or a known pattern (maybe your own monitoring caused it). Most of the time, weird behaviors in systems are due to mistakes (e.g., someone pushed debug logging to verbose, which filled the disk).

Hanlon's Razor teaches you to investigate methodically. Of course, once you rule out the stupid, then consider malice. But it's often a waste of energy to assume the worst intent first.

This principle also helps preserve the spirit of goodwill within teams and among consumers.

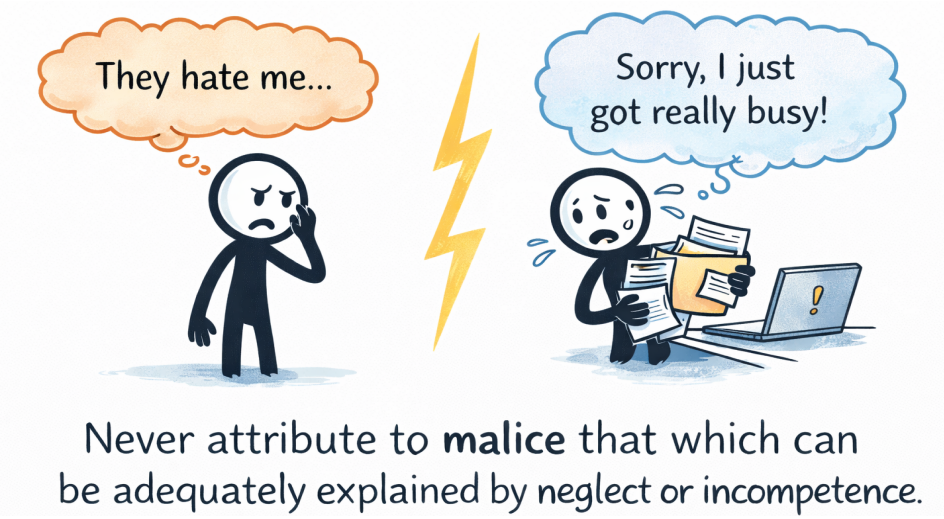

Figure 46.1 **Hanlon's Razor**

Related to

- Occam's Razor
- Goodhart's Law
- Postel's Law

Further Reading

- Dekker, S. (2014). *The field guide to understanding human error* (3rd ed.). CRC Press.
- Etsy Engineering. (2012). *Blameless postmortems. Code as Craft.* https://www.etsy.com/codeascraft/blameless-

postmortems/
- Hanlon's razor. (n.d.). *Wikipedia*. https://en.wikipedia.org/wiki/Hanlon%27s_razor

OceanofPDF.com

47. Occam's Razor

The simplest explanation is often the most accurate one.

Key Takeaways

- *Simple code is easier to understand, maintain, and debug, whereas complex code has more potential points of failure. Remove or avoid unnecessary moving parts in system architecture.*
- *When debugging, start with the simplest explanation, such as a misconfiguration or an off-by-one error, before jumping into it.*
- *Adding more features to the app can only complicate a product without adding real value. Occam's Razor reminds us that less is more in many cases.*

Overview

Occam's Razor is originally a problem-solving principle that reminds us that if multiple solutions are possible, the simplest one is preferred.

In software, this principle supports the *KISS ("Keep It Simple, Stupid")* approach. When we decide to keep our implementations simple, we reduce the risk of further complications (i.e., bugs).

For example, if a software architecture can achieve its goals with a monolith and one database, there is no need to split it into five microservices and two databases. These extra components would

only introduce complexity and potential integration and coordination issues, but no benefits.

This is a reminder to avoid the temptation of overengineering and keep things simple.

Origins

Occam's Razor dates back to William of Ockham, a philosopher from 14th-century England. His original version in Latin, *Pluralitas non est ponenda sine necessitate*, can be translated to *"plurality should not be posited without necessity,"* or *"Don't multiply entities or assumptions beyond what's required."*

Occam's Razor has become a cornerstone of scientific methodology and logical reasoning over time.

Examples

One of our projects offered a particularly good example of Occam's Razor in action. Our team identified that the web application was only slow on Mondays. We spent a week trying to debug the issue. We thought that one of the cron jobs might be causing the problem because it was consuming all the disk space, or maybe that the weekend log rotation might have corrupted the cache. Also, we considered that there was just a general overload from the week's traffic on the load balancer. Finally, someone checked the obvious metrics for the server. It turns out that the monitoring agent ran a full disk scan every Monday at 9 AM, which spiked the CPU to 100% for 10 minutes. It was an easy fix. The problem wasn't complex. It was a scheduled task that no one remembered setting up.

Here is another example. Let's say that the web application is found to be successfully operating as a single service. Why add microservices that could make deployment and debugging unnecessarily complicated when we can improve reliability by re-simplifying its architecture? According to Occam's Razor, we could try to solve the problem in a simpler way, such as by using optimized queries or caching. As Einstein said, "Everything should be made as simple as possible, but not simpler," which best applies to this law in engineering.

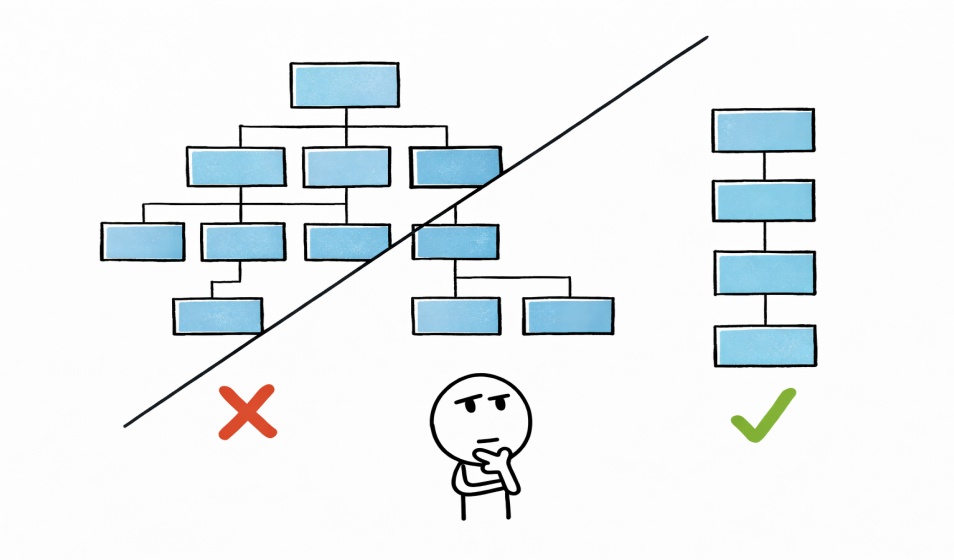

Figure 47.1 **Occam's Razor**

Related to

- The KISS Principle
- YAGNI
- Gall's Law

Further Reading

- Lipton, P. (2004). *Inference to the best explanation* (2nd ed.). Routledge.
- Sober, E. (2015). *Ockham's razors: A user's manual*. Cambridge University Press.

OceanofPDF.com

48. The Sunk Cost Fallacy

Sticking with something just because you've invested in it only keeps you stuck.

Key Takeaways

- *Don't be stuck based on past investments. Understand when something is not paying off. The mere fact that "we've invested a year in it already" is not an excuse to continue if it's clearly not working.*
- *Healthy engineering organizations pivot or stop projects that no longer make sense. Continuing a bad project just because of sunk costs usually leads to greater loss.*
- *One way to avoid sunk-cost bias is to use clear measures and decision points (e.g., the rule "If we don't meet performance target X by the end of next quarter, we'll re-evaluate or discontinue the project").*
- *Teams should feel safe admitting a project isn't working without fear of blame.*

Overview

The Sunk Cost Fallacy is fundamentally a *cognitive bias* that challenges our decision-making. In economics and psychology, it's well documented that humans irrationally value what they've already invested (money, time, or effort), even when those investments can't be recovered.

A common situation in software development is teams that continue to work on a feature or product without making progress, simply because “we have come this far.”

Let’s assume that an enterprise software development project has gone 50% over budget and has some issues. The proper approach would be to redesign or even cancel such a project, but some people will resist it because of the millions already spent. They don’t want to waste the prior investment, even though it is harmful for the company. This often leads to *project death marches*, in which increasingly more resources are used, with diminishing returns.

Understanding the Sunk Cost Fallacy allows engineers and managers to make more rational choices: Past costs should be *ignored* when deciding the future. Instead, consider current facts and future benefits. If a rewrite or a switch to a third-party solution will clearly save time and money *from this point forward*, Occam’s Razor-like simplicity would be to do that, regardless of how much was sunk into the old approach.

Overcoming sunk-cost bias is tough because it requires admitting that past efforts were in vain. But in technology, as in many other domains, it’s crucial to *learn to let go* when evidence demands.

Origins

The concept of sunk costs comes from economics. The term *sunk cost fallacy* was popularized by behavioral economists and psychologists in the late 20th century.

Notably, Nobel laureates Daniel Kahneman and Amos Tversky studied this and related decision biases in the 1970s. Behavioral economist Richard Thaler officially coined the term around 1980.

Examples

Let's consider a company that has spent 2 years building a custom in-house Content Management System (CMS), only to find it buggy and dragging down team productivity. A better analysis might show it's wiser to adopt an off-the-shelf CMS (classic buy vs. build problem). Yet the engineering leadership might hesitate and say, "We invested 2 years into this. If we abandon it now, it means that time (and money) was wasted!" If they persist solely for that reason, they fall for the Sunk Cost Fallacy.

In one of our projects, our team built custom deployment tools based on the Cake build system (in .NET), because we could not find anything that would fit our needs back then. But the ecosystem caught up. GitHub Actions and ArgoCD could do everything that our tool does, even more. Meanwhile, the internal tool eats 20% of one engineer's time, and breaks in ways nobody fully understands. But whenever someone suggests switching, the answer is always, "We've already built so much around it." We were not weighing the ongoing maintenance tax against the one-time migration cost. We were anchored to years of past effort. Here's the test: If nothing existed today, would you choose to build this tool from scratch? If the answer is no, the only thing keeping you there is inertia disguised as investment. Almost every team has one of these: A custom tool, a homegrown ORM, and an internal platform that survives not because it's the best choice, but because it already exists.

A positive example is when companies *cut losses,* such as Google. For instance, they are known for periodically killing off underperforming products (see the Google Graveyard at https://killedbygoogle.com/). In essence, this practice means that companies don't need to move forward with bad decisions, once it's clear an initiative isn't working.

On a daily level, developers see this when debugging or implementing a feature. One might spend a whole day on the wrong approach and not get results. The Sunk Cost Fallacy says, "Keep at it. Otherwise, today was a waste!" But sometimes the better move is to step back, try a different approach, or seek help from someone. This abandons the sunk cost of the initial effort.

Also, teams that conduct premortems or use agile sprints to reassess work are better at avoiding sunk-cost traps. They can improve the process by asking, "Knowing what we know now, should we continue on this path?" and are willing to change course if the answer is no.

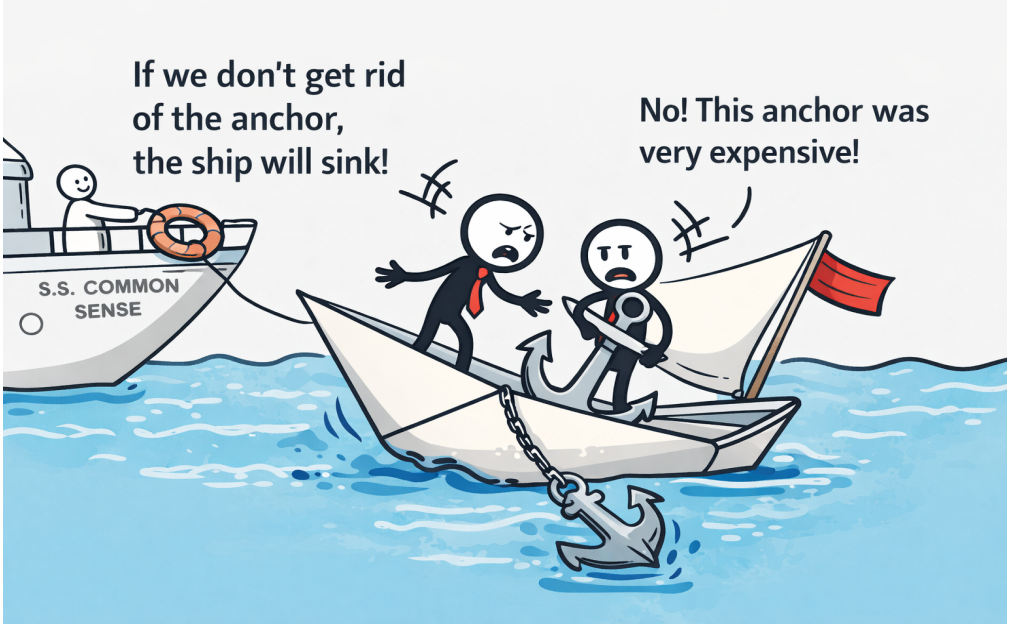

Figure 48.1 **The Sunk Cost Fallacy**

Related to

- Occam's Razor
- Technical Debt
- The Second-System Effect

Further Reading

- Arkes, H. R., & Blumer, C. (1985). The psychology of sunk cost. *Organizational Behavior and Human Decision Processes, 35*(1), 124–140.
- The Decision Lab. (n.d.). *The sunk cost fallacy.* https://thedecisionlab.com/biases/the-sunk-cost-fallacy
- Kahneman, D. (2011). *Thinking, fast and slow.* Farrar, Straus and Giroux.

OceanofPDF.com

49. The Map Is Not the Territory

Our representations of reality are not the same as reality itself.

Key Takeaways

- *Know that your design docs, UML diagrams, or architecture schematics are abstractions. Don't confuse the blueprint with the actual running software.*
- *When implementing a system, expect that unforeseen factors will emerge that weren't captured in the initial designs. Be prepared to adapt the plan as you discover new terrain during development and testing.*
- *Models and designs are still valuable. They guide development and decision-making, but you should be willing to question assumptions and revise the model when evidence from the real system contradicts it.*

Overview

The Map Is Not the Territory is a mental model originating from the field of general semantics. It encapsulates the idea that our perceptions or conceptual models of things are not the things themselves.

In software, we constantly create maps: A requirements document is a map of user needs, and an architecture diagram is a map of how

components *should* interact. Our mental understanding of a codebase is a personal map of how we think the code works.

These are all useful and even necessary. Without any maps or models, we couldn't plan or reason about complex systems. However, problems arise when we forget the map's limits.

A typical example would be overconfidence in the initial design. The team has designed a system on paper, laying out the modules and the data flow between them. But when the time comes to begin coding, they realize that specific modules cannot communicate over the network due to latency issues, not because of assumptions they'd made earlier. It doesn't invalidate the design phase. It just means the team has to iterate on the design based on observation.

As the famous statistician George Box said, "All models are wrong, but some are useful."

Origins

The phrase was popularized and used effectively in "Science and Sanity" (1933) by Alfred Korzybski, a Polish–American scholar. Here, Korzybski highlighted how our language and knowledge mislead us into believing our constructs actually represent reality.

"The map is not the territory" became a central mantra in general semantics and later in systems thinking.

Examples

This concept can be demonstrated in a software application if we consider the relationship between design diagrams and code. For

instance, let's say that we have a scenario in which you are required to produce a diagram representation of a new microservice system that involves communication from Service A to Services B and C, using Kafka topics. Perhaps the diagram assumed "the network is reliable" or "latency is negligible," but in the real territory of cloud infrastructure, those assumptions fall apart. The team then updates the map (design) to include a retry mechanism or a schema validation step.

Here is an example in performance modeling. Let's assume that an engineer might model a database capable of handling 10,000 queries per second based on specs (a map of expected performance). In production, though, due to the specific query patterns and data distribution, it yields only 5,000 qps. The model didn't account for certain query plan edge cases; reality was more complex. In response, the engineer refines the performance model and perhaps adds monitoring to better align future expectations with observed reality.

In machine learning, a trained model is literally a map of training data to predictions. When deployed, if it encounters data that doesn't fit the training distribution (i.e., the territory has features the map didn't include), the projections can be very wrong.

The Agile methodology itself can be seen as an example of "map vs. territory." Instead of a big 2-year plan, which is a very detailed map made in advance, Agile supports short iterations and takes continuous feedback from the real territory to correct its maps.

In one of our teams, velocity was tracked by a team, who averaged 45 points per sprint. The management ran their projections based on this. For example, they would say, “135 points equals exactly three sprints.” But that was a lie. Without even realizing it, the team had designed their estimations to fit the lie. Simple stories were still estimated with 8 points, while complex stories that took a week were still estimated with just 3 points, due to unexpected complexity. Nobody changed that.

Then we had a critical release that broke down the illusion. The dashboard said on the track, but the Staging environment was marked as broken. Stories were marked as done since the code was merged. However, nothing was working end to end. This is what “map is not territory” means. Points are not time. Velocity is not productivity. A chart displaying 45 points does not show real delivery of value to customers.

The team in the end adopted outcome-based tracking: cycle time, deploy frequency, and features completed instead of story points. True, it was not perfect, but the new map was closer to the territory.

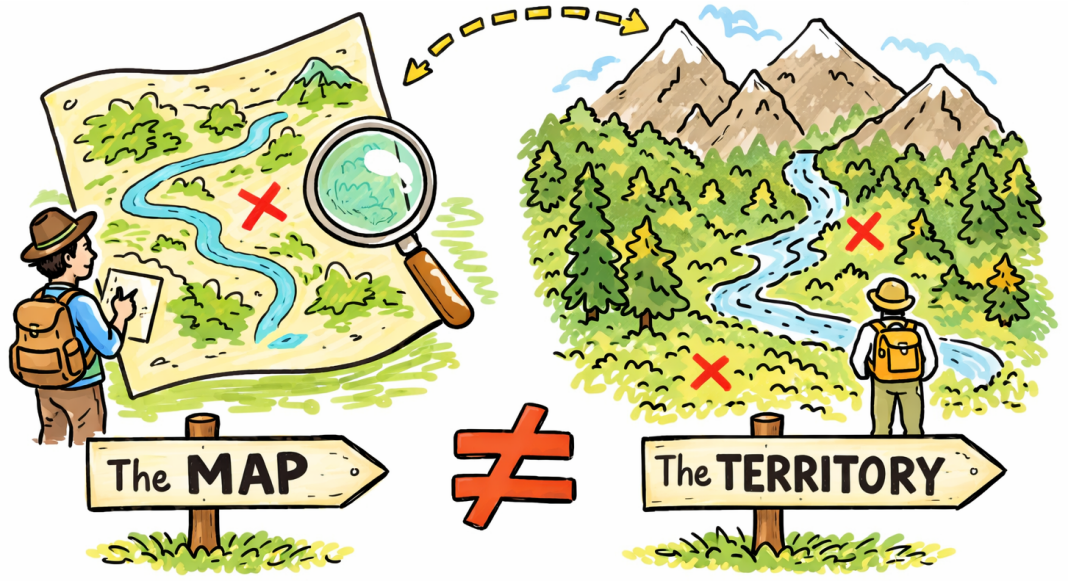

Figure 49.1 **The map is not the territory.**

Related to

- Goodhart's Law
- Gall's Law
- The Law of Leaky Abstractions

Further Reading

- Bateson, G. (1972). *Steps to an ecology of mind*. Chandler Publishing.
- Korzybski, A. (1933). *Science and sanity: An introduction to non-Aristotelian systems and general semantics.* Institute of General Semantics.

OceanofPDF.com

50. Confirmation Bias

It is a tendency to favor information that supports our existing beliefs or ideas.

Key Takeaways

- *When reviewing code or debugging, pay attention whether you're only looking for evidence that supports your initial hunch.*
- *Challenge yourself when actively thinking about problems. If you have an opinion about an issue, ask, "What would I expect to see if I'm wrong?" By posing this question, we can counteract our natural bias to confirm what we already believe.*
- *Regarding team decision-making (e.g., tech stacks and/or design decisions), seek input from people with differing opinions. Bias confirmation can be overcome by looking into alternative solutions. For example, if the whole team feels that technology X is the superior choice, assign someone to look into the opposing view on technology X. Base decision-making on objective criteria (facts, not opinions). Automated tests, performance criteria, and experimentation (A/B tests) can provide truths regardless of human bias.*

Overview

We all like to be right. Confirmation bias is essentially our mind's way of cheating to feel right more often. Psychologically, once we form

an opinion, we subconsciously filter information, noticing the bits that support our view and ignoring those that contradict it.

In software, a typical scenario happens during debugging. Suppose a developer is convinced that a recent change in Module A caused a production issue. They will likely comb through Module A's code and logs intensively. If logs in Module B, which the developer assumes is fine, are also throwing errors, the developer might not even look there, thus missing the real cause, which might be in Module B.

A lesson that has emerged from design conversations is that once designers fall in love with their own approach, they are likely to overlook evidence that another approach might be better.

Awareness of this bias has led designers to incorporate these critical thinking skills into their practices. This involves asking, "What am I missing? Maybe there is another explanation?" encouraging development of where one's beliefs are called into question.

Origins

Psychologists have studied confirmation bias for decades. Peter Cathcart Wason, an English cognitive psychologist, was one of the first to identify it.

In 1960, he conducted a famous experiment (called Wason's rule discovery task) in which participants had to guess a rule for a sequence of numbers. What he found was that people tended to test sequences that would confirm their initial thoughts rather than those that could prove them wrong.

Wason created the term “confirmation bias” to explain this phenomenon. Since then, countless studies have confirmed the existence of confirmation bias in human reasoning.

Examples

In code reviews, a reviewer who trusts a colleague’s skills might skim over potential issues, assuming the code is probably fine (confirming the belief that the colleague writes good code). Or the opposite can happen: A reviewer expecting sloppy code from a junior developer might find issues that aren’t really important.

Confirmation bias also affects testing. A developer may write unit tests that assert the code works as expected on typical inputs (following the happy path), but might not try edge cases that could break it.

In one of our projects, a developer saw API response times spike after a Friday deploy. Nancy was skeptical of a colleague’s database query change, so she already had her theory. She filtered the dashboard for database metrics, found slower queries, and spent 2 days optimizing them. Each fix helped the query but didn’t touch the actual problem.

The real cause was sitting on the same dashboard she’d been staring at: A third-party API had degraded from 100ms to 500ms on Friday, the same day as the deploy. It was a pure coincidence. A colleague who had no prior knowledge of it spotted it in minutes, because he was looking at the full picture. This is a classic example of confirmation bias. Nancy decided what was wrong before proper

investigation and only saw the materialization of her decision. The data was right, but she had already decided what it meant.

In a team context, confirmation bias can appear in planning meetings. Suppose the team has a belief that “users aren’t interested in Feature X” (perhaps because one big client said so). They might notice all the feedback that complains about Feature X and ignore input from users who actually love it. This biased perception could lead them to remove a feature that a minority of users value. To combat this, product managers often look at data (i.e., usage analytics) rather than evidence to get a more objective picture.

On the positive side, teams have developed habits to reduce confirmation bias:

- Code reviews can challenge obvious assumptions with a fresh pair of eyes.
- Testing practices where developers write tests specifically aimed at breaking their own code are also helpful.
- In post-mortems, teams ask, “What went wrong, and why didn’t we see it?”

By checking for confirmation bias, engineers can become better at solving problems with an open and critical mind.

Figure 50.1 **Confirmation Bias**

Related to

- The Dunning–Kruger Effect
- Hanlon's Razor
- Goodhart's Law

Further Reading

- Nickerson, R. S. (1998). Confirmation bias: A ubiquitous phenomenon in many guises. *Review of General Psychology, 2*(2), 175–220.
- Wason, P. C. (1960). On the failure to eliminate hypotheses in a conceptual task. *Quarterly Journal of Experimental Psychology, 12*(3), 129–140.

OceanofPDF.com

51. The Hype Cycle & Amara's Law

We tend to overestimate the effect of a technology in the short run and underestimate the impact in the long run.

Key Takeaways

- *Early on, people overestimate what the tech can do in the near term, which leads to unrealistic expectations.*
- *In the longer run, the same technology's true potential emerges, and it is often greater than initially imagined. Amara's Law reminds us that we tend to underestimate the long-term effects of a significant innovation.*
- *For developers and tech leaders, this means not getting swept up in the hype cycle. Be patient and evaluate new tools by their proven value, not just the buzz.*
- *Treat innovation as a limited budget. Good teams use stable, proven tech for most needs and adopt a hyped new technology only when it truly addresses a real problem.*

Overview

In practice, this law is the familiar boom-and-bust *hype cycle* seen in tech. Gartner's Hype Cycle model captures five stages a new technology typically follows:

1. **Technology trigger (spark).** A new technology emerges, and people and the media start talking about it. In reality, it cannot ship any real results.

2. **Peak of inflated expectations (hype)**. Organizations begin to invest in developing the new technology. Others still wait for more evidence.
3. **Trough of disillusionment.** Organizations start to invest in experiments and development. Their actual outcomes are currently far less than what they had anticipated. While some businesses reduce their losses, others persist in falling victim to the illusion of sunk costs.
4. **Slope of enlightenment.** At this point, we have a better understanding of the technology and what it can do. Products have advanced far beyond their initial prototypes. Businesses still make development-related investments.
5. **Plateau of productivity.** Now, the new tech has become mainstream. The majority are not only aware of the latest technology but also have embraced it. The long-term relevance of the technology increases with a deeper understanding of its effect.

This Gartner's Hype Cycle is essentially Amara's Law, only in graphic form. The peak represents our overestimation of the near term, while the trough represents the time when many misjudge the long-term potential that will eventually be achieved.

In software engineering, it is essential to take this law into account. This means that many buzzword technologies (a new framework, methodology, or tool) are initially promoted *as game-changers*. Still, they usually decline in popularity when they show that they cannot deliver promised results.

However, after that initial reality check, some of these technologies quietly *mature* and find their real usability. Some of them even become impactful over time. The lesson for engineers is to have a *healthy skepticism during hype*.

So, don't blindly jump on every new technology that emerges. Also keep an open mind for how it will be accepted in the long run. A technology that's overhyped today might still become invaluable a few years down the line after improvements and real-world learnings.

In other words, don't throw the tech out with the hype's collapse.

Origins

Amara's Law is named after Roy Amara, an American researcher and futurist who was president of the Institute for the Future. Amara was a scientist who began his career as a Navy electronics technician during World War II.

Years later, in 1995, the *Gartner Hype Cycle* framework was introduced by Gartner analyst Jackie Fenn. Fenn had noticed a recurring pattern in the maturity of emerging technologies and created a visual graph to describe it.

Examples

History is full of examples that follow Amara's Law and the hype cycle. A good one is Artificial Intelligence (AI), popular today. AI saw its first big hype wave in the 1960s and 1970s, when people assumed general AI and robotic helpers were just around the corner. When those grand promises didn't show up, AI research funding

dried up. An AI winter set in. Many have abandoned AI as something more theoretical than practical. But over the decades, steady progress continued. Fast-forward to today, AI techniques are indeed transforming industries, from machine learning in healthcare to generative AI in software.

Also, only a few years ago, *microservices architecture* was hyped as the cure for all scaling issues. Later, many teams hit the trough of disillusionment when they encountered the complexity of managing hundreds of services. Now, a few years on, a more balanced, wiser use of microservices is emerging, delivering better benefits in the right situations.

The key takeaway from all these examples is that *timing matters the most*. By remembering the Hype Cycle and Amara's Law, you will know that not all technologies will meet the initial hype. The real effect will become apparent over time.

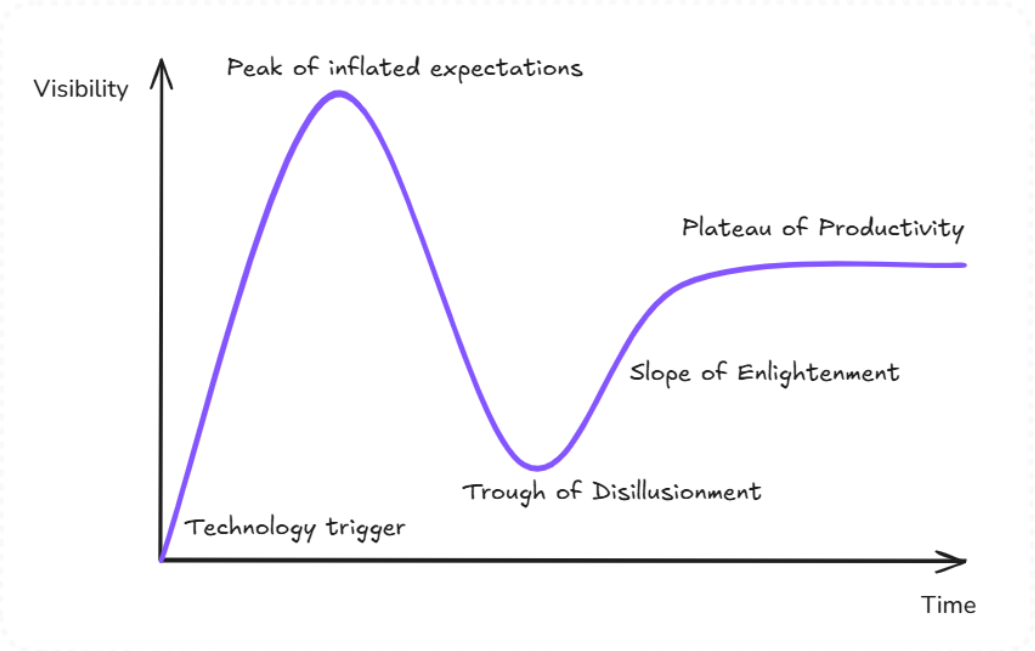

Figure 51.1 **The Gartner Hype Cycle**

Related to

- Goodhart's Law
- Gall's Law
- The Second-System Effect

Further Reading

- Gartner. (n.d.). *Gartner Hype Cycle*. https://www.gartner.com/en/research/methodologies/gartner-hype-cycle
- Institute for the Future. (n.d.). https://www.iftf.org/

OceanofPDF.com

52. The Lindy Effect

The longer something has been in use, the more likely it is to continue being used.

Key Takeaways

- *The Lindy Effect states that if something (e.g., a technology, tool, concept) has persisted for a long time, it will likely continue for much longer.*
- *For developers, Lindy's Law is a reminder to invest in timeless skills and proven fundamentals (algorithms, core languages, design principles) rather than chasing every new framework that might be obsolete in a few years.*
- *Consider the Lindy Effect when evaluating technology. Things that have been around for some time stick around longer in the future (i.e., choose boring technology).*

Overview

The Lindy Effect is a concept that turns our intuition about aging on its head. We know that living creatures wear out over time. We are sure that a 70-year-old person has less life expectancy than a 20-year-old. But in technology, it's often the opposite: The longer something has been in use, the more likely it is to continue being used.

Time acts as a filter here. Anything irrelevant, flawed, or fragile tends to disappear over time, while what remains after many years is

usually high-quality and valuable.

An idea or tool that has survived for decades has shown *robustness* to change or competition, so we can be more confident it will still exist in the future.

This principle has served as a reality check in the software development industry regarding the shiny object syndrome. Technology changes so fast, and we always want to learn about the latest and greatest library or the new tools as soon as they are released.

However, the Lindy Effect also tells us to think longer-term in our assessment of innovative technology. The majority of so-called innovations in the world of software have little to no relevance to contemporary ideas. Rather, they have taken the ideas of the past and reinvented them.

Modern *Agile/Scrum* practices were anticipated by principles the U.S. Air Force pioneered in the 1960s. Object-oriented programming, though popularized in the 1980s, is built on old principles.

The lesson is that truly fundamental ideas in software (e.g., algorithms, data structures, protocols, and paradigms) change slowly. In contrast, many new frameworks or technologies have short-term popularity and then disappear after a few years.

Applying Lindy's law to software means favoring those things that have stood the test of time. For a developer, that might mean learning languages such as Python, Java, or C, as well as algorithms, rather than overfocusing on the hottest new JavaScript framework.

It also involves reading books or papers on technology that are more than 20 years old but remain useful, given the information they provide.

Origins

The term Lindy Effect was coined outside of software. It was created in 1964 by journalist Albert Goldman. Goldman named it after Lindy's Delicatessen in New York City, a famous hangout for entertainers and media folk at the time.

A group of comedians who frequented Lindy's observed a pattern in show business: The longevity of a Broadway show (or a comedian's TV run) seemed to predict its future run length.

In a 1964 article for *The New Republic*, Goldman noted that if a show had already run for X number of weeks, it could be expected to run for about another X weeks. This means that if a show had lasted, the longer it was likely to keep going. This observation about Broadway later became known as Lindy's Law.

Mathematicians and thinkers later popularized the concept. *Nassim Nicholas Taleb* brought it to a broad audience in his books *The Black Swan* (2007) and *Antifragile* (2012). Taleb used it to describe the aging of nonperishable things such as books, ideas, technologies, or traditions.

Examples

A great example of the Lindy Effect in action is COBOL, a language from 1959. Although over 60 years old, it still quietly runs *a large*

amount of the world's finance and government infrastructure. Some estimates are that 70%–80% of all global financial transactions rely on COBOL code running, mainly in banking systems and mainframes.

But, what about the JavaScript frameworks used for web development? Over the past decade or so, many frameworks such as AngularJS, Ember.js, and Knockout.js emerged and fell out of favor. However, the underlying language and APIs that interact with browsers are unchanged. In fact, they have been around for over 25 years and will continue to exist even longer.

One scenario that happened in one of the teams where I worked was when we started a new project in 2014. We chose the then-popular Knockout.js library. A few years later, Knockout was considered outdated. Many teams migrated to newer frameworks such as React or Angular. Yet React itself first appeared in 2013, and because it has survived and gained widespread adoption, it will remain strong for many years to come.

The Lindy Effect is also significant when choosing what to learn. For example, many of the tools hyped on Hacker News a few years ago are already forgotten, but specific skills consistently pay off. If we need to choose between learning a cutting-edge programming framework and SQL (a language from the 1970s), the Lindy approach suggests that SQL is still *highly* relevant today and will likely remain so in the future.

However, AI is undermining this effect today. The Lindy Effect was valid because it was costly and dangerous to replace old systems. A

million-line COBOL codebase won't get refactored if there's no one left who fully grasps its behavior. AI upends this. An agent can analyze old code, comprehend it, and translate it into a new language. The old systems were maintained not out of respect but out of fear of breaking something no one really understood. When AI eliminates this fear, survival history ceases to be a barrier to entry. Basic concepts such as algorithms and data structures remain valid. However, for particular implementations and languages, the answer to "How long has this survived?" becomes, "Is this still the best way to solve this problem?"

In summary, the Lindy Effect teaches us to view time as the ultimate test for choosing technologies. When making decisions, it's wise to ask: *Has this been around long enough to prove its worth?* If yes, there's a good chance it will remain in the future.

After all, if something old is *still here*, that alone says something about its value and resilience.

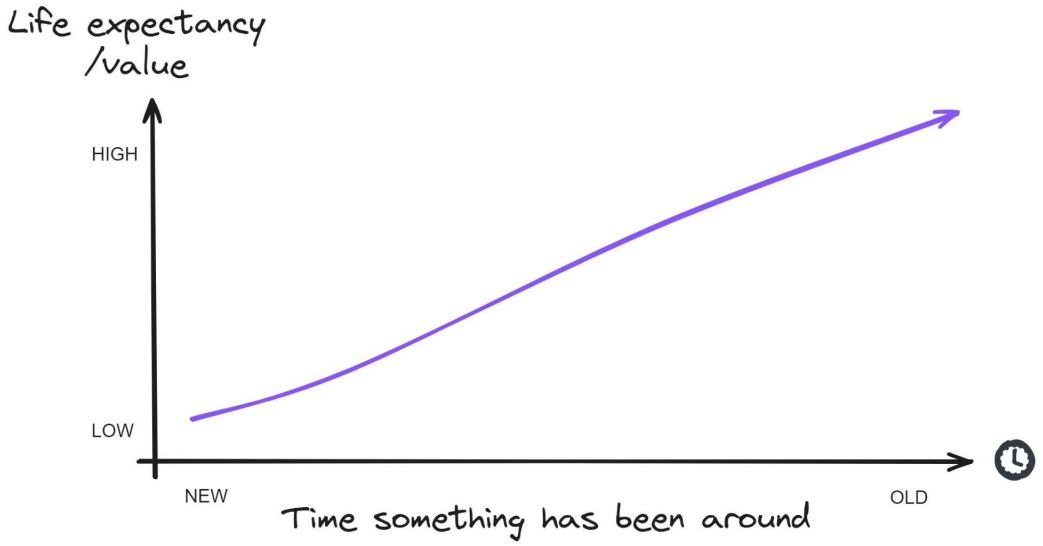

Figure 52.1 **The Lindy Effect**

Related to

- Hyrum's Law
- Postel's Law
- The Hype Cycle & Amara's Law

Further Reading

- Taleb, N. N. (2012). *Antifragile: Things that gain from disorder*. Random House.
- Taleb, N. N. (2014). *An expert called Lindy. In The Bed of Procrustes: Philosophical and practical aphorisms* (Expanded ed.). Random House.

OceanofPDF.com

53. First-Principles Thinking

It implies breaking a complex problem into its most basic blocks and then building up from there.

Key Takeaways

- *Do not accept the problem as it is. Break the problem down into its inherent parts and then question your assumptions: Are they really true, or are they just accepted standards? Once you grasp the underlying requirements, you may develop new solutions.*
- *Just because "everyone uses Framework Y for this" isn't a reason you should do so. The First-Principles Thinking pushes you to ask why people do certain things the way they do.*
- *Rather than saying, "This feature will take 3 months because that's how long similar features took," break the feature down. Estimate from the basics, which can sometimes show that the similar feature had additional issues that you may not have anymore.*

Overview

Without first-principles thinking, we solve problems by drawing on existing solutions ("We'll do X because that other person or project that did X"). This is good for incremental improvements, but it's not suitable for fundamental changes.

For example, when using first principles, one should ask, "What are we really trying to accomplish? What are constraints, and what are

assumptions?" This is an effective strategy, for instance, in debugging and optimization problems.

Let's say that our system is running slower than expected. A usual approach would be, "We need to scale up the servers vertically, because last time we had this problem, scaling helped us." A first-principles approach would go deeper: *What really causes the response time: C*PU processing, I/O, or network latency? Which of those is the actual bottleneck? Perhaps you find that the CPU is 90% idle, but the app spends time waiting on network calls. From that conclusion, a solution might be to batch network calls or compress payloads.

In software design, first principles challenge the way current systems work and ask, "If we were starting from scratch today, how would we build this?" This kind of thinking led to innovations such as microservices (questioning the monolith assumption) and serverless computing (questioning the need to manage servers at all).

The drawback of this approach is that it's mentally intensive and not always necessary. Many problems can be solved by applying known patterns (and our minds like patterns). So, we need to know when to apply first principles versus when to use standard approaches.

Origins

The term *first principles* has its roots in classical philosophy and science. Aristotle referred to "first principles" as the foundational propositions or assumptions that cannot be deduced from anything else, the bedrock of knowledge.

Throughout history, great thinkers have emphasized starting from fundamental truths, like René Descartes, for example (“I think, therefore I am.”) as a basis for philosophy.

In modern times, First-Principles Thinking has gained fame in the business and tech world, partly through Elon Musk, who often credits it for enabling SpaceX and Tesla to innovate. Musk noted that instead of thinking “rockets are expensive, we’ll never afford one,” he broke down the cost of rocket raw materials and realized they could be obtained much cheaper. Then he reasoned that building rockets in-house was the right approach.

Examples

Elon Musk wanted to buy a rocket in 2002 to send humans to Mars. He asked Russian rocket manufactures and received quotes at $65 million per launch. The reason given by the aerospace community was, “rockets are very costly.” Instead, he asked, “What is this thing called a rocket made of (aluminum alloys, copper, carbon fiber, titanium, and inconel, etc.) and how much does this stuff cost as raw material in the commodity market?” Well, as it turned out, it costs 2% of the complete, finished rocket price. This also meant that 98% of the rocket price resulted from inefficiencies, supplier margins, and decades of “cost-plus” government contracting with no reason to ever improve the pricing model.

From that understanding, SpaceX integrated the manufacturing vertically by making 85% of components internally, took lessons from the auto industry, and moved toward rapid testing rather than reviewing for years. The result is Falcon 9 launches priced at about

$67 million, with costs continuing to fall as reusability improves. The Space Launch System from NASA costs $4.1 billion for a single mission. The cost to reach orbit per kilogram has dropped from $54,500 for the Space Shuttle to $2,720 for Falcon 9. First-principles thinking did not improve the existing approach but merely showed the existing approach was flawed and had to be recreated from ground truth upward.

In a software context, consider a scenario in which a company pays licensing fees for a proprietary analytics platform to process data. An engineer can challenge this by examining what the platform actually does: It ingests data, runs some statistical computations, and generates reports. Those are the first principles of the problem. These could have been implemented using open source software and some custom programming. The developer developed a proof-of-concept in Python to show how 90% of the solution could be implemented at a fraction of the cost.

As another example, let's say a team was tasked with improving the load time of a web application, which was about 5 seconds (we want to be <1s). Many team members assumed they needed a faster backend or maybe a CDN (analogous to similar past issues). One developer instead started from first principles of page load: bytes delivered, number of network requests, render-blocking operations. They found that it was loading 2 MB worth of images and making dozens of HTTP requests. The core solution involved image compression and bundling, which resulted in a dramatic reduction in loading time to 1.5 seconds without even involving the backend. This was initially ignored by the team, who felt that "our server is slow."

First-Principles Thinking can truly excel in areas of performance and cost optimization because wherever you can express the fundamentals (i.e., time, space, cost), you will achieve clarity that pierces through assumptions.

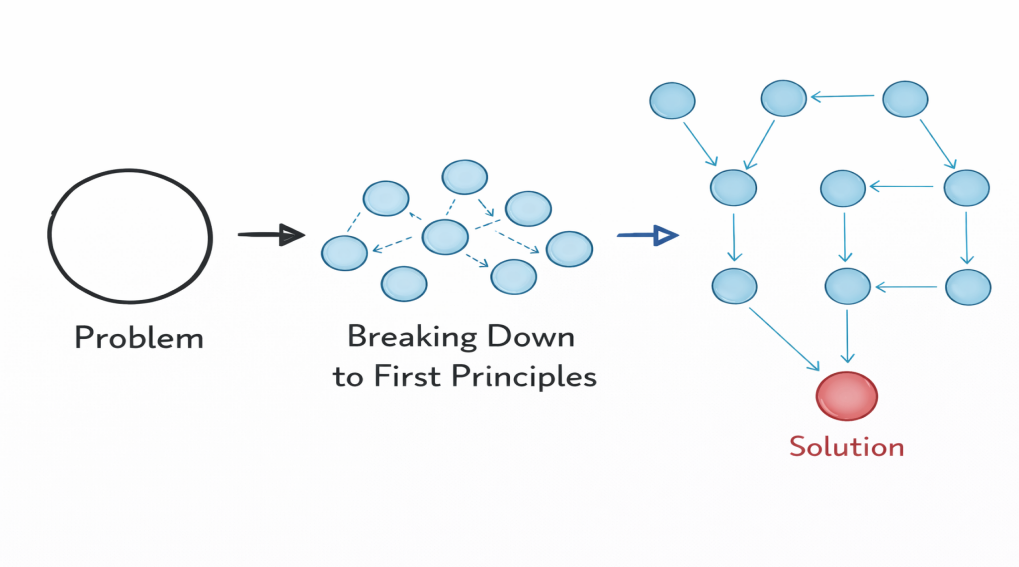

Figure 53.1 **First-Principles Thinking**

Related to

- Occam's Razor
- The KISS Principle
- Gall's Law

Further Reading

- Aristotle. (1984). Posterior analytics (J. Barnes, Ed. & Trans.). In *The complete works of Aristotle: The revised Oxford translation* (Vol. 1). Princeton University Press. (Original work published ca. 350 B.C.E.)

- Pólya, G. (1945). *How to solve it: A new aspect of mathematical method.* Princeton University Press
- Pearl, J., & Mackenzie, D. (2018). *The book of why: The new science of cause and effect.* Basic Books.

OceanofPDF.com

54. Inversion

Inversion means solving a problem by considering the opposite outcome and working backward from it.

Key Takeaways

- *For any goal, also ask the inverted question. If the goal is "deliver the project on time," we should think about what would make us miss the deadline. Making a list of those factors (scope creep, underestimating tasks, etc.) can help us achieve those goals and avoid issues.*
- *Use techniques such as pre-mortems. Our project failed, and we tried to determine what could have caused it. This critical thinking can show risks that optimistic planning overlooks.*
- *In design and testing, we can account for edge cases and use cases of your system. For example, how could a malicious user break this API? Inverting in this way leads to better solutions (adding validation, rate limiting, etc., once you understand how things break).*

Overview

The core idea is that specific insights or solutions become clear when you look at the *problem space* from the opposite angle. In software engineering, one natural application is in *error handling and reliability engineering*. Instead of designing only for the best case (the system handles 1000 requests per second gracefully), you also

deliberately create for the worst case (What if the database goes down? What if latency spikes? What if a bug causes a memory leak?). By thinking of how things can fail, you add features like circuit breakers, retries, or failover strategies, which prevent those failures or at least make the system tolerant to them.

Inversion pairs well with First-Principles Thinking. First, you identify fundamental things that could go wrong (i.e., first principles of failure), and then you address them. It's a form of defensive design or planning that, paradoxically, leads to robust offensive outcomes (success) when we avoid possible problems.

Origins

Inversion is explained by the famous quote from investor Charlie Munger: "All I want to know is where I'm going to die, so I'll never go there." Munger was inspired by a 19th-century mathematician, Carl Gustav Jacobi, who often solved problems by inverting them, and coined the phrase "Invert, always invert."

In philosophy and Stoic practice, we also see a flavor of inversion. The Stoics had an exercise called *premeditatio malorum* (premeditation of evils), which means imagining in advance the worst things that could happen, just to be prepared. This is essentially inversion applied to life.

Examples

A good example of inversion in software engineering is Netflix's Chaos Monkey tool. Usually, engineers try to keep systems running. Chaos Monkey does the opposite. It randomly terminates servers in

production (the worst-case scenario) to test if the system is resilient to such failures. This approach allowed Netflix to build fault-tolerant services that can survive server failures.

In software development, it can be applied by writing a *failing test first* (by using Test-Driven Development). TDD starts by defining the failure (i.e., writing a test for functionality that doesn't exist or is broken) and then writing code to make it pass. This makes us think about how the code could be behaving incorrectly or what a failure is before implementation.

Once, I was a part of a software development team, which used this method before implementing a new payment processing system. Their mindset was no longer "We need to design a dependable system." It was revised to "If one year later the system has catastrophically failed, what happened?" Yes, they actually took a day to think on how bad things can go. A database slow down on Black Friday? A payment gateway going down with no fallback? A bad deploy locking the transactions table during peak hours? Card data leaks via the logs? Reconciliation problems piling up silently all week long?

The team realized that they should add read replicas with failover and also a second payment gateway. In addition, they introduced freezes on deploys during peak hours and automated alerting on reconciliation drift. What happened is that the system handled its first Black Friday without incident, not because the team planned for success, but because they planned for failure.

Another scenario is when an engineering manager might invert a team morale problem. Instead of only asking “How can I make the team happier and more motivated?” they would ask, “What things are making the team demotivated right now?” and find, say, that frequent after-hours deploys are burning people out. By eliminating or reducing those negative factors (perhaps by automating deploys or shifting them to work hours), the team’s morale improves.

Inversion often complements forward-thinking. A balanced approach is to design the best solution you can (forward-reasoning), then invert. Study it to understand how it might fail or when it would no longer be best, and use that insight to refine the solution.

Figure 54.1 **Inversion**

Related to

- First-Principles Thinking

- Murphy’s Law
- Premature Optimization

Further Reading

- Munger, C. T. (2005). *Poor Charlie’s almanack: The wit and wisdom of Charles T. Munger* (P. D. Kaufman, Ed.). Donning Company Publishers.
- Taleb, N. N. (2012). *Antifragile: Things that gain from disorder*. Random House.

OceanofPDF.com

55. The Pareto Principle (80/20 Rule)

80% of the problems result from 20% of the causes.

Key Takeaways

- *Identify the top 20% of factors that contribute to 80% of whatever you care about. This could be 20% of features that account for 80% of usage, or 20% of bugs that cause 80% of crashes. Prioritize your time and resources on those vital few for maximum impact.*
- *Don't spend the same effort on everything. If a particular area of code is rarely run, it probably doesn't need the same optimization or attention as a hotspot.*
- *The ratio is not always exactly 80/20, but the idea is the same. Use profiling, analytics, and data gathering to empirically identify where the imbalances are.*
- *On a personal level, as an engineer or manager, identify which tasks produce most of your value. It might be coding core features or coaching team members.*

Overview

The Pareto Principle is more of an observation than a law, but it's often the 80/20 pattern (or something similar, like 90/10, 70/30) that appears.

In software, people quickly noticed similar patterns. For example, “80% of the time in a program is spent in 20% of the code.” This insight is significant for optimization: Rather than microoptimizing everything, profile to identify the hot 20% of code and optimize that.

The Pareto Principle can influence product development, too. Often, a few core features satisfy the majority of user needs. Thus, when planning a Minimum Viable Product (MVP), teams focus on that core set of features, rather than trying to do everything. It’s an essential check against equal prioritization. Everything is *not* equally important.

Overall, the Pareto Principle encourages the strategic allocation of effort and maximizes outcomes by investing where returns are highest.

Origins

The principle is named after Vilfredo Pareto, an Italian economist who noted in 1906 that about 80% of Italy’s land was owned by 20% of the population.

In the 1940s, quality management pioneer Joseph M. Juran adopted the idea when analyzing industrial defects, calling it the “vital few and trivial many” rule. Juran observed that by focusing on the vital few quality issues, one could improve quality with less effort than trying to tackle all problems at once.

Examples

Microsoft shared a good example in software. They found that around *20% of bugs in Windows and Office were responsible for 80% of all crashes and errors observed.* And that a tiny 1% of bugs caused around 50% of errors. With that insight, Microsoft focused on the critical 1% (and 20%) of bugs to dramatically improve update stability. This is a textbook Pareto fix. Address the small set of high-impact issues to make most users happier quickly.

Another example is a software product team that analyzed usage analytics. They discovered that out of 50 features in their application, only about 5–10 features (i.e., ~10–20%) were used by the majority (80%+) of users. Other app features were rarely used. This guided the team to make the app interface around those core features simpler and remove unused ones.

Technical Debt and code health follow the Pareto pattern. A small part of the codebase drives most defects, slowdowns, and “it’s scary to touch” work. When those parts are also changed often, those slices are your *hot spots.* They have a *high cost of change* (buggy, complex, challenging to test, long review cycles). Instead of spreading refactors evenly, aim debt work at these hot spots first: track commits + incidents to find them, then pay down debt where it compounds. Add tests at the seam, simplify the worst paths, reduce coupling, and tighten interfaces.

Similarly, in web performance, a small number of requests or assets often account for most of the page load time. By fixing those (such

as a single large image or a single long database query), you get the biggest win.

In practice, many teams use Pareto thinking in retrospectives or planning. If a particular type of issue caused most of the delays in the last sprint, address that type (don't sweat the one-off odd matters as much). In maintenance, it's common to find that a small number of modules account for most bugs; those modules likely need refactoring or additional tests.

In essence, applying the Pareto Principle means always asking, "What are the few things making the biggest difference here?" and then focusing on those.

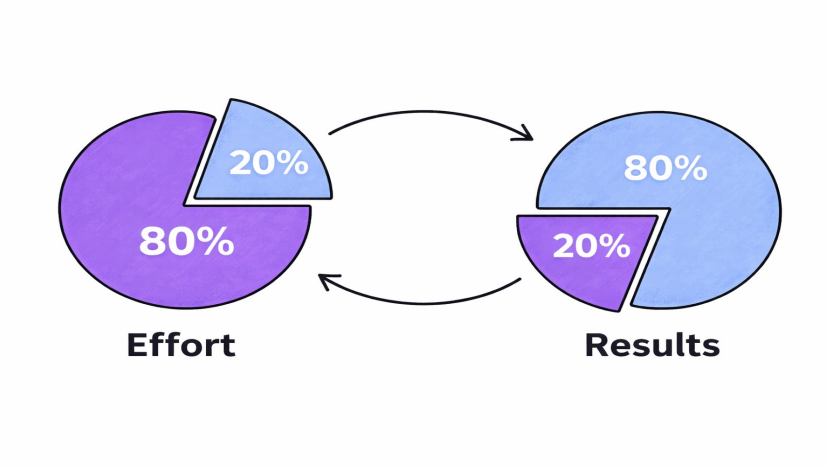

Figure 55.1 **The Pareto Principle**

Related to

- Premature Optimization
- The KISS Principle
- Tesler's Law

Further Reading

- Greene, T. (2002). *Full text of 10/02 Ballmermemo.* eWeek. https://www.eweek.com/enterprise-apps/full-text-of-10-02-ballmer-memo/
- Koch, R. (1997). *The 80/20 principle: The secret to achieving more with less.* Doubleday.
- Pareto, V. (1896–1897). *Cours d'économie politique* (Vols. 1–2). F. Rouge. (Original lectures delivered at the University of Lausanne.)

OceanofPDF.com

56. Cunningham's Law

The best way to get the correct answer on the Internet is not to ask a question but to post the wrong answer.

Key Takeaways

- *People online love correcting errors. If you're stuck on a problem, sometimes proposing a solution (even if it might be wrong) can get you to the right solution faster.*
- *Asking a question might evoke only silence, but confidently stating something (even if incorrect) often provokes a reaction.*
- *Instead of endlessly asking "How should we do X?" propose a draft or prototype. Even if it's not entirely correct, having a concrete starting point often gets more feedback ("No, not like that. Do it this way.").*

Overview

Cunningham's Law arose in the context of online communities, particularly wikis and forums. The underlying observation is that *assertions* often draw more engagement than *questions*.

On Wikipedia (where Ward Cunningham's wiki concepts evolved), if someone posts incorrect information, other editors will likely rush in to fix it.

In software engineering culture, you might have seen a similar pattern on Q&A sites such as Stack Overflow or mailing lists. A user

asks a question and gets a few responses, but if someone posts an answer that's slightly off, others quickly chime in to correct it.

From a knowledge management perspective, people have an inherent desire to demonstrate their knowledge and rectify errors. Cunningham's Law turns that into a strategy for getting information.

Cunningham's Law also explains the value of *interaction* in problem-solving. By putting something (even if flawed) out there, you transform a passive question into an active discussion. It is related to the idea of fail-fast or iterating in public: You put an attempt out and let feedback correct the course.

However, the law's Internet roots also remind us that context matters. On some platforms, posting wrong information can mislead others or harm your credibility.

Origins

Cunningham's Law is named after Ward Cunningham, the American programmer best known for creating the first wiki software. The actual phrasing of the law is often attributed to a colleague of Cunningham, Steven McGeady, who formulated the adage and named it after Cunningham.

According to McGeady, Ward Cunningham gave him advice in the 1980s regarding early Internet forums (Usenet), and McGeady later described it in a comment on a *New York Times* blog in 2010.

The irony is that Ward Cunningham's invention, the wiki, is a platform built precisely on the idea that people will collaboratively correct and

improve content.

The French have a proverb, *Prêcher le faux pour savoir le vrai* ("Preach the false to know the true."), which captures a similar concept.

Examples

An interesting example in practice is when a developer is stuck on how to configure a specific open source library. They've posted a question on a forum and gotten no replies. Instead of using Cunningham's Law, they post: "For anyone else struggling, the solution is to set `ConfigMode` to `False`." If this is wrong, it's almost guaranteed that within a short time, someone (possibly the library maintainer) will reply, "Actually, no, you should leave `ConfigMode` true, it's another setting you need to change." Suddenly, the developer has the correct answer.

The most obvious example of this in the field of software engineering is Stack Overflow. Simply post a question such as "How do I sort a dictionary by value in Python?" and forget about it. The question will likely have zero response after several hours. Everyone assumes that someone else will answer the question. Post a wrong approach, something like, "I am using the following code to sort a dictionary by value: '`for k in sorted(dict.keys())`', and the code only sorts the dictionary by key," and the response will be fast. Anyone who is experienced will not be able to resist a wrong approach. They will correct the wrong code and go on to explain why the wrong code was incorrect, as well as present other approaches.

The reason is practical: An open-ended query forces the answerer to make an assumption about your context, while an incorrect answer at least provides them with something to build from. It helps to limit the problem and understand how you're thinking about it.

Here is another example. Let's say we have a developer who is new to a code base, who is not sure in a team setup about how to implement some feature. Instead of asking everyone about it, they implement it in a way that seems right and submit a pull request. The senior engineers reviewing the PR might say, "Hmm, this approach works but isn't optimal. It would be better to use the existing utility function X or pattern Y." By *doing* and effectively presenting a proposed answer, the new developer gets the team's knowledge and ends up with the right approach.

It's also important to mention that Cunningham's Law works best when people who know the topic well are watching and are willing to correct. If you post a wrong answer in a forum of equally lost people, you might spread misinformation or just get agreement from others who don't know it's bad.

Therefore, it is essential to employ this strategy in the appropriate context, such as a highly trafficked forum comprising experts or among teammates who rigorously review code.

Figure 56.1 **Cunningham's Law**

Related to

- Linus's Law
- The Broken Windows Theory

Further Reading

- Leuf, B., & Cunningham, W. (2001). *The Wiki way: Quick collaboration on the Web*. Addison-Wesley.
- Raymond, E. S. (2001). How to ask questions the smart way. http://www.catb.org/~esr/faqs/smart-questions.html

OceanofPDF.com

Part VIII
Conclusion

OceanofPDF.com

1. When the Laws Collide

You have just read 63 laws and principles that describe how software systems, teams, and decisions behave. It is likely that you have noticed how these laws often contradict one another.

According to Conway's Law, structure follows organization. The Inverse Conway Maneuver, however, says that you can flip this structure around. And both of these are true statements, but for different environments.

There is no law that is applicable to everything. This isn't a flaw in these concepts. And that is the point.

When two laws appear to be at cross purposes, the chances are that the frame from which you're viewing the situation is incorrect. Brooks's Law declares, "Adding people to a late project makes it later." Linus's Law, however, says that "Given enough eyes, all bugs are shallow." The difference? Brooks describes coordination overhead when having deadlines. Linus describes open source development with distributed, asynchronous review. So we have different contexts and different forces.

The question here is to understand which forces matter the most.

Are you 6 months into a rewrite with increasing complexity? Gall's Law matters. Are you defending a fixed deadline with stakeholders pushing for more people? Brooks's Law matters. Do you choose between microservices and a monolith? Here, the CAP Theorem, Fallacies of Distributed Computing, and Hyrum's Law matter.

When you need a decision to make, ask which laws apply. Then ask which ones don't. That second question matters more.

Knuth's Optimization Principle ("Premature Optimization") says that we shouldn't optimize early. But if you're building a high-frequency trading system, "premature" has a different meaning than when you're prototyping some simple project. The principle still applies, but you're just working in a different operating range.

Each law in this book has a context where it dominates and a context where it fades. Learning to read that context is the real skill. The laws don't give you answers. They give you better questions.

2. Using the Laws Together

Knowing individual laws is useful, but the real leverage is combining them.

Most engineering problems don't involve one law. They involve three or four pulling in different directions. A scaling failure (Part V) often traces back to an organizational mismatch (Part II) or a design principle violation (Part VI). The seven categories in this book, Architecture, People, Time, Quality, Scale, Code, and Decision-Making, are lenses, not silos. Real problems cross all of them.

The challenge is to determine which laws to apply and prioritize, and which laws to temporarily put aside. Here are some of the ways the laws can be used as tools for diagnosing and planning.

Diagnosing a Team

The moment we notice that a team is stuck, moving slowly, and burning out, the temptation should not be to apply a fix right away. Instead, a small list of laws should be followed as a diagnostic checklist.

Start with Conway's Law: Does the team structure match the system architecture? If not, communication overhead is quietly draining velocity. Then check Brooks's Law: Has someone recently added headcount, expecting it to speed things up? If so, the ramp-up cost might explain the slowdown better than any process issue.

Take, for example, the Ringelmann Effect. Did the individuals' efforts on the team decrease when it was enlarged? If it is a team of more than eight or nine members, then there could be social loafing or diffusion of responsibility.

Check for Goodhart's Law: Is the team optimizing for a metric that no longer represents the goal? I've seen teams hit their sprint velocity targets every cycle while actual product quality declined. The metric became the target, and the original purpose got lost.

Then ask about Technical Debt, and especially Broken Windows: Has the codebase accumulated enough small quality issues that nobody bothers fixing anything anymore? If developers have stopped caring about code quality, the environment is shaping the behavior, not the individuals.

Note that you don't need to know all 63 laws and principles. Five or six should explain 80% of problems in most problematic teams. A

good place to begin is with the organizational laws (Conway's, Brooks's, Ringelmann Effect), then consider any metric distortion effects (Goodhart's), and finally, consider codebase effects (Broken Windows, Technical Debt).

Evaluating a Proposed Process Change

Before adopting a new process, tool, or practice, run it through a few laws to pressure-test it.

First, let's use Gall's Law as follows: Is this an incremental or a big-bang approach? If someone is going to suggest replacing the entire CI/CD process, then Gall's Law tells us that's probably going to be a failure. However, if someone is going to suggest replacing one part of the process at a time, then the chances are much higher. So, keep things simple and then scale them up.

You can use the KISS Principle as a filter. Does the change make things simpler, or does it just add a new layer of complexity? A process that exists only to control other processes is a red flag. Can you describe the change in a simple way? If not, it's probably too complex to implement.

Next comes the Law of Unintended Consequences. We should consider what second-order effects could be triggered by this change. For instance, if a new rule we create on code reviews that need approvals seems simple, but what happens when one approver is away? This change also could push developers to create small PRs, just to reduce review friction. So, the fix to one problem leads to a new one.

Now, apply Parkinson's Law. Speaking of constraining by using processes, be honest with yourself about whether a given process will, indeed, expand in proportion to any time it will save. I was a member of some teams where estimations took more time than the job itself.

Last but not least, consider Goodhart's Law: "Any metric you measure will come to be gamed." If this process introduces a new metric, will people be optimizing for the metric or for the outcome? Think about what gaming looks like for this metric and whether that gaming would still produce useful work.

Ensuring a Roadmap

We can say that roadmaps tend to be optimistic, but laws can ground them.

When you make a 6-month plan, remember that things usually take more time than you think they will. This is what Hofstadter's Law says: "It always takes longer than you expect, even when you take into account Hofstadter's Law." So if your 6-month roadmap does not have some time built in for unexpected things, it is probably too optimistic. You should always think about the things that can go wrong and add some time to your 6-month roadmap. If you do not do this, your 6-month plan might not work out as you hope.

Then apply the Ninety-Ninety Rule. The first 90% of the work takes 90% of the time, and the last 10% takes the other 90%. If the roadmap shows a feature as "90% done," it's probably halfway done. What is left is probably the integration, edge cases, testing, and documentation.

Check for the Second-System Effect. Is this roadmap essentially a rewrite of something that already works? If so, the team will be tempted to add every feature they wished the first version had. That scope creep is predictable and deadly.

For scaling plans, we can apply Amdahl's Law. It should give us the answer to what's the serial bottleneck. No amount of parallelization can help us if 20% of the work can't be parallelized. If your roadmap depends on scaling a system, identify the part that doesn't scale and plan to improve that first.

When we consider systems that have a longer lifetime, we have to think about Lehman's Laws of Software Evolution. In general, they state that a system must be continually evolved; otherwise, it will become steadily less satisfactory. If we do only new features and no maintenance, the existing system will decay.

Finally, apply Brooks's Law to the staffing plan. If we plan to hire a new person in our team and to be productive in a month or two, challenge that. In reality, I've found that it can take months, not weeks, to make a valuable contribution to an existing codebase.

A Quick Decision Framework

When we have to make an engineering decision, you can use the following three-step process:

- **Name the forces.** Which laws apply here? Write them down. We found that usually three to five are relevant.
- **Rank the constraints.** Which force is the strongest right now? If you need to ship a project in 2 weeks, a deadline constraint

(Hofstadter's Law, Ninety-Ninety Rule) dominates a scaling concern (Amdahl's Law). The ranking can change during the time.

- **Check the opposite.** For each law you rely on, look for where it breaks. If you're adhering to YAGNI as a reason not to implement a feature, consider whether it's one of those rare cases when the cost of implementing it later is higher than the cost of implementing it now.

This approach won't eliminate mistakes, but it compresses the time between "we have a problem" and "we understand why."

3. Why These Laws Last

Software moves fast. Frameworks come and go, and deployment models that dominated 5 years ago are already legacy in some organizations.

But the laws in this book haven't changed much because they explain timeless principles of how organizations, people, and code work.

Brooks's Law was formulated in 1975 and still explains what happens when you add people to a late project today, in 2026. Conway's Law was formulated in 1967 and still explains the architecture of systems we build now. Gall's Law was formulated by a pediatrician writing a book about systems theory in 1975, and that is still forecasting rewrites that fail.

These laws are persistent because they don't deal with technology, they deal with something much more solid: human behavior,

mathematical limits, and organizational physics.

Conway's Law isn't really about software architecture at all. It's actually about how communication boundaries affect the output. The same force operates anywhere people build complex things together. Brooks's Law isn't about programming but about coordination costs in any complex collaborative work. Goodhart's Law isn't about software metrics. It's about what happens whenever measurement replaces judgment.

That is why these laws apply equally whether you are writing a program in Java, deploying services in Kubernetes, or building agents that use AI for power. The tools we use are always changing. The way people work together on Java, Kubernetes, or AI projects does not change.

What has changed is the speed at which these forces compound. Teams are smaller. Release cycles are shorter. Systems are more distributed. The feedback loops are tighter, so the consequences of not paying attention to these laws manifest much faster. A team that violates Gall's Law in a 2-week sprint will feel it within a month, not after a year-long waterfall cycle.

AI-assisted development makes this even more pronounced. When developers can produce code faster, architectural decisions matter more, not less. Generating more code faster doesn't help if Conway's Law means your team structure is fighting your system design. Faster output amplifies the effects of bad structure.

The laws in this book are not about any specific era of software engineering, but about the recurring forces that shape all engineering

work. As long as humans build complex systems in concert under constraints such as time, knowledge, and coordination, these laws will hold.

You won't remember all of them, and you don't have to. Remember five or six in relation to your current problems. Browse through the rest when new problems arise. This book is not intended to be memorized from cover to cover, but to be reviewed selectively when you get stuck.

These patterns came from people who got burned and wrote them down. You'll get burned, too, by things this book doesn't cover, in contexts that don't yet exist. When that happens, write your own law. Add to what's here.

Keep notes as you use the laws. One line per project, incident, or rewrite: which law helped, which law misled, and which condition changed. Over time, you'll build your own version of this catalog, tuned to your domain, your team, and your systems.

That version will be more useful than this one.

If you're an individual contributor, these laws will help you anticipate problems before they happen. If you're a tech lead, they give you vocabulary for trade-off discussions that would otherwise be just opinions. If you're an engineering manager or CTO, they're diagnostic tools for organizational problems that look like technical ones. Regardless of your role, go back to whichever section fits the kind of problem you currently have. The table of contents is organized by problem type, not alphabetically, so you should look there.

OceanofPDF.com

Acknowledgments

This book wouldn't exist without the people who shaped how I think about software, teams, and systems.

Thank you to Dr Rebecca Parsons and Addy Osmani for writing the forewords. Having two people I deeply respect open this book means more than I can properly say here.

Thank you to my colleagues across two decades of building software at EPS, Zühlke Group, 3MD, and many other clients with whom I worked. Working alongside talented engineers, architects, and managers across finance, banking, energy, transportation, insurance, and telecommunications taught me that these laws aren't just theoretical. Every law in this book has a story from a real project behind it. Some of those projects went well, but most of the interesting lessons came from the ones that didn't.

Thank you to the reviewers who read early drafts and made this book better than it would have been on its own: Adam Tornhill, Maarten Dalmijn, Matthias Patzak, Vlad Khononov, Jacqui Read, Thiago Ghisi, Alex Rashkov, Sean Goedecke, Marvin Ferreira, Luca Mezzalira, Owain Lewis, Ryan Murphy, Romano Roth, Annie Vella, David Pereira, Jovan Cukalovic, Fiodar Sazanavets, Uros Popovic, Tosho Trajanov, Maciej Jędrzejewski. Your honest feedback helped me catch errors, make better arguments, and pointed out the places where I was wrong, or not wrong enough.

Also, special thanks to my editor, Lana Todorovic, who brought clarity and precision to a manuscript that needed both. Good editing is

invisible work, and this book is better for every line she improved.

Thank you to my community of over 400,000 engineers, managers, and architects who read my writing through Tech World With Milan. Your questions, pushback, and real-world examples shaped this book more than you know. Writing for practitioners keeps the work honest.

And most of all, thank you to my family. For the evenings and weekends this book consumed. For the patience when I was deep in thought about some law instead of being present. For supporting a project that makes sense only if you believe these ideas are worth sharing.

This book is for all of you.

OceanofPDF.com